마크다운 방언 *ndhmarkdown* 사양서

Nova de Hi

2026년 11월 10일

이 문서는 `ndhmd4tex.py`가 인식하는 markdown → LaTeX 변환 문법을 정리한 참조 가이드다. 표준 Pandoc markdown과 여러 지점에서 다르므로, 특히 **따옴표**, **리스트 이어쓰기**, **설명 리스트(`::`)**, **커스텀 환경(`:::`)** 부분은 꼭 확인하고 쓰는 것이 좋다.

---

1. 실행 방법

```
python3 ndhmd4tex.py 입력.md                    # 입력.tex 생성
python3 ndhmd4tex.py 입력.md -o 출력.tex        # 출력 경로 지정
python3 ndhmd4tex.py 입력.md --config mapping.yaml   # 명령 매핑 override
python3 ndhmd4tex.py 입력.md --setup 바탕.md    # 이 파일의 헤더를 바탕으로 (1.1 참고)
python3 ndhmd4tex.py 입력.md --setup a.md,b.md   # 여러 파일을 순서대로 병합 (21.3 참고)
python3 ndhmd4tex.py --init-setup               # 기본 헤더를 현재 폴더에 꺼내고 끝
python3 ndhmd4tex.py 입력.txt --encoding cp949  # 입력 인코딩 직접 지정
python3 ndhmd4tex.py 입력.md --lint       # 변환하지 않고 마크업 실수만 점검 (1.3 참고)
python3 ndhmd4tex.py 입력.md --fetch-images  # URL 그림을 내려받는다 (15.1 참고)
python3 ndhmd4tex.py 입력.md --pdf      # pdflatex 로 컴파일 후 뷰어로 열기
python3 ndhmd4tex.py 입력.md --pdfxe    # xelatex 로 컴파일
python3 ndhmd4tex.py 입력.md --pdflua   # lualatex 로 컴파일
python3 ndhmd4tex.py 입력.md --pdfxe --brief  # 컴파일러 출력은 숨기고 핵심만
python3 ndhmd4tex.py 입력.md --dump-meta  # 병합된 최종 front matter만 출력하고 종료
python3 ndhmd4tex.py 입력.md --pandoc     # 본문 변환만 pandoc에 맡긴다 (1.4 참고)
python3 ndhmd4tex.py 입력.md --pandoc="--number-sections"   # pandoc 인자와 함께
python3 ndhmd4tex.py 입력.md --mmd        # 본문 변환만 MultiMarkdown에 맡긴다 (1.4.1 참고)
```

`--config`를 생략하면 front matter의 `mapping:` 키(상대경로는 입력 파일 기준)를 찾아 자동으로 사용한다. `--pdf`류 옵션은 `kpsewhich`로 `ksruncltex.lua`를 찾아 `texlua`로 실행하며, front matter의 `compileoption:` 문자열을 셸 인자로 그대로 덧붙인다. 컴파일이 끝나면 플랫폼 기본 뷰어(macOS `open`, Linux `xdg-open`, Windows는 연결 프로그램)로 PDF를 연다. 다른 뷰어를 쓰고 싶으면 환경변수 `NDHMARKDOWN_PDF_VIEWER`에 실행 파일과 인자를 지정한다(예: `okular --unique`). 지정한 것을 찾지 못하거나 실행에 실패하면 경고를 찍고 플랫폼 기본 뷰어로 물러난다.

콘솔 출력은 두 갈래다. `--verbose`(기본값, 굳이 안 써도 된다)는 지금까지 설명한 그대로 — `--pdf`류나 `--latexmk`를 쓰면 그 컴파일러 자신의 로그(실행 명령어, xelatex/latexmk의 진행 상황)가 화면에 그대로 흐른다. **`--brief`를 주면 이 컴파일러 로그를 숨기고, "컴파일합니다: (실제 명령줄)"/"PDF를 엽니다: ..." 같은 핵심 진행 상황만 보여준다** — 컴파일이 실패했을 때만 예외로, 숨겼던 로그를 그제서야 화면에 보여준다("조용하다"와 "실패해도 단서가 없다"는 서로 다른 문제이기 때문이다).

`--nopreview`를 주면 컴파일은 그대로 하되 끝난 뒤 PDF 뷰어를 열지 않는다(`--pdf`/`--pdfxe`/`--pdflua`/`--lat`와 함께 쓴다). 뷰어가 이미 떠 있거나, 스크립트·CI에서 PDF만 만들고 싶을 때 쓴다. `--brief`와 함께 써도 된다.

이 도구가 직접 내는 메시지(오류·경고·안내 전부, `--verbose`/`--brief` 어느 쪽이든)에는 앞에 `[ndhmd]` 가 자동으로 붙는다 — 콘솔 출력을 `grep '\[ndhmd\]'` 등으로 걸러서 이 도구의 진행 상황만 따로 보고 싶을 때를 위한 것이다. 컴파일러 자신의 로그(`--verbose`일 때 그대로 흐르는 것, `--brief`에서 실패 시에만 보여주는 것)에는 이 접두어가 안 붙는다 — "이 도구가 한 말"과 "컴파일러가 한 말"이 항상 구분되게 하려는 것이다. `--help`/`--version` 출력에는 접두어가 없다(표준적인 도움말/버전 형식을 그대로 지키기 위해서다).

입력 파일은 **UTF-8을 먼저 시도하고, 실패하면 `cp949`, `euc-kr`로 물러선다.** 한국어 윈도우에서 만든 `.txt` 원고가 cp949인 경우가 흔하기 때문이다. 물러섰을 때는 경고를 찍으므로, 글자가 깨져 보이면 `--encoding`으로 직접 지정하거나 파일을 UTF-8로 다시 저장하면 된다. 이 규칙은 `{{포함}}` 파일과 `setup:` 파일에도 똑같이 적용된다.

파일 이름을 잘못 적었거나, 폴더를 넘겼거나, PDF처럼 텍스트가 아닌 파일을 넘기면 트레이스백 대신 무엇이 잘못됐는지 알려주고 종료 코드 1로 끝난다.

줄바꿈은 읽을 때 LF로 통일한다. 윈도우 편집기가 만든 CRLF 원고를 그대로 두면 줄 끝의 `\r` 때문에 줄 단위 규칙들이 조용히 빗나간다 — 헤딩의 `{label=...}`이 제목에 글자로 남거나, 코드 펜스의 닫는 "` 줄이 인식되지 않아 그 아래 문서 전체가 코드 블록에 삼켜지는 식이다.

`--dump-meta`는 변환을 하지 않고, `setup:` 체인까지 모두 병합한 최종 front matter를 YAML로 출력하고 끝낸다. 설정 값의 출처가 여러 곳(CLI / 현재 파일 / setup 파일 / 기본값)으로 나뉘므로, "이 폰트 설정이 대체 어디서 왔나"를 확인할 때 쓴다(21장 참고).

`--dump-mapping`은 `mapping:`/`mappingrule:`/`--config`를 전부 병합한 최종 매핑 규칙을 YAML로 출력하고 끝낸다. `--dump-meta`가 보여주는 것은 front matter의 `mapping:` 값(YAML 파일 **경로**)일 뿐, 그 파일들을 실제로 읽어서 병합한 내용까지는 아니다 — 여러 `mapping.yaml`을 나열했을 때 `extra_inline`처럼 이어붙는 키와 그 외 "뒤가 이긴다"는 키가 섞여 있고, 거기에 `mappingrule:`과 `--config`까지 우선순위대로 얹히므로, "지금 이 문서에서 `**굵게**`가 정말 어떤 템플릿으로 나가는지"는 파일들을 눈으로 대조하는 것보다 이렇게 병합 결과를 직접 보는 게 정확하다.

```
python3 ndhmd4tex.py 입력.md --dump-mapping
```

`--dump-meta`와 `--dump-mapping`을 **함께 주면 순서대로(먼저 meta, 다음 mapping) 콘솔에 출력**하고 끝낸다 — 어느 하나만 있어도, 둘 다 있어도 실제 변환은 하지 않는다. 각 출력 끝에는 `====== end of meta =======`/ `====== end of mapping =======` 줄이 붙어서, 둘을 한꺼번에 볼 때 어디서 한쪽이 끝나고 다른 쪽이 시작하는지 한눈에 구분된다.

1.1. 헤더 없는 원고와 기본 헤더 (`--setup`, `--default`)

front matter를 한 줄도 적지 않은 원고도 제대로 조판되게 하는 장치다. 헤더의 바탕이 될 파일을 다음 순서로 찾는다.

1. `--setup 파일` 로 지정한 파일(콤마로 여러 개 나열 가능, 21.3 참고). 문서 자신의 `setup:`보다 **우선한다**(`--config`가 front matter의 `mapping:`을 이기는 것과 같은 원칙).
2. 문서 자신의 `setup:` (21장).
3. `--default 파일`로 지정한 파일, 안 줬으면 `ndhmd-default.nmd` — **현재 작업 폴더**를 먼저 보고, 없으면 **프로그램이 놓인 폴더**를 본다. 배포할 때 스크립트와 함께 딸려 가는 파일이다. PyInstaller로 묶은 실행 파일에서는 찾는 자리가 조금 다르다(바로 아래 참고).
4. 아무것도 없으면 예전처럼 `\documentclass{book}` 한 줄로 떨어진다.

찾은 파일은 `setup:`으로 지정한 것과 똑같이 취급된다. 즉 **문서 자신의 front matter가 그 값을 덮어쓴다.** 강제가 아니라 기본값이다. `--dump-meta`를 쓰면 어떤 파일에서 왔는지 체인으로 보여준다.

```
$ python3 ndhmd4tex.py 헤더없는글.md --dump-meta
# setup 체인: 헤더없는글.md -> /home/nova/ndhmd-default.nmd
class: oblivoir
...
```

`ndhmd-default.nmd`의 본문은 읽지 않는다. front matter만 쓰이므로 아래에 메모를 적어 두어도 된다. 확장자가 `.md`가 아닌 것은 편집기에서 일반 원고와 구분하기 위해서다.

`--default`(축약형 `-d`)는 이 3번 자리의 파일 자체를 임시로 바꿔치기한다. `--setup`과는 성격이 다르다 — `--setup`은 "이번 실행만큼은 무조건 이 헤더를 쓰라"는 최우선 지시라 문서 자신의 `setup:`마저 덮어쓰지만, `--default`는 훨씬 약하다. "아무 헤더도 정하지 않은 문서들에 한해, 어떤 파일을 기본으로 쓸지"만 바꾸는 것이라, 문서가 `setup:`을 스스로 정했다면 여전히 그게 이긴다(2번이 3번보다 우선하는 순서 자체는 그대로다).

```
python3 ndhmd4tex.py 글.md --default 회사양식        # 회사양식.nmd 를 찾는다
python3 ndhmd4tex.py 글.md -d 회사양식.nmd           # 확장자를 이미 적었으면 그대로
```

- 확장자가 없으면 `.nmd`를 자동으로 붙인다. 이미 있으면 그대로 존중한다.
- 찾는 위치는 `ndhmd-default.nmd`를 찾던 곳과 똑같다(작업 폴더 → 스크립트/exe 옆 → 공유 리소스 폴더). 절대경로를 주면 그 경로만 본다.
- `--dump-meta`에도 반영된다 — `--default`로 지정한 파일을 못 찾았으면, 그 사실 자체가 진단 메시지에 나온다(아무 설명 없이 `ndhmd-default.nmd`를 못 찾았다고만 나오면 헷갈리기 때문이다).

`--default NONE` — 문서 구조는 빼고 본문 조각만 만들기

`--default NONE`(정확히 대문자)은 위와 전혀 다른, 훨씬 강한 지시다. "기본 헤더 자리에 아무것도 안 넣는다" 정도가 아니라, **문서를 감싸는 껍데기 자체를 만들지 않고, 마크다운 본문만 LaTeX 조각으로 낸다.** `\documentclass` 도, `\begin{document}`/`\end{document}`도 나오지 않는다.

전형적인 용례: 공저자가 이미 pandoc으로 `main.tex`를 관리하고 있고, 그 파일에 대한 권한은 없이 **본문 내용만** 그 문서에 맞는 형태로 넘겨줘야 할 때. `--config`/`mapping:`으로 그 문서의 관례(제목 레벨, 커스텀 문법 등)에 맞춰 두고 조각만 뽑아 전달하면 된다.

```
python3 ndhmd4tex.py 글.md --default NONE -o 조각.tex
```

```
\section{제목}

본문 \emph{강조}와 \hl{하이라이트}.
```

이 결과물은 **그 자체로는 컴파일되지 않는다** — 감싸는 문서가 없기 때문이다. 다른 문서에 `{{tex:조각.tex}}`로 끼워 넣거나, 직접 손으로 프리앰블을 붙일 때 쓰는 용도다.

무시되는 것은 "문서 구조"뿐이고, "마크업을 어떻게 번역할지"는 그대로 살아 있다. 이 둘은 서로 다른 층이다.

- **무시된다(자동으로)**: `title`, `class`, `preamble`, `author`, `date`, `setup:`, `maketitle`, `tocloft` — 이 키들은 오직 문서 껍데기를 조립하는 코드에서만 읽히므로, 껍데기 자체를 안 만들면 자연히 아무 영향이 없다. front matter를 안 읽는 게 아니라, 여전히 정상적으로 해석하되 (`setup:` 체인 포함) 그 결과 중 이 키들만 쓰일 자리가 없어지는 것이다.
- **여전히 적용된다**: `mapping:`, `mappingrule:`, `toplevel:`, `pandocoption:`, `--config`. 이것들은 "본문의 마크업을 어떻게 번역할지" 를 정하는 것이라, 조각을 끼워 넣을 대상 문서의 관례(제목 레벨, `extra_inline`으로 추가한 커스텀 문법 등)를 맞추려면 여전히 필요하다. `--config`를 주면 평소처럼 그 매핑이 적용된다.
- **`--pdf`/`--pdfxe`/`--pdflua`/`--latexmk`는 전부 무시된다** — 컴파일할 완전한 문서 자체가 없기 때문이다. 경고를 찍고 컴파일 시도 자체를 건너뛴다.
- **`--syncinfo`도 무의미하다** — 컴파일 결과가 없으니 줄 번호를 대응시킬 대상이 없다. 경고를 찍고 `.nmdsync`는 만들지 않는다.
- **`--pandoc`과는 함께 쓸 수 있다.** 본문 변환을 pandoc에 맡기되, 역시 `\documentclass`/`\begin{document}` 없이 본문 조각만 나온다. 다만 `extra_inline` 같은 이 방언 고유의 매핑 규칙은 pandoc 모드 자체의 기존 제약과 마찬가지로 이 조합에서도 적용되지 않는다 — pandoc이 본문 변환 전체를 넘겨받으므로.
- `--dump-meta`를 함께 주면, 해석된 메타 전체를 보여주면서 "문서 구조는 무시되고 마크업 번역 규칙만 실제로 쓰인다"는 안내를 덧붙인다.

실행 파일(.exe)로 배포할 때

PyInstaller로 묶으면 `__file__`이 exe 옆을 가리키지 않는다. 실측하면 이렇다.

빌드 방식	`__file__`	`sys.executable`
`--onefile`	임시 압축 해제 폴더(`Temp\_MEIxxxx`)	exe 자신
`--onedir`	`앱폴더\_internal`	`앱폴더\앱.exe`

`--onefile`의 임시 폴더는 실행이 끝나면 사라지므로 사용자가 거기에 파일을 둘 수조차 없다. 그래서 얼어붙은 상태에서는 **exe가 놓인 폴더**를 본다. 찾는 순서는 이렇게 된다.

1. 현재 작업 폴더
2. `ndhmd4tex.exe` 가 놓인 폴더
3. exe 안에 함께 묶인 자료 폴더(`sys._MEIPASS`)

3번 덕분에 **기본 헤더를 exe 안에 넣어 배포할 수 있다.** 윈도우에서는 이렇게 묶는다(자료 파일의 구분자가 `;`인 것에 주의 — 리눅스·맥은 `:`이다).

```
pyinstaller --add-data="ndhmd-default.nmd;." --onefile ndhmd4tex.py
```

`.spec` 파일을 직접 손볼 때는 아래 한 줄에 해당한다.

```
datas=[('ndhmd-default.nmd', '.')],
```

이러면 사용자가 아무것도 두지 않아도 기본 헤더가 먹고, 옆에 파일을 두면 그것이 우선한다. 프로젝트 폴더에 두면 다시 그것이 우선한다.

`--init-setup` — 묶여 있는 기본 헤더 꺼내기

`--onefile`로 묶으면 기본 헤더가 exe 안에 들어가 사용자가 열어볼 수도, 고칠 수도 없다. 꺼내 놓으라고 있는 것이 `--init-setup`이다.

```
D:\작업> ndhmd4tex.exe --init-setup
D:\작업\ndhmd-default.nmd 를 만들었습니다. (원본: C:\...\Temp\_MEI123456\ndhmd-default.nmd)
이 파일을 고치면 이 폴더의 원고에 그대로 반영됩니다.
```

작업 폴더가 탐색 순서의 맨 앞이므로, 꺼낸 사본이 그 순간부터 exe 안의 것을 가린다. 이미 같은 이름의 파일이 있으면 덮어쓰지 않고 오류로 끝낸다. 입력 파일 없이 이 옵션만 주어도 된다.

기본 헤더를 못 찾았을 때는 `--dump-meta`가 어디를 뒤졌는지 보여준다.

```
$ ndhmd4tex.exe 글.md --dump-meta
# ndhmd-default.nmd 을 찾지 못했습니다. 찾아본 곳:
#   D:\작업\ndhmd-default.nmd
#   C:\Program Files\ndhmd\ndhmd-default.nmd
#   C:\Users\...\Temp\_MEI123456\ndhmd-default.nmd
```

주의할 점 두 가지.

- 이 장치는 헤더가 없는 문서에만 걸리는 것이 아니다. `setup:`이 없는 문서면 **전부** 걸린다. 그래서 작업 폴더에 `ndhmd-default.nmd`를 두면 그 폴더의 모든 원고가 영향을 받는다 — 책 한 권을 한 폴더에서 작업할 때는 이게 바로 원하는 동작이지만, 무심코 둔 파일이 다른 원고까지 바꿔놓을 수도 있다.
- `class`만 덮어쓰고 `classoptions`를 그대로 두면 어긋난다. 기본 헤더가 `chapter` 옵션을 주는데 문서에서 `class: article`로만 바꾸면 `\documentclass[..., chapter]{article}`이 되어 컴파일이 깨진다. 클래스를 바꿀 때는 `classoptions`와 `toplevel`도 함께 손볼 것.

프리앰블은 `preamble:`로 적으면 **갈아치우고**, `preamble+:`로 적으면 기본 헤더의 것 **뒤에 덧붙는다**(21장). 기본 헤더를 살리면서 패키지만 보태고 싶다면 `preamble+:`를 쓴다.

1.2. 설정 파일을 여러 문서가 나눠 쓰기

`mapping.yaml`과 `setup:`용 헤더 파일은 여러 원고가 함께 쓰라고 있는 것이다. 그래서 문서 가까이에서 못 찾으면 **공유 리소스 폴더**를 뒤진다.

찾는 순서는 이렇다. 가까운 것이 언제나 이긴다.

1. 절대경로면 그대로.
2. 문서 가까이 — `setup:`은 그것을 적은 파일 옆, front matter의 `mapping:`은 최상위 입력 파일 옆, CLI의 `--setup`/`--config`는 작업 폴더 기준.
3. 환경변수 `NDHMARKDOWN_RESOURCE_DIRECTORY`. 경로 구분자(윈도우 `;`, 그 외 `:`)로 여러 곳을 늘어놓을 수 있다.
4. 사용자 폴더의 관례적인 자리 — `~/Library/ndhmd4tex/resources` (macOS), `~/ndhmd4tex/resources`, `~/.config/ndhmd4tex/resources`.
5. 프로그램이 놓인 폴더의 `resources/`. 배포판에 예제를 함께 담을 자리다.

```
export NDHMARKDOWN_RESOURCE_DIRECTORY=~/문서/ndhmd-공용
python3 ndhmd4tex.py 원고.md          # setup: basicpreamble.nmd 를 거기서 찾는다
```

- 4번의 세 자리를 플랫폼별로 하나만 보지 않고 **전부** 훑는다. 설정을 여러 기계에서 동기화해 쓰는 경우가 흔한데, 없는 폴더는 어차피 걸러지므로 잃는 것이 없다.
- 못 찾으면 **찾아본 경로를 전부 보여준다.** 탐색 경로가 여럿인 기능에서 "왜 안 찾지"를 추적할 방법이 없으면 곤란하기 때문이다.
- 1.1의 `ndhmd-default.nmd`도 같은 폴더들을 본다(작업 폴더 → 프로그램 폴더 → 리소스 폴더 → 실행 파일 안 사본 순).
- **`{{포함}}`은 이 탐색을 하지 않는다.** 그쪽은 설정이 아니라 원고 내용이라 문서 기준으로만 찾는 것이 옳다.

여러 `mapping.yaml`을 나눠 쓰기

front matter의 `mapping:`은 `setup:`과 마찬가지로 파일 하나뿐 아니라 **리스트**도 받는다. 회사·팀 공통 규칙 파일과 프로젝트 전용 규칙 파일을 나눠 두고, 한 문서에서 둘 다 쓰고 싶을 때를 위한 것이다.

```
mapping: [공통.yaml, 프로젝트전용.yaml]
```

합치는 원칙은 두 갈래로 나뉜다.

- **`extra_inline`을 뺀 나머지 보통의 키는 `setup: [a, b]`와 완전히 같은 원칙 — 뒤에 적은 파일이 이긴다.** `공통.yaml`과 `프로젝트전용.yaml`이 둘 다 `headings:`를 정의했다면, `프로젝트전용.yaml` 쪽이 남는다. 이 프로젝트의 front matter 전반에 걸친 override 규칙(나중/더 구체적인 것이 이긴다)과 일관되게 맞춘 것이다.
- **`extra_inline`만은 예외로, 모든 파일에 걸쳐 그대로 이어붙는다.** 위 원칙(뒤가 이긴다)을 그대로 적용하면 `프로젝트전용.yaml`의 `extra_inline`이 `공통.yaml`의 것을 통째로 지워 버

려서, “여러 파일에 나눠 적은 확장 규칙을 다 함께 쓴다”는 애초의 목적을 못 살린다. 그래서 이 키만 덮어쓰기가 아니라 이어붙이기로 처리한다 — **공통**.yaml에 정의한 규칙과 **프로젝트 전용**.yaml에 정의한 규칙이 서로를 지우지 않고 함께 적용된다.

- environments:(커스텀 환경, 22장)는 이 특별 처리가 따로 필요 없다 — 애초에 리스트가 아니라 {**환경 이름**: **정의**} 딕셔너리라서, 서로 다른 이름을 정의한 파일들은 이미 자연스럽게 다 함께 남고, 같은 이름을 여러 파일이 재정의해도 begin/end 같은 필드 단위로 병합된다 (부분 재정의도 그대로 살아남는다).
- --config(CLI)는 지금은 파일 하나만 받는다. 여러 파일을 나열하고 싶으면 이 리스트 문법은 front matter의 mapping:에서만 쓸 수 있다.
- 파일 목록 중 하나라도 못 찾으면 오류로 멈춘다(어느 파일을 못 찾았는지 명시하고, 찾아본 경로도 함께 보여준다).

하위 폴더까지 뒤지기

mapping:/setup:에 **경로 없이 파일 이름만** 쓰면(deep/mapping.yaml처럼 직접 경로를 적은 게 아니라 그냥 mapping.yaml), 위 4번(사용자 폴더의 관례적인 자리) 안에서 바로 못 찾았을 때 그 폴더 **아래를 재귀적으로도** 뒤진다. resources/**회사용**/mapping.yaml, resources/**개인용**/mapping.yaml처럼 용도별로 폴더를 나눠 정리해 두고도 원고에는 그냥 mapping: mapping.yaml 만 적고 싶을 때를 위한 것이다.

```
~/ndhmd4tex/resources/
├── deep/
│   ├── mapping.yaml          ← 이게 선택된다
│   └── deeper/
│       └── mapping.yaml      ← 이건 후보에서 밀린다
```

- **“먼저 발견되는 것”은 “최대한 상위(얕은) 디렉터리에 있는 것”이다.** 위 예시처럼 deep/과 deep/deeper/에 동명 파일이 둘 다 있으면, 더 얕은 deep/mapping.yaml이 선택된다. 같은 깊이에 후보가 여럿이면 폴더 이름 알파벳 순으로 정하므로, 몇 번을 실행해도 결과가 같다.
- **경로가 이미 적혀 있으면 재귀를 안 한다.** mapping: deep/deeper/ mapping.yaml처럼 이미 상대경로를 명시했다면 그건 “정확히 여기서 찾으라”는 뜻이므로, 재귀로 다른 후보를 끼워 넣지 않는다.
- **문서 옆(near)이 재귀보다 항상 우선한다.** 문서 바로 옆이나 작업 폴더에 같은 이름의 파일이 있으면, 공유 리소스 폴더 안에서 재귀 탐색을 벌이기 전에 이미 그걸로 확정된다.
- 못 찾았을 때의 오류 메시지에는 재귀로 뒤진 경로를 하나하나 다 나열하는 대신 resources/**/mapping.yaml 처럼 와일드카드로 “이 폴더 아래 어딘가”라고만 표시한다 — 폴더가 깊거나 파일이 많으면 경로를 전부 나열하는 게 오히려 안 읽힌다.
- 여러 폴더에 동명 파일이 있어서 어느 것이 선택됐는지 헷갈리면, 아래 --search-resource가 재귀로 찾은 것까지 전부(얕은 순서대로) 보여준다.

1.2.1. `--search-resource` — 어디서 찾는지 직접 확인하기

위 탐색 순서를 일일이 외울 필요 없이, 실제로 어디서 발견되는지 바로 물어볼 수 있다.

```
python3 ndhmd4tex.py --search-resource            # 실행 파일/스크립트 자신의 위치만
python3 ndhmd4tex.py --search-resource article.nmd  # 현재 폴더 + 모든 리소스 폴더
```

- 인자 없이 쓰면 `$0`에 해당하는 것, 즉 지금 실행 중인 스크립트나 실행 파일 자신의 경로를 보여준다. PyInstaller exe에서는 임시 압축 해제 폴더가 아니라 exe 자신의 경로가 나온다.
- **그 뒤에 `ndhmd-default.nmd`가 실제로 어디서 오는지도 함께 보여준다.** 1.1의 탐색 순서(작업 폴더 → 프로그램 폴더 → 리소스 폴더 → exe 안 번들 사본)를 전부 훑어, 발견된 것을 그 순서대로 나열한다. 여러 곳에 있으면 전부 나오고, **맨 위의 것이 실제로 쓰이는 것**이다. exe 안의 번들 사본까지 정확히 잡히므로, 그것이 왜 어느 임시 폴더를 가리키는지도 확인할 수 있다.
- 파일 이름을 주면 **첫 매치에서 멈추지 않고** 현재 폴더와 모든 공유 리소스 폴더를 전부 뒤져 발견된 경로를 빠짐없이 보여준다. 여러 곳에 같은 이름이 있어 어느 것이 실제로 쓰이는지 헷갈릴 때를 위한 것이다 — `setup:`이나 `mapping:`이 조용히 딴 파일을 물어 오는 상황을 확인하는 용도다. **경로 없이 순수 이름만 줬다면 공유 리소스 폴더 아래를 재귀적으로도 뒤져서, 하위 폴더에 있는 동명 파일까지 얕은 순서대로 다 보여준다** — `mapping:`/`setup:`이 실제로 고르는 것(가장 얕은 것)은 이 목록의 어디쯤에 있는지 보면 바로 알 수 있다. **`ndhmd-default.nmd`를 이름으로 주면** 위의 인자 없는 형태와 같은 탐색 순서(1.1 참고)까지 함께 뒤진다 — 이 파일은 스크립트/exe 바로 옆이라는, 공유 리소스 폴더와는 다른 자리에 두는 것이 가장 흔하기 때문이다.
- 못 찾으면 찾아본 경로를 전부 보여주고 종료 코드 1로 끝난다.

1.2.2. `--guide` — 가이드 문서를 바로 열기

`~/ndhmd4tex/guide`(macOS는 `~/Library/ndhmd4tex/guide`, XDG로는 `~/.config/ndhmd4tex/guide`도 함께 본다 — `resources/`와 완전히 같은 3단계 구조를 `guide/`에 적용한 것뿐이다)에 이 가이드나 퀵스타트 같은 문서(`.md`, `.pdf`)를 놔두면, `texdoc`처럼 명령 하나로 PDF를 바로 열 수 있다.

```
python3 ndhmd4tex.py --guide                      # ndhmd4tex_dialect_guide.pdf
python3 ndhmd4tex.py --guide=ndhmd4tex_dialect_guide_en  # 영문판(확장자 자동 보완)
```

- 인자 없이 쓰면 `ndhmd4tex_dialect_guide.pdf`를 연다.
- 확장자를 안 붙이면 `.pdf`를 자동으로 붙인다(`--default`의 `.nmd` 자동 보완과 같은 관례).
- 찾는 순서는 `resource_dirs()`와 같다 — 환경변수 `NDHMARKDOWN_GUIDE_DIRECTORY` → 사용자 폴더의 관례적인 자리 → 프로그램이 놓인 폴더의 `guide/`.
- 못 찾으면 찾아본 경로를 전부 보여준다. 폴더 자체가 하나도 없어도 "어디에 두어야 하는지" 알 수 있도록, 실제 존재 여부와 무관하게 관례적인 후보 경로를 그대로 보여준다.
- PDF 뷰어를 여는 방식은 `--pdf` 계열이 컴파일 후 여는 것과 같다(`NDHMARKDOWN_PDF_VIEWER` 환경변수로 바꿀 수 있다).

1.3. `--lint` — 흔한 마크업 실수 점검

이 방언에는 **오류를 내지 않고 조용히 다른 결과가 나오는** 실수가 몇 가지 있다. 변환은 성공하고 PDF도 나오는데 리스트가 문단으로 뭉개져 있거나 각주가 엉뚱하게 늘어나 있는 식이라, 눈으로 잡기 어렵다. `--lint`는 변환하지 않고 그런 자리만 훑는다.

```
$ python3 ndhmd4tex.py 원고.md --lint
원고.md:12: 오류: 리스트 마커가 문단에 흡수됐습니다. 앞 항목 안에 빈 줄이 있어
리스트가 끊긴 것입니다 - 항목 본문의 빈 줄을 없애십시오.
원고.md:25: 경고: `[^ndh2024x]` 는 정의가 없어 "ndh2024x" 라는 인라인 각주가 됩니다.
원고.md: 오류 2건, 경고 1건
```

점검 항목은 이렇다. 짐작으로 고른 것이 아니라 이 문서를 쓰면서 실제로 밟았던 실수들이다.

등급	항목
오류	front matter 들여쓰기에 탭 — 헤더 전체가 무시됨
오류	front matter의 YAML이 깨짐 — 헤더 전체가 무시됨
오류	리스트 항목 안의 빈 줄로 리스트가 끊김
오류	헤딩 아래에 빈 줄 없이 본문이 붙음
오류	`::: {이름}` — `:::` 와 `{` 사이 공백
오류	`:::` 블록이 닫히지 않음
오류	코드 펜스가 닫히지 않음
오류	각주 정의 식별자에 쓸 수 없는 문자
오류	`{{포함}}` 파일이 없거나 확장자를 뺌
경고	정의 없는 `[^식별자]` — 인라인 각주로 둔갑
경고	산문 속 공백 둘러싼 `::` — 설명 리스트로 오인 (중괄호형 `{용어} :: 설명`은 대상이 아님)
경고	표처럼 보이는데 둘째 줄이 정렬 구분선이 아님
오류	grid 표를 해석할 수 없음(모서리·세로선 위치 어긋남)
경고	열린 적 없는 `:::` 닫는 줄

코드 펜스 안과 인라인 코드 스팬 안은 검사하지 않는다. 문법을 예시로 적어둔 것을 실수로 오인하지 않기 위해서다. 오류가 하나라도 있으면 **종료 코드 1**로 끝나므로 Makefile이나 스크립트에서 검사 단계로 끼워 넣을 수 있다.

맨 앞 두 항목이 특히 중요하다. YAML이 깨지면 `class`도 `setup`도 `mapping`도 전부 먹지 않는데, 변환은 그대로 성공하므로 알아채기 어렵다. **LaTeX 편집기에서 프리앰블을 복사해 붙일 때 딸려 오는 탭**이 흔한 원인이다 — 그쪽에서는 탭이 그냥 공백이라 아무 표시가 나지 않는다.

`{{포함}}` 하위 파일까지 함께 검사한다(20장과 같이 한 단계). 단행본이라면 마스터 파일은 `{{...}}` 줄만 늘어놓은 껍데기이고 본문은 전부 하위 파일에 있으므로, 거기를 안 보면 검사할 것이 남지 않는다.

```
$ python3 ndhmd4tex.py 책.md --lint
parts/ch1.md:6: 오류: 리스트 마커가 문단에 흡수됐습니다. ...
parts/ch2.md:4: 경고: `[^ndh2024x]` 는 정의가 없어 ...
책.md (+포함 2개): 오류 1건, 경고 1건
```

병합해서 한 덩어리로 보지는 않는다. 그러면 줄 번호가 어느 파일 몇 행인지 알 수 없게 되기 때문이다. 파일마다 따로 보되 **각주 정의만은 전체에서 모아 공유한다** — 참조는 마스터에, 정의는 하위 파일에 두는 일이 흔한데 따로 검사하면 그것이 전부 오탐이 된다.

이 점검은 변환을 막지 않는다. 여기 걸리는 것들은 대부분 변환 자체는 성공하는 입력이므로, 고칠지 말지는 저자가 판단한다.

1.4. `--pandoc` — 본문만 pandoc에 맡기기

이 방언 대신 **pandoc의 마크다운**으로 본문을 쓰고 싶을 때 쓴다. 나머지는 그대로 이 스크립트가 담당하므로, 결과적으로 **트랜스클루전이 되는 pandoc LaTeX 처리기**가 된다. 컴파일과 뷰어 열기도 평소와 똑같이 `--pdfxe` 등으로 이어서 하면 된다.

```
python3 ndhmd4tex.py 책.md --pandoc --pdfxe
```

처리 순서는 이렇다.

1. front matter를 떼어내 보관한다. **pandoc에는 본문만 넘긴다** — 넘기면 pandoc이 제 나름의 메타데이터로 해석해서 `preamble`이나 `setup:` 체계와 충돌한다.
2. `{{파일}}` 트랜스클루전(20장)을 **먼저** 해석해 통짜 본문으로 만든다. pandoc이 모르는 문법이라 우리가 병합해 줘야 한다. 서브 파일의 front matter는 평소처럼 떨어져 나간다.
3. 본문을 pandoc에 표준입력으로 넘겨 LaTeX 조각을 받는다. `--standalone`을 쓰지 않으므로 `\documentclass`나 `\begin{document}`는 나오지 않는다.
4. 보관해둔 front matter로 프리앰블을 만들어 감싼다. `class`, `classoptions`, `preamble`, `title`, `author`, `date`, `setup:` 상속까지 평소와 완전히 동일하게 동작한다.

`toplevel:`은 pandoc에도 전달된다. `part` / `chapter` / `section` 중 하나면 `--top-level-division`으로 넘어간다. 이게 없으면 `book` 클래스에서도 `# 제목`이 `\section`으로 나가버린다. `subsection` 이하는 pandoc에 대응물이 없어 무시된다.

pandoc에 추가 인자를 주는 방법은 두 가지다. 둘 다 주면 front matter 쪽이 먼저, CLI 쪽이 뒤에 붙는다.

```
# front matter
pandocoption: -f markdown-auto_identifiers --wrap=none
```

```
python3 ndhmd4tex.py 책.md --pandoc="--listings --number-sections"
```

CLI에서는 **반드시 `=`로 붙여 써야 한다.** `--pandoc "--listings"`처럼 띄어 쓰면 값이 `-`로 시작하는 탓에 argparse가 그것을 다음 옵션으로 오해한다.

무시되는 옵션

프리앰블과 `\documentclass`는 이 모드에서 front matter가 정한다는 것이 전제다. 그것을 건드리려는 pandoc 옵션은 경고와 함께 버린다.

무시하는 옵션	이유
`-s`, `--standalone`	pandoc이 제 `\documentclass`를 붙인 완성 문서를 내놓는다
`--template`	위와 같다 — 지정하면 standalone이 켜진다
`-H`, `--include-in-header`	프리앰블 삽입. 이것도 standalone을 켠다
`-B`, `-A`, `--include-before-body`, `--include-after-body`	마찬가지로 standalone을 켠다
`-o`, `--output`	결과가 엉뚱한 파일로 새어나간다
`-t`, `--to`, `-w`, `--write`	출력이 LaTeX이 아니게 된다
`--pdf-engine`	`-o`와 함께일 때만 의미가 있다
`--self-contained`, `--embed-resources`	LaTeX 출력과 무관하다

`-H`나 `-B`가 목록에 있는 것이 뜻밖일 수 있다. 이름만 보면 조각 모드에서도 동작할 것 같지만, pandoc은 이 옵션들을 받으면 **암묵적으로 standalone을 켠다.** 그대로 두면 `\documentclass`가 우리 문서 안에 한 번 더 박힌다.

반대로 `--toc`, `-N`/`--number-sections`, `-V`, `--listings`, `--metadata`, `--top-level-division`, `--wrap`, `-f`는 그대로 통과한다. 프리앰블에 넣을 것이 있다면 pandoc이 아니라 front matter의 `preamble:`에 적으면 된다.

`-f markdown-auto_identifiers`를 권할 만하다. pandoc은 기본적으로 헤딩마다 `\hypertarget{...}`과 `\label{...}`을 붙이는데, 한글 제목이면 `uxbcf8uxbb38` 같은 알아볼 수 없는 라벨이 되고 **`preamble`에 `hyperref`가 없으면 컴파일이 깨진다**. 끄면 `\chapter{본문 장}`처럼 깔끔하게 나온다.

이 모드에서는 이 문서가 설명하는 방언 문법(`:::` 환경, `[[색인]]`, `[#라벨]`, `showexam`, 조사 매크로 등)이 **하나도 동작하지 않는다.** 본문은 전적으로 pandoc의 문법이다. 살아 있는 것은 front matter와 `{{파일}}` 둘뿐이다.

1.4.1. `--mmd` — 본문만 MultiMarkdown에 맡기기

`--pandoc`과 같은 틀이다(0.20.2에서 추가). front matter(`---` YAML)는 이 스크립트가 읽고, `{{파일}}` 트랜스클루전도 먼저 풀어서 통짜 본문으로 만든 뒤 **본문만** `multimarkdown -t latex`에 넘긴다. 돌아온 LaTeX 조각을 front matter로 만든 프리앰블로 감싼다. 실행 파일은 PATH에서 `multimarkdown`을 먼저, 없으면 `mmd`를 찾는다.

```
python3 ndhmd4tex.py 옛문서.md --mmd --pdfxe
python3 ndhmd4tex.py 옛문서.md --mmd="--smart"
```

추가 인자는 `mmdoption:` front matter 키로도 줄 수 있고, 둘 다 있으면 CLI가 나중에 붙는다. CLI에서는 `=`로 붙여 쓴다. 출력 형식과 조각 모드를 바꾸는 `-t`/`--to`, `-o`/`--output`, `-f`/`--full`, `-s`/`--snippet`, `-b`/`--batch`, `-e`/`--extract`는 경고하고 버린다. `--pandoc`과는 함께 쓸 수 없다.

헤딩 단계 — `toplevel`과 `Base Header Level`. mmd는 헤딩 단계를 문서 맨 앞 메타데이터 블록의 `Base Header Level: N`으로만 정한다. 이 스크립트는 다음 순서로 값을 정한다.

1. front matter의 `toplevel:`(`part`/`chapter`/`section`)이 있으면 그것이 이긴다.

2. 없으면 front matter의 `baseheaderlevel:`(정수)을 쓴다.
3. 둘 다 없고 본문 맨 앞 메타데이터 블록에 `Base Header Level`이 있으면 손대지 않는다. 아무것도 없으면 mmd 기본값이다.

`toplevel`을 N으로 바꾸는 표는 코드에 박아 두지 않는다. mmd 버전마다 `#`이 `\part`인지 `\chapter`인지가 다를 수 있어서, 시작할 때 설치된 mmd에게 `# ~ ######`을 한 번 돌려 보고 실제로 나온 명령으로 N을 계산한다. 탐침이 실패하면 경고하고 `part=1, chapter=2, section=3` 표를 쓴다. 본문의 메타데이터 블록에 이미 `Base Header Level`이 있는데 `toplevel`과 다르면 경고하고 `toplevel`에 맞춰 그 줄을 고친다. 블록이 있고 그 줄이 없으면 블록 끝에 한 줄을 더하고, 블록이 없으면 앞에 새 블록을 붙인다.

주의할 점은 다음과 같다.

- mmd의 `Title`, `Author`, `Date`는 **읽어 오지 않는다.** mmd는 메타데이터를 `---`로 구분하지 않고 첫 줄이 **키**`:` **값** 꼴이면 블록으로 보기 때문에, 본문에 남은 그 줄이 front matter의 것인지 mmd 쪽 것인지 이 스크립트가 가릴 수 없다. 제목과 저자는 front matter에 적는다.
- mmd의 `{{TOC}}`는 파일이 아니라 목차 지시자라서 트랜스클루전 해석에서 빼고 그대로 mmd에 넘긴다.
- mmd가 LaTeX 조각 대신 완성 문서를 내놓으면(출력의 첫 의미 있는 줄이 `\documentclass`나 `\input`일 때만 그렇게 본다) `\begin{document}` 안쪽만 취하고 경고한다. 본문에 `\begin{document}`가 verbatim 예제로 들어 있어도 오인하지 않는다.
- front matter의 YAML이 깨지면(예: **키**`:` 뒤의 탭) 그 블록은 본문으로 취급되어 mmd가 직접 읽는다. 본문 맨 앞의 `---`로 감싼 블록 안에 `Base Header Level`이 있어도 위의 규칙대로 찾아서 처리한다.
- mmd가 내는 LaTeX는 `\href`, `tabulary`, `longtable` 같은 것을 쓴다. 필요한 패키지는 front matter의 `preamble:`이 책임진다.
- 이 방언의 기능(`||`, grid table, `showexam`, `notes:`, `:::` 등)은 이 모드에서 작동하지 않는다. 본문 문법은 mmd의 것이다. `--syncinfo`와 `--lint`도 마찬가지로 쓸 수 없다. `.tex` 맨 앞 출처 주석 끝에는 `with multimarkdown syntax`가 붙는다.

1.5. `--syncinfo` — PDF에서 원본 md로 역방향 검색

`--syncinfo`를 주면 변환과 동시에 `{`**출력**`}.nmdsync` 파일을 함께 만든다. `ndhmdsync.py`(별도로 배포되는 짝 스크립트)와 함께 쓰면, 컴파일된 PDF의 특정 위치를 클릭했을 때 원본 `.md` 파일의 해당 위치가 편집기에서 곧바로 열리는 사슬을 완성할 수 있다.

```
PDF 클릭 --[SyncTeX]--> tex 줄 --[.nmdsync]--> md 줄 --[code --goto]--> VS Code

python3 ndhmd4tex.py 책.md --syncinfo --pdfxe
```

- **synctex 옵션을 직접 챙길 필요가 없다.** `--syncinfo`를 주면 컴파일 명령에 이 옵션을 자동으로 덧붙인다 — `ksruncltex`(`compileoption:`) 에는 `--synctex=1`(대시 둘, `ksruncltex`의 관례), `latexmk` (`latexmkoption:`)에는 `-synctex=1`(대시 하나, `latexmk`의 정식 문

법)로 각자 맞춰서 붙인다. `compileoption:`/`latexmkoption:`에 이미 `-synctex=...`나 `--synctex=...`를 넣어 뒀다면(대시 개수 무관) 그 값을 그대로 존중하고 손대지 않는다. `-synctex=1`은 "PDF의 이 위치 → tex 파일의 이 줄"을 해결해 주는 LaTeX 엔진 자체 기능이고, `--syncinfo`는 그 뒤를 잇는 "tex 파일의 이 줄 → 원본 md의 이 줄"을 담당한다. 둘을 이어 쓰면 PDF에서 markdown까지 한 번에 갈 수 있다.

- PDF 뷰어(SumatraPDF, Skim, Okular 등)의 "역방향 검색" 명령에 `ndhmdsync.py`를 등록해 두면 이 사슬이 자동으로 완성된다. 기본 에디터는 VS Code, 기본 뷰어는 실행 중인 OS에 맞춰 자동 선택되지만, 다른 에디터·뷰어를 쓰거나 표준 위치가 아닌 곳에 설치했다면 `ndhmdsync.ini` 설정 파일로 바꿀 수 있다(`ndhmdsync.ini.example` 참고). 자세한 사용법은 `ndhmdsync.py --help` 참고.
- **`.nmdsync` 없이 `.synctex.gz`/`.synctex`만 있는 문서(즉 이 `ndhmd4tex.py`가 아니라 손으로 쓴 LaTeX을 곧장 컴파일한 문서)도 같은 역방향 검색 설정으로 다룰 수 있다.** `ndhmdsync.py`는 `.nmdsync` 를 못 찾으면 `.synctex.gz`/`.synctex`가 있는지 확인해서, 있으면 markdown 역추적은 포기하고 `.tex` 파일 자신을 그 줄에서 연다 — `ndhmdsync.ini`의 `[TEXEDIT]` 섹션에 `command`를 정해 두면 그 명령 으로(예: `texworks -p=%l "%f"`), 안 정해 뒀으면 OS가 `.tex` 확장자에 연결해 둔 기본 프로그램으로 그냥 연다(줄 이동은 안 되지만 최소한 파일은 열린다). PDF 뷰어의 역방향 검색 명령을 한 번만 등록해 두면 markdown 기반 문서든 순정 LaTeX 문서든 같은 설정으로 둘 다 다뤄진다는 뜻이다.
- **정밀도는 블록 단위다.** 문단·리스트 항목·표·환경 하나하나가 원본의 몇 번째 몇 번째 줄에서 왔는지는 정확하지만, 그 안에서 정확히 어느 낱말이었는지까지는 추적하지 않는다 — PDF에서 대충 클릭해도 편집기에서 근처로 이동한다는, 이 기능의 목적에는 이 정밀도로 충분하다.
- **`{{파일}}`을 쓰는 문서도 지원한다.** 이 방언 자체가 파일 포함을 1단계까지만(재귀 없이) 허용하도록 설계돼 있는 덕분에, 병합된 본문의 각 줄이 어느 파일의 몇 번째 줄에서 왔는지 정확히 추적할 수 있다. `.nmdsync`의 각 항목은 자기 소스 파일을 따로 가지므로, `ndhmdsync.py` 쪽은 손댈 것이 없다 — 애초에 여러 소스 파일이 섞인 `.nmdsync`를 다루도록 만들어져 있었다.
  - 2단계 이상 중첩된 `{{...}}`는 이 방언의 기존 규칙 그대로 재귀 해석되지 않고 리터럴 텍스트로 남으며, sync 정보도 정확히 그 지점(중첩을 시도한 파일의 그 줄)을 가리킨다.
  - **알려진 제약**은 있었지만 `ndhmain:` front matter 키로 해결할 수 있다(아래 참고).
- **`--pandoc`, `--mmd`와 함께 쓸 수 없다.** 본문 변환을 pandoc/mmd 서브프로세스에 통째로 맡기므로, 원고의 몇 줄이 결과의 몇 줄이 됐는지 이 스크립트가 알 도리가 없다. 이 경우도 경고 후 `.nmdsync` 없이 변환만 진행한다.
- 각주 **정의**(`[^id]: ...`) 줄은 sync 대상에 없다 — 그 내용은 정의된 자리가 아니라 **참조된 자리**(`[^id]`)에 삽입되므로, 정의 줄 자체는 출력 어디에도 대응하는 곳이 없다. 참조 지점을 기준으로 찾는 편이 어차피 더 쓸모 있다.

포함된 서브파일을 직접 열어 놓고 `ndhmdsync.py`로 정방향 검색을 하면, 출력 `.tex`/`.pdf` 경로의 기본 유추(서브파일과 같은 이름으로 찾는다)가 맞지 않는다 — 서브파일은 그 자체로 컴파일

되는 게 아니라 최상위 파일에 흡수되므로 당연한 문제다. 서브파일이 자기가 어느 최상위 문서에 흡수되는지 스스로 알 도리가 없으니, 그 관계를 서브파일 자신의 front matter에 직접 적어 두는 방식으로 해결한다.

```
---
ndhmain: main.md
---
```

`ndhmd4tex.py`는 이 키를 전혀 안 읽는다(`{{포함}}`될 때 서브파일의 front matter 전체가 통째로 버려지므로 컴파일에는 아무 영향이 없다). 순전히 `ndhmdsync.py`가 정방향 검색에서 기본 출력 경로를 유추할 때만 참고하는 힌트다. 값이 없거나 자기 자신을 가리키면(identity) 평소처럼 그 파일 자신을 최상위 문서로 본다. `--tex`/`--pdf`/`--sync-file`을 명시적으로 주면 이 힌트보다 항상 우선한다.

값은 상대경로로 적어도 된다. `main.md`가 `{{sub/ch1.md}}`처럼 서브폴더 안의 파일을 포함하는 흔한 구성이라면, `ch1.md` 쪽은 자기 자신을 기준으로 한 상대경로를 쓰면 된다.

```
---
ndhmain: ../main.md
---
```

`ndhmdsync.py`를 어느 작업 폴더에서 실행하든(`sub/` 안에서든, 완전히 다른 곳에서든, 절대경로로 부르든) 이 상대경로는 `ch1.md` 자신이 실제로 있는 위치를 기준으로 풀려서 언제나 정확히 `main.md`(그리고 그 옆의 `main.nmdsync`)를 찾아간다.

1.6. `--latexmk` — ksruncltex 대신 표준 도구로 컴파일하기

`--pdf`/`--pdfxe`/`--pdflua`는 이 스크립트 저자가 만든 `ksruncltex`를 통해 컴파일한다. `ksruncltex`가 없거나, 이미 `latexmk`를 쓰고 있어서 그대로 쓰고 싶다면 `--latexmk`를 대신 쓴다.

```
python3 ndhmd4tex.py 책.md --latexmk
```

아무 값도 안 주면 `latexmk -xelatex -interaction=nonstopmode`가 기본으로 실행된다(이 방언이 한글을 다루므로 xelatex을 기본 엔진으로 삼는다).

`latexmk`는 상황(bibtex냐 biber냐, makeindex냐 xindy냐, synctex을 켤지 등)에 따라 조합이 달라지는 도구라, `ksruncltex`의 `compileoption:` 한 줄처럼 무조건 하나로 뭉뚱그릴 수 없다. 그래서 이 스크립트는 그 조합을 대신 판단하지 않는다 — `latexmk` 자신의 옵션 문자열을 그대로 받아서 검증 없이 그대로 넘길 뿐이다.

```
python3 ndhmd4tex.py 책.md --latexmk="--synctex=1 --xelatex"
# front matter 에 적어 두면 --latexmk 만 쳐도 매번 적용된다
latexmkoption: "-xelatex -synctex=1"
```

- CLI에서는 **반드시 `=`로 붙여 써야 한다** — `--pandoc`과 같은 이유로, 값이 `-`로 시작하면 `argparse`가 다음 옵션으로 오해한다.
- front matter의 `latexmkoption:`이 먼저, CLI의 `--latexmk="..."` 값이 나중에 이어붙는다(`pandocoption:`/`--pandoc`과 같은 순서 규칙). 아무 옵션도 안 주면 그때만 `-xelatex`

기본값이 붙는다 — 하나라도 주면 그 기본값은 아예 끼워 넣지 않는다(원고 작성자가 고른 엔진과 충돌하는 것을 막기 위해서다).

- `-interaction=nonstopmode`는 옵션을 뭘 줬든 항상 추가로 붙는다 — 없으면 오류가 났을 때 `*latex` 엔진이 터미널 입력을 기다리며 멈춰 버리기 때문이다.
- `latexmk`는 대시 한 개(`-xelatex`)가 정식 문법이지만, 대시 두 개 (`--xelatex`)도 관대하게 받아들인다(Perl의 `Getopt::Long` 특유의 동작 — 실측으로 확인했다). 다만 **`biber`용 옵션은 애초에 없다** — `-biber`도 `--biber`도 `latexmk`가 모르는 옵션이라며 오류를 낸다. `biblatex`이 `.bcf` 파일을 만들어 두면 `latexmk`가 그 존재만 보고 알아서 `bibtex` 대신 `biber`를 쓴다.
- 잘못된 옵션을 주면 `latexmk` 자신이 명확한 오류를 낸다 — 이 스크립트가 옵션을 대신 검증하거나 고쳐 주지 않는다. `ksruncltex`의 `compileoption:`과 같은 철학이다.
- `--pdf`/`--pdfxe`/`--pdflua`/`--latexmk`는 서로 배타적이다(동시에 쓸 수 없다).

front matter에 `latexmkoption:`은 있고 `compileoption:`은 없으면, `--pdf`/`--pdfxe`/`--pdflua`만 쳐도 `--latexmk`를 안 붙여도 자동으로 `latexmk`가 불린다. 원고 자체가 "나는 `latexmk`로 컴파일한다"는 뜻을 담게 되는 것이다 — 엔진은 명령줄에서 고른 그대로 따라가고(`--pdfxe` → `-pdfxe`, `--pdf` → `-pdf`, `--pdflua` → `-pdflua`), 그 뒤에 `latexmkoption:`의 값이 그대로 이어붙는다.

```
latexmkoption: "-synctex=1"
```

```
python3 ndhmd4tex.py 책.md --pdfxe
# 위 front matter가 있으면 실제로는 다음과 같이 실행된다:
#   latexmk -pdfxe -synctex=1 -interaction=nonstopmode 책.tex
```

이 절의 `-synctex=1` 예시들은 **`--syncinfo` 없이 SyncTeX만 켜고 싶을 때** 여전히 유효하다. `--syncinfo`를 함께 쓴다면 이렇게 직접 안 넣어도 된다 — 1.5절에서 설명하듯 자동으로 붙는다(이미 넣어 뒀다면 그 값을 그대로 쓴다).

`compileoption:`이 있으면(값이 빈 문자열이어도 **키가 존재하기만 하면**) 이 자동 전환은 일어나지 않는다 — 그 원고는 여전히 `ksruncltex`를 쓴다는 뜻으로 해석한다. 둘 다 없는 원고는 당연히 지금까지처럼 `ksruncltex` 그대로다. `--latexmk`를 명시적으로 쳤을 때(값을 주든 안 주든)의 동작은 이 자동 전환과 무관하게 위에서 설명한 그대로다.

---

2. Front matter

파일 맨 앞에 `---`로 감싼 YAML 블록을 쓴다.

```
---
title: "문서 제목"
author: "저자명"
date: "\today"          # 생략 시 기본값이 \today, 빈 문자열이면 \date 자체를 생략
class: article           # 기본값 book
classoption: "11pt,a4paper"   # classoptions 도 동의어로 허용
```

```
toplevel: section         # 아래 "3. 헤딩" 참고
mapping: mymap.yaml       # --config 대신 여기서 지정해도 됨
setup: mymaster.md        # 다른 파일의 헤더를 물려받는다 (21장 참고)
compileoption: "-shell-escape"   # --pdf 계열 컴파일 시 추가 인자
beforestart: |            # \documentclass 보다 앞에 놓일 것
  \DocumentMetadata{lang=ko}
preamble: |
  \usepackage{kotex}
  \usepackage{amsmath}
theme: dark               # 모르는 키 -> \def\NMtheme{dark}
---
```

author 리스트 예시:

```
author:
  - Nova de Hi
  - A. U. Thor^[thor's affiliation]
```

```
\author{Nova de Hi \and A. U. Thor\thanks{thor's affiliation}}
```

- `title` / `author` / `date`가 있으면 각각 `\title{}` / `\author{}` / `\date{}`로 들어간다. 키가 없으면 해당 명령 자체가 생략된다. `date`는 키가 없으면 `\date`를 내보내지 않아 클래스의 기본(보통 `\today`)을 따르고, **키는 있는데 값이 비어 있으면**(`date:` 또는 `date: ""`) `\date{}`를 내보내 날짜를 지운다.
- 생성된 `.tex`의 **첫 줄**에는 `% converted from ndhmd by ndhmd4tex version 0.19.1` 같은 출처 주석이 붙는다(`--pandoc`이면 끝에 `with pandoc syntax` 추가, 날짜·경로는 넣지 않아 같은 입력은 늘 같은 출력). `--default NONE`(본문만 출력)에는 붙지 않는다.
- **`author`는 리스트도 받는다.** 여러 저자를 `\author{A \and B}` 형태로 나열하는 LaTeX 관례를 그대로 따라, 항목을 `\and`로 이어붙인다. 각 항목 안의 `^[...]`(16장의 인라인 각주와 같은 문법)는 `\footnote`가 아니라 `\thanks`로 바꾼다 — 소속·연구비 등을 표시할 때 title page에서 쓰는 LaTeX 관례다. `title`과 마찬가지로 `author`도 raw 텍스트로 다룬다 — 이 `^[...]` 치환과 리스트 이어붙이기 말고는 다른 마크다운 서식을 처리하지 않는다(아래 예시 참고).
- `preamble`은 `\documentclass` 바로 다음, `\begin{document}` 이전에 raw로 삽입된다. `kotex`, `amsmath`, `\newtheorem` 등 필요한 패키지·정의는 여기에 쓴다.
- `beforestart`는 **`\documentclass`보다 앞에** 놓인다. 형식은 `preamble`과 같다. `\DocumentMetadata`나 `\PassOptionsToPackage`처럼 클래스 로딩 전에 와야 뜻이 있는 명령을 위한 자리다. `beforestart+`로 상속받아 덧붙일 수도 있다.
- **`preamble`과 `beforestart`의 내용은 손대지 않고 그대로 나간다.** 이스케이프도 인라인 서식 처리도 하지 않으므로 expl3 문법을 그대로 쓸 수 있다.

```yaml
preamble: |
  \ExplSyntaxOn
  \tl_set:Nn \l_tmpa_tl { my string }
  \ExplSyntaxOff
```

밑줄이 `\_`로 바뀌는 것은 **본문**에 적었을 때다. 본문에서 expl3 코드가 필요하면 7.1의 인라인 raw 스팬이나 10장의 `raw` 코드 펜스로 감싼다 — `` `\ExplSyntaxOn\tl_use:N \l_tmpa_tl\ExplSyntaxOff`{=latex} `` 처럼.

- **`graphicspath`는 `preamble`의 맨 끝에** 다음 두 줄을 붙인다.

```yaml
graphicspath: ["./","figures/"]
```

```
\IfPackageLoadedTF{graphicx}{}{\usepackage{graphicx}}
\graphicspath{{./}{figures/}}
```

`graphicx`가 이미 로드돼 있으면(원고 작성자의 `preamble`이 직접 불렀든, 다른 패키지가 딸려 불렀든) 다시 부르지 않는다 — `\IfPackageLoadedTF`는 별도 패키지 없이도 쓸 수 있는 LaTeX2e 커널 내장 명령이다. 문자열 하나만 줘도 된다(`graphicspath: "figures/"`). 경로 문자열 자체는 가공 없이 그대로 `{}`로 감싸 나간다 — `raw=` 계열 속성들과 같은 태도로, 이 방언이 LaTeX 경로 문법을 대신 판단하려 들지 않는다.

2.1. YAML 값에 백슬래시를 쓸 때

`\`는 YAML의 **따옴표 안에서만** 이스케이프 문자다. 그래서 같은 값을 두 가지로 쓸 수 있다.

표기	결과
`date: \today`	`\today` — 따옴표 없이 쓰면 그대로
`date: "\\today"`	`\today` — 큰따옴표 안에서는 `\\`로 겹쳐 적는다
`date: '\today'`	`\today` — 작은따옴표 안에서는 겹치지 않는다
`date: "\today"`	**탭 문자** + `oday`

마지막 줄이 함정이다. 큰따옴표 안의 `\t`는 탭, `\n`은 줄바꿈으로 해석되므로 `\today`, `\newpage`, `\textbf` 같은 명령이 조용히 다른 것이 된다. 오류가 나지 않아 알아채기 어렵다.

따옴표를 쓰지 않는 것이 가장 안전하다. 값에 `:`나 `#`이 들어가서 따옴표가 필요하다면 **작은따옴표**를 쓸 것. 여러 줄이 필요하면 `preamble: |`처럼 블록 표기를 쓰면 되고, 그 안에서는 이스케이프를 신경 쓰지 않아도 된다.

2.2. 모르는 키는 매크로로 넘어간다

스크립트가 쓰지 않는 키를 적어두면 `\def\NM<키>{값}`으로 프리앰블에 넘어간다. 프리앰블보다 먼저 놓이므로 프리앰블에서 그 값을 읽어 쓸 수 있다.

```yaml
---
theme: dark
mysize: 42
---
```

```
\def\NMtheme{dark}
\def\NMmysize{42}
```

- 접두사 NM은 mapping.yaml의 metadata_macro_prefix로 바꾸거나, 빈 문자열로 두어 기능을 끌 수 있다. multimarkdown처럼 접두사 없이 \def\theme{dark}로 넘기면 원고에 흔한 낱말이 그대로 제어열 이름이 되어 클래스나 패키지의 명령을 소리 없이 덮어쓸 수 있다.
- **키는 소문자로 낮춰서** 매크로를 만든다. Theme: dark도 \NMtheme이 된다. LaTeX 매크로 이름은 대소문자를 구별하므로, 원고 쪽 표기가 어떻든 프리앰블은 한 가지로 쓰면 되도록 한 것이다. 낮췄을 때 이름이 겹치면 뒤엣것만 남고 경고를 찍는다.
- Class:처럼 **알려진 키의 대소문자만 다른 이름**을 쓰면 그 키로 쓰이지 않고 \NMclass 매크로만 만들어진다. 오류가 나지 않으므로 경고로 알린다.
- **키 이름이 알파벳만으로 되어 있어야** 넘어간다. my_size나 size2처럼 숫자·밑줄·붙임표가 든 이름, 한글 이름은 제어열 이름이 될 수 없어 조용히 건너뛴다.
- 값이 스칼라(문자열·수·참거짓)인 것만 넘어간다. 리스트나 사전은 건너뛴다.
- class / classoption은 \documentclass[옵션]{클래스}를 구성한다.
- 이 헤더가 문서마다 반복되어 길어지는 것이 성가시다면, 공통 부분을 다른 파일에 빼두고 setup: 키로 물려받을 수 있다. **21장**을 볼 것.

2.3. maketitle / tocloft — 제목·목차 자동 삽입

\begin{document} 바로 다음에 \maketitle, \tableofcontents, \listoffigures, \listoftables를 자동으로 넣어 준다(pandoc처럼).

```
maketitle: true
tocloft:
  toc: true
  lof: true
  lot: true
  asterisk: true
```

```
\begin{document}

\maketitle
\tableofcontents*
\listoffigures*
\listoftables*
```

본문...

- maketitle: true만 켜면 \maketitle 한 줄만 나온다. tocloft:는 그 아래에 원하는 목차류만 켠다 — toc/lof/lot 중 켜지 않은 것은 그냥 안 나온다.

- 순서는 언제나 `\maketitle` → `\tableofcontents` → `\listoffigures` → `\listoftables` 고정이다.
- **`asterisk: true`는 `class`가 `memoir` 또는 `oblivoir`일 때만 실제로 의미가 있다.** 이 두 클래스는 `\tableofcontents*`가 "그 목차 자신을 목차 항목 목록에 넣지 않는다"는 뜻으로 정확히 동작한다(`memoir`의 `\newlistof` 매크로가 제공하는 기능). 반면 `article`/`report`/`book` 같은 표준 클래스에는 이런 별표 버전이 아예 없어서, `\tableofcontents*` 를 그대로 내보내면 **컴파일은 되지만 그 `*` 가 명령으로 소비되지 않고 목차 페이지에 그대로 문자로 찍힌다** (실측으로 확인한 함정이다). 안전한 두 클래스가 아닐 때 `asterisk: true`를 쓰면 경고를 찍지만, 강제로 막지는 않는다 — 커스텀 클래스에 별표 버전을 직접 정의해 뒀을 수도 있기 때문이다.
- 이 두 키도 front matter의 다른 키들처럼 `setup:`으로 상속된다.

이 기능은 캐주얼한 글이나 간단한 article 한 편에는 편리하지만, 단행본 저작에는 오히려 함정이 될 수 있다. `\maketitle`은 단순한 표지 하나를 찍을 뿐이다 — oblivoir/memoir는 그보다 훨씬 공을 들인 표지를 만들 수 있는 `titlingpage` 환경을 따로 제공하는데, 이 기능은 그쪽을 전혀 건드리지 않는다. 책 한 권 분량의 원고라면 대개 다음을 원고마다 직접 결정해야 한다.

- `\frontmatter`/`\mainmatter`/`\backmatter` — 앞/본/뒤 어느 부분이냐에 따라 쪽번호 체계(로마 숫자/아라비아 숫자)와 장 번호 매김이 달라진다. 이 기능은 이 셋 중 어느 것도 대신 넣어 주지 않는다.
- `\pagestyle`, `\chapterstyle` — 책마다 고유하게 고르는 지면·장 제목 디자인이라, 자동으로 정해 줄 수 있는 값이 없다.

이런 것들은 front matter의 `preamble:`이나 `beforestart:`에 직접 적어 두는 편이 맞다. `maketitle`/`tocloft`는 딱 그 이름이 말하는 만큼만 한다 — 표지 한 장과 목차류 몇 개를 문서 맨 앞에 끼워 넣는 것, 그 이상도 이하도 아니다.

---

3. 헤딩

```
# 1단계 제목
## 2단계 제목
### 3단계 제목 {unnumbered}
#### 4단계 제목 {label=sec:foo}
##### 5단계
###### 6단계
```

- 기본 매핑(16단계): `chapter`, `section`, `subsection`, `subsubsection`, `paragraph`, `subparagraph`. **`article` 클래스에는 `\chapter`가 없으므로, `article`을 쓸 때는 반드시 front matter에 `toplevel: section` (또는 `part`)을 지정해서 1단계가 `\section`부터 시작하도록 맞춰야 한다.**

- 헤딩 뒤 `{...}` 속성:
  - `unnumbered` 또는 `*` → 번호 없는 헤딩(`\section*` 등)
  - `label=이름` 또는 `#이름` → `\label{이름}` 자동 삽입
  - `toc="짧은 제목"` → `\section[짧은 제목]{긴 제목}`. 목차와 페이지 헤더에는 짧은 쪽이, 본문에는 긴 쪽이 나가게 하고 싶을 때 쓴다. `unnumbered`와 함께 쓰면 `*`판에는 대괄호 인자 자체가 없으므로 조용히 무시된다 — LaTeX이 어차피 받지 않는 조합을 오류로 알리기보다, 번호 없는 제목은 그대로 살리는 쪽을 골랐다.
  - 예: `## 서론 {unnumbered, label=sec:intro}`
  - 예: `# 아주 긴 절 제목이지만 목차에는 짧게 {toc="짧은 제목"}`
- 헤딩 한 줄 전체가 `HEADING_RE`에 매치되어야 하며, 헤딩 블록은 반드시 한 줄짜리 블록이어야 한다(앞뒤로 빈 줄로 구분).

---

4. 문단과 줄바꿈

같은 문단 안에서 줄을 나눠 쓰면 공백으로 이어붙여 하나의 문장으로 합쳐진다 (줄바꿈 자체는 출력에 영향을 주지 않음). **줄 끝에 스페이스 두 칸**을 남기면 그 지점에서 LaTeX 강제 줄바꿈(\\)이 삽입된다.

```
이 문장은 여러 줄로 나눠 썼지만
하나의 문단으로 합쳐진다.

이 줄 끝에는 스페이스가 두 칸 있다.
그래서 여기서 강제로 줄이 바뀐다.

    빈 줄로 문단을 구분한다.
```

4.1. `||` — 문단을 나누지 않고 이어 붙이기

한국어에서는 조사로 시작하는 문단(`라고 말하기 시작했다.`)이나 목록·수식 뒤에 이어지는 문단을 **들여쓰기 없이** 이어야 할 때가 있다. 그때 `\noindent`를 손으로 쓰는 대신, `||`만 있는 줄을 둔다.

```
"간단한 발화문."
||
라고 말하기 시작했다.

평문단이 끝나는 위치.
||
+ this is a sample
||
이어지는 문단
```

"간단한 발화문."라고 말하기 시작했다.
평문단이 끝나는 위치.

- this is a sample

이어지는 문단

- ||는 **구조상으로는 빈 줄과 같다.** 앞뒤를 별개 블록으로 갈라 주므로 || 바로 뒤의 + **항목**은 빈 줄 없이도 리스트로 인식된다(11장).
- 다만 **출력에서는 빈 줄을 넣지 않는다.** 그래서 LaTeX은 두 블록을 한 문단으로 이어 읽는다. 목록·수식 환경 뒤에서는 이 때문에 들여쓰기가 생기지 않는다.
- 앞 블록과 뒤 블록이 **둘 다 평문 문단이면** 앞 문단 끝에 %를 붙인다. 줄바꿈이 공백으로 남아 ''**라고**가 '' **라고**가 되는 것을 막는 것이다. 한국어 이음을 기본으로 한 것이라 공백이 들어가는 변형은 없다.
- || **앞에 빈 줄이 있으면** 앞 문단 끝에 % 대신 \\를 넣어 그 자리에서 행을 나눈다(앞뒤가 모두 평문 문단일 때만). 같은 문단 안에서 행만 나누고 싶을 때 **문장**\\를 손으로 쓰는 대신 쓴다. || **뒤**의 빈 줄은 뜻이 없다. 뒤 문단이 [나 *로 시작하면 \\가 그것을 선택 인자로 삼지 않도록 {}를 덧붙인다. 앞뒤가 평문 문단이 아니면(리스트·인용·헤딩 등) 빈 줄이 있어도 \\ 없이 그냥 이어진다. 예는 아래 목록 뒤에 있다.
- 문서의 맨 처음·끝이나 ||가 연달아 나오면 --lint가 경고한다.
- 코드 펜스 안의 ||는 글자 그대로다. 파이프 표 안에 ||만 있는 줄이 있으면 표가 거기서 끊긴다.
- ::: 환경 안에서도 쓸 수 있다.
- **새 문단이되 들여쓰기만 없애는 것이 아니다.** 이음은 한 문단으로 합치는 것이라, 문단을 끊고 들여쓰기만 없애고 싶다면 지금처럼 \noindent를 쓴다.

|| 앞에 빈 줄을 두면 행이 나뉜다.

첫 행

||
둘째 행

첫 행
둘째 행

---

5. 인라인 서식

문법	결과
굵고 기울임	\textbf{\emph{...}}
굵게	\textbf{...}
기울임	\emph{...}
~~취소선~~	\sout{...}
^^방점^^	\dotemph{...} (전통 편집물의 방점 표시; preamble에 \dotemph 정의 필요)
`인라인 코드`	\texttt{...} (내부는 자동 이스케이프됨)

/*/~~로 감싼 내부 텍스트는 escape_text만 거치고 재귀적으로 다시 처리되지 않는다(즉 굵게 안에 링크·각주 등을 중첩할 수 없다). 단, **따옴표 스마트 변환은 예외적으로 이 서식 처리보다 먼저 실행되므로 **"인용문"**처럼 굵게 안의 따옴표는 정상적으로 변환된다.

5.1. 규칙 추가하기 — extra_inline

위 다섯 규칙은 mapping.yaml의 inline에 들어 있는데, 이 키는 리스트형이라 손대면 **전체가 교체된다.** 규칙 하나 더하자고 다섯 개를 통째로 베껴 오는 것은 번거롭고 기본값이 바뀌면 뒤처진다. extra_inline은 기본 목록을 그대로 둔 채 **앞에** 끼워 넣는다.

```yaml
extra_inline:
  - name: highlight
    pattern: '==(.+?)=='
    template: '\hl{%s}'
```

- **뒤가 아니라 앞**이다. 규칙은 위에서부터 차례로 적용되므로 긴 델리미터가 먼저 와야 한다. ***...*** 규칙을 뒤에 붙이면 앞선 **...**가 먼저 먹어 버려 아무 일도 일어나지 않는다.
- 뒤집어 말하면, 기본 규칙보다 **짧거나 겹치는** 델리미터를 앞에 두면 기본 규칙을 가려버린다. *...*를 앞에 두면 **굵게**가 깨진다. 새 델리미터를 더할 때만 쓰고, 기존 것을 바꿀 때는 inline 자체를 override 할 것.
- extra_inline 안에서도 순서가 그대로 지켜지므로, 겹치는 규칙을 여럿 넣을 때는 긴 것을 먼저 적는다.

인자를 둘 이상 받는 규칙

캡처 그룹은 **개수만큼 그대로** template에 꽂힌다. 그래서 %s를 여럿 두면 된다.

```yaml
extra_inline:
  - name: dialog
    pattern: '\*\[(.+?)\]\[(.+?)\]'
    template: '\Dialog{%s}{%s}'
  - name: dialog_short
    pattern: '\*\[(.+?)\]'
    template: '\Dialog{%s}'
```

```
*[춘향][화난 목소리] 어디 계셨습니까?

\Dialog{춘향}{화난 목소리} 어디 계셨습니까?
```

두 인자짜리를 한 인자짜리보다 **먼저** 적어야 한다. 순서가 바뀌면 `*[춘향]`까지만 먹고 `[화난 목소리]`가 평문으로 남는다.

각 그룹은 따로 `escape_text`를 거치므로 인자 안의 `%`나 `&`는 알아서 안전해진다. 매치에 참여하지 않은 선택 그룹은 빈 문자열이 되고, 그룹 수와 `%s` 수가 어긋나면 경고를 찍고 원문을 그대로 둔다.

---

6. 따옴표 (스마트 따옴표 변환)

LaTeX은 여는/닫는 따옴표에 다른 문자를 쓴다. 이 스크립트는 직선 따옴표(", ')를 자동으로 변환해준다.

종류	여는 부호	닫는 부호	판정 기준
큰따옴표 "	`` (백틱 2개)	'' (아포스트로피 2개)	아래 참고
작은따옴표 '	` (백틱 1개)	' (그대로)	아래 참고

판정 방식: 문자열 맨 앞이거나, 공백 / ([{ * ~ 바로 뒤에 오는 따옴표만 "여는" 것으로 인식해 먼저 치환한다. 그 외에 남은 따옴표는 전부 "닫는" 것으로 간주해 일괄 치환한다. **닫는 따옴표 뒤에 무엇이 오는지는 검사하지 않는다** — 한국어는 `"인용"을`처럼 조사가 공백 없이 바로 붙는 경우가 많아서, pandoc처럼 "뒤에 공백/구두점이 오는가"로 닫는 위치를 판정하는 방식은 쓸 수 없기 때문이다.

```
그는 "안녕하세요"라고 말했다.
→ 그는 ``안녕하세요''라고 말했다.

그는 '좋다'며 웃었다.
→ 그는 `좋다'며 웃었다.

**"강조된 인용"**도 정상 변환된다.
→ \textbf{``강조된 인용''}
```

이스케이프: 스마트 변환을 원하지 않는 직선 따옴표는 `\"`, `\'`로 쓰면 백슬래시가 제거되고 문자 그대로(", ') 출력된다.

```
그는 \"진짜\" 직선따옴표를 원했다.
→ 그는 "진짜" 직선따옴표를 원했다.
```

다음 문맥의 따옴표는 스마트 변환의 영향을 받지 않는다(이미 placeholder로 보호된 뒤이므로): 인라인 코드, 인라인/디스플레이 수식, raw 매크로, 그림/링크의 `{속성="값"}`이나 `(url "제목")`, front matter의 YAML 값.

6.0.1. quotes — 결과 문자를 고르기

위 표는 기본값 `quotes: tex`의 결과다. mapping(`mapping:`, `mappingrule:`, `--config`)의 `quotes` 키로 바꿀 수 있다(0.20.4).

값	큰따옴표	작은따옴표
`tex` (기본)	``` `` '' ```	``` ` ' ```
`unicode`	U+201C " U+201D "	U+2018 ‘ U+2019 ’
4글자 `"「 」『 』"`	처음 두 글자(여는, 닫는)	뒤 두 글자(여는, 닫는)
2글자 `"《 》"`	그 두 글자	`tex` 방식 그대로
리스트 `["<<", ">>"]`	여러 글자짜리는 같은 길이의 리스트로	(4개 요소면 뒤의 둘)

`unicode`는 4글자 `""‘’`의 줄임말이다. 여는/닫는 판정과 이스케이프(`\"`, `\'`), 코드·수식 안의 따옴표 처리는 `tex`일 때와 똑같고 바뀌는 것은 치환 결과 문자뿐이다. 결과 문자는 뒤 단계가 건드리지 못하게 보호된다. 잘못된 값은 경고하고 `tex`로 둔다. `--pandoc`/`--mmd`에는 적용되지 않는다(pandoc의 smart 확장, mmd의 `--smart`가 맡는다).

- 유니코드 따옴표는 xelatex/lualatex에서 쓰는 글꼴에 그 글리프가 있을 때 나온다. pdflatex에서는 `inputenc`가 필요하다.
- 닫는 작은따옴표 문자는 `don't`의 아포스트로피에도 쓰인다. `unicode`에서는 U+2019라서 맞는 결과지만, 「 」『 』처럼 쓰면 `don』t`가 되므로 그 자리는 `\'`로 직선을 지키거나 `\textquoteright{}`를 쓴다.

6.1. 줄임표 (...)

마침표 세 개는 `\ldots{}`로 바뀐다. 마침표를 그냥 나란히 찍으면 간격이 붙어버려 조판상 틀린 결과가 나오기 때문이다.

말줄임...이렇게. 네 개....는 줄임표+마침표.

말줄임\ldots{}이렇게. 네 개\ldots{}.는 줄임표+마침표.

마침표가 넷 이상이면 왼쪽부터 셋씩 먹는다.

- **끝의 `{}`는 장식이 아니라 필수다.** 한글 음절이 letter(catcode 11)인 조판 환경에서 `\ldots이렇게`라고 나가면 TeX이 그 전체를 하나의 제어열 이름으로 읽어 `Undefined control sequence`가 난다. `{}`가 그 경계를 끊는다 (7장의 매크로 이름 종결자와 같은 이야기다).
- 명령은 `mapping.yaml`의 `ellipsis`로 바꿀 수 있다. 한국어 조판에서 줄임표는 폰트에 따라 모양과 간격이 제각각이라 갈아끼울 일이 생긴다.

```yaml
ellipsis: '\dots{}'          # amsmath. 앞뒤 문맥에 따라 간격 조절
ellipsis: '\textellipsis{}'  # 텍스트용 줄임표
ellipsis: '……'                # 유니코드 줄임표 두 개(한국어 전통 6점)
ellipsis: ''                 # 변환하지 않고 마침표 세 개 그대로
```

- 변환하지 않는 자리: 코드 스팬(`` `1...5` ``), 코드 펜스, 수식, 이미 쓰여 있는 `\ldots`, 링크의 URL, 색인어. 이 규칙이 도는 시점에는 그것들이 모두 보호되어 있다.

- 다만 `[[표시...|색인어]]`의 **표시 텍스트**도 색인과 함께 raw로 취급되어 변환되지 않는다(16장 참고). 그 자리에 줄임표가 필요하면 명령을 직접 적을 것.
- 유니코드 줄임표 …를 직접 입력한 것은 손대지 않고 그대로 내보낸다.

---

7. 인라인 수식 / 매크로 / raw LaTeX

```
인라인 수식: $x^2 + y^2 = 1$ 또는 \(x^2+y^2=1\)

raw 매크로도 그대로 통과한다: \textbf{직접 쓴 매크로} \cite[p.10]{key}

특수 이스케이프 시퀀스: \\ \, \; \% \& \# \_ \{ \}
```

- `$...$`(줄바꿈 불가) 또는 `\(...\)` 는 인라인 수식으로 보호되어 그대로 출력된다.
- `\명령어[선택인자]{인자}{인자}...` 형태의 매크로는 **내용을 건드리지 않고 그대로** 통과시킨다(중첩 중괄호 1단계까지 지원). 즉 필요하면 markdown 안에서 직접 LaTeX을 섞어 써도 안전하다.
- `\\`, `\,`, `\;`, `\%`, `\&`, `\#`, `\_`, `\{`, `\}` 는 짧은 이스케이프 시퀀스로 별도 인식되어 그대로 통과한다.
- **한국어 조사 매크로**: `\은`, `\는`, `\이`, `\가`, `\을`, `\를`, `\와`, `\과`, `\으로`, `\로` 그리고 `\<` 는 고정 리터럴 토큰으로 인식되어 그대로 통과한다(단행본 편집에서 조사 자동 처리용 매크로를 자주 쓰기 때문). 이 목록은 정확히 이 문자열만 인식하며, 뒤에 공백 없이 바로 이어지는 한글 단어를 매크로 이름에 그리디하게 포함시키지 않는다. 예시와 세부 규칙은 아래 참고.
- **매크로 이름 종결자 `{}`**: 이름 끝에 빈 중괄호를 붙이면 리터럴 목록에 없는 이름도 매크로로 통과한다 — `\이라{}고`, `\에서{}부터`처럼. TeX에서 제어열의 끝을 표시하면서 공백은 넣지 않는 표준 관용구이고, 영문 이름에서는 원래부터 되던 것(`\LaTeX{}의`)을 한글 이름까지 넓힌 것이다. 같은 규칙으로 `\강조{중요}`처럼 인자를 받는 한글 이름 매크로도 통과한다. 아래 참고.
- 매크로 이름 뒤의 **선택인자 대괄호는 여러 개** 올 수 있다. natbib의 `\citep[see][p.~33]{key}`가 대표적이다. 다만 중괄호 **뒤에** 오는 대괄호는 인식하지 못한다(`\raisebox{2pt}[0pt][0pt]{x}`) — 그런 경우는 7.1의 인라인 raw 스팬을 쓴다.
- 이 목록을 벗어나는 raw LaTeX 조각은 7.1의 인라인 raw 스팬을 쓴다.
- 이 리터럴 토큰 목록은 `cfg["literal_macros"]`(기본값이 바로 위 목록)와 `cfg["extra_macros"]`(기본값 빈 리스트)에서 오며, 둘 다 **`mapping.yaml`로 커스터마이즈할 수 있다**. 프로젝트별로 토큰 몇 개만 추가하고 싶으면(기존 목록을 그대로 둔 채) `extra_macros`를 쓴다. 예시는 아래 참고.

조사 매크로 그리디 매칭 방지 예시:

```
값\은 이렇게 된다. 값\은뭐라고말하기어렵다.
```

값\은 이렇게 된다. 값\은뭐라고말하기어렵다.

(뒤에 붙는 "뭐라고말하기어렵다"는 매크로 이름에 먹히지 않고 그대로 남는다.) `\으로`는 `\로`보다 먼저 검사되어, 더 긴 토큰이 항상 우선 인식된다. 이 목록에 없는 조사(예: `\도`)는 일반 텍스트로 취급되어 18장의 자동 이스케이프 규칙을 그대로 받는다(백슬래시가 `\textbackslash{}`로 바뀜).

`\라`와 `\이라`가 기본값에 없는 이유:

이 조사는 실제 문장에서 홀로 끝나는 일이 드물고 거의 언제나 보조사가 뒤에 덧붙는다 — `\이라고`, `\이라는`, `\이라도`, `\이라서`처럼. 리터럴 목록은 "정확히 이 문자열"만 인식하므로 그런 활용형을 규칙만으로는 감당할 수 없다.

여기서 **실패하는 방식이 둘로 갈리는 점을 조심해야 한다.**

원고	결과	문제
`값\라`	`값\textbackslash{}라`	눈에 띄게 깨지므로 금방 찾는다
`값\이라`	`값\이라`	**겉보기엔 멀쩡하지만 틀렸다**

`\이라`가 위험한 쪽이다. `\이`가 리터럴 토큰이라 먼저 매치되고 `라`는 평문으로 남는다. 출력 문자열은 똑같아 보이지만, LaTeX은 이것을 "`\이` 조사 + 글자 라"로 읽는다. 앞말이 모음으로 끝나면 `\이`가 가로 전개되어 `이라`가 아니라 `가라`가 찍힌다. 오류도 경고도 없다.

해결책: 이름 끝에 `{}` 를 붙인다.

```
값\이라{}고 하고, 값\이라{}는 것, 값\이라{}도 그렇다.
```

값\이라{}고 하고, 값\이라{}는 것, 값\이라{}도 그렇다.

빈 중괄호가 "매크로 이름은 여기까지"라는 선언이라, 스크립트는 어디까지가 이름인지 추측할 필요가 없다. 그래서 `\이라`가 리터럴 목록에 없어도 통과하고, `\이`에게 먹히지도 않는다. 활용형이 몇 가지든 목록에 등록할 필요가 없다.

이건 스크립트 편의만을 위한 표기가 아니다. **LaTeX 쪽에서도 어차피 필요하다.** 한글 음절이 letter(catcode 11)인 조판 환경에서 `\이라고`라고 쓰면 TeX은 이를 `이라고`라는 하나의 제어열로 읽어 `Undefined control sequence`를 낸다. `{}`가 그 경계를 끊어준다. 즉 원고에 쓰는 표기와 출력이 같고, 둘 다 같은 이유로 필요하다.

같은 규칙이 인자를 받는 한글 이름 매크로에도 적용된다.

```
\강조{중요한 말}, \색{red}{글자}, \인용[p.10]{책}
```

중괄호가 **최소 하나는 있어야** 한다는 점이 핵심이다. 그게 없으면 `\에서부터` 같은 것에서 어디까지가 이름인지 알 수 없어, 리터럴 목록을 두었던 그리디 매칭 문제가 그대로 돌아온다.

쓰는 활용형이 몇 개로 정해져 있어서 `{}`를 매번 타이핑하기 싫다면, 그것들을 `extra_macros`에 등록해도 된다. 길이 내림차순 자동 정렬 덕분에 순서는 신경 쓰지 않아도 된다.

```
extra_macros: ['\이라고', '\라고', '\이라는', '\라는', '\이라', '\라']
```

`extra_macros`로 토큰 추가하는 예시:

```yaml
# mapping.yaml - \>, \| 를 raw 매크로로 추가
extra_macros:
  - "\\>"
  - "\\|"
```

기본 조사 매크로 목록 자체를 바꾸고 싶으면(드묾) `literal_macros`를 통째로 override한다(리스트형 키라 override 시 전체가 교체된다 — 19장 참고). 어느 쪽이든 새로 추가한 토큰들 사이의 프리픽스 충돌(예: `\ab`와 `\abc`를 동시에 등록)은 걱정할 필요 없다 — 매번 길이 내림차순으로 자동 정렬해서 정규식을 새로 만들기 때문에, 등록 순서와 무관하게 항상 더 긴 토큰이 먼저 매치된다. 다만 이 정렬은 `literal_macros`와 `extra_macros`를 합친 목록 안에서만 작동한다. 목록에 아예 없는 긴 형태(위의 `\이라` 같은)는 정렬의 보호를 받지 못하고 짧은 토큰에 먹힌다.

7.1. 인라인 raw 스팬 — 문장 한복판의 LaTeX

위의 매크로 통과는 `\명령[선택인자]{인자}` **모양일 때만** 동작한다. 그 모양을 벗어나는 조각은 일반 텍스트로 간주되어 18장의 자동 이스케이프를 맞는다. 문장 한복판에 그런 조각을 심어야 할 때 쓰는 탈출구가 인라인 raw 스팬이다. Pandoc의 raw attribute와 같은 문법으로, 코드 스팬 바로 뒤에 포맷 이름을 붙인다.

그룹 중괄호: 이건 `{\bfseries 강조}`{=latex} 이다.
중첩 2단: `\textbf{a \emph{b \textsc{c}}}`{=latex} 끝.
중괄호 뒤 선택인자: `\raisebox{2pt}[0pt][0pt]{올림}`{=latex} 끝.
인자 안 수식: `\fbox{$x^2$}`{=latex} 끝.

닫는 백틱과 여는 중괄호는 **붙어 있어야** 한다. 사이에 공백이 있으면 평범한 코드 스팬으로 처리된다.

위 네 가지가 매크로 패턴으로 처리되지 않는 대표적인 경우다. 스팬 없이 그냥 쓰면 이렇게 깨진다.

그냥 썼을 때	결과
`{\bfseries 강조}`	그룹 중괄호는 매크로가 아니라서 `\{`, `\}`로 이스케이프된다
`\textbf{a \emph{b \textsc{c}}}`	중첩 중괄호를 1단까지만 인식해 뒤쪽이 어긋난다
`\raisebox{2pt}[0pt][0pt]{올림}`	선택인자는 매크로 이름 바로 뒤에서만 인식된다
`\fbox{$x^2$}`	인자 안의 `$`와 `^`가 이스케이프된다

포맷 이름은 `latex`, `tex`, `raw`를 인정한다(`raw_inline_formats`로 바꿀 수 있다). **그 밖의 포맷을 겨냥한 스팬은 통째로 버려진다** — Pandoc과 같은 동작이다. LaTeX 출력에 넣을 자리가 없기 때문이다.

앞`<br>`{=html}뒤

앞뒤

블록 단위로 raw LaTeX을 넣을 때는 이 문법이 아니라 10장의 `raw` / `tex` / `latex` 코드 펜스를 쓴다. 이 스팬은 문단을 나눌 수 없는 **문장 중간**을 위한 것이다.

7.2. 물결표(~) 정책

~와 \~는 **둘 다 원문 그대로 LaTeX 소스에 나간다.** 변환기는 자리만 지키고, 그것이 묶음빈칸인지 물결표인지는 LaTeX 쪽이 정한다.

원고	출력
그림~1	그림~1
파일은 \~/bin	파일은 \~/bin
1\~6단계	1\~6단계

이렇게 정한 이유가 있다. LaTeX에서 ~는 묶음빈칸(\nobreakspace)이고 \~는 악센트 명령이라, 둘 다 **LaTeX 쪽에서 재정의할 수 있는 것들**이다. 특히 한국어 조판에서는 \~ 뒤에 숫자나 한글이 오면 물결표로 바꿔주는 식의 패키지를 쓰기도 한다. 변환기가 \~를 \textasciitilde{}로 미리 굳혀버리면 그런 정의가 개입할 자리를 없애버린다. 그래서 의미 부여를 LaTeX에 넘긴다.

\~를 살려두려면 **프리앰블에 정의가 있어야 한다.** 아무 패키지도 없는 순정 LaTeX에서 \~는 뒤 글자에 물결 악센트를 씌우는 명령이라, 1\~6은 16으로, \~/bin은 /bin으로 조판된다 — 물결표가 사라진 것처럼 보인다. 정의가 없는 문서라면 \newcommand{\~}{\textasciitilde{}}처럼 직접 잡아주거나, 아예 원고에 \textasciitilde{}를 쓴다.

wavedash — \~를 물결표로 쓰기

위 기본 동작(wavedash: default)은 ~와 \~를 둘 다 LaTeX에 맡긴다. 한국어 원고에서 "13장"의 \~를 곧바로 파동 대시(물결표)로 쓰고 싶다면 mapping에 wavedash를 준다. 값이 default가 아니면 **~는 그대로 묶음빈칸(nobreak space)이고, \~만 이 값으로 바뀐다.**

```
wavedash: FF5E              # 유니코드 코드포인트(16진수): ~ (전각 물결)
wavedash: "U+301C"          # U+ 접두도 된다: ~ (WAVE DASH)
wavedash: " ~ "             # 글자 그대로 한 글자
wavedash: \koreanWaveDash   # LaTeX 매크로 (뒤에 {}가 자동으로 붙는다)
wavedash: default           # 기본값 - 옛 동작 그대로
```

- 코드포인트는 따옴표로 감싸거나 U+를 붙이는 것을 권한다. 3030처럼 숫자만 쓰면 YAML이 정수로 읽지만 16진 자리수로 해석하고, 0x3030은 YAML이 10진 정수로 바꿔 버려 엉뚱한 글자가 된다.
- 매크로는 프리앰블(preamble:)에 직접 정의해 두어야 한다. 출력은 \koreanWaveDash{}다.
- 알아볼 수 없는 값이면 경고를 찍고 default로 처리한다.
- 적용 범위는 본문·제목·각주·표 셀 등 변환기가 처리하는 텍스트다. 인라인 코드 스팬 안의 ~는 아래 예외대로 항상 \textasciitilde{}이고, front matter 값(title: 등)은 원래 LaTeX 원문이라 \~를 바꾸지 않는다.
- 유니코드 문자를 쓸 때는 선택한 글꼴이 그 글리프를 갖고 있어야 하고 (pdflatex는 입력 인코딩 설정도 필요하다), 확실하지 않으면 매크로가 안전하다.

`tilde` — `~`와 `^`의 방언

기본(`tilde: latex`)은 위에서 설명한 대로 `~`를 LaTeX에 맡기고 `^`는 `\textasciicircum{}`으로 내보낸다. 다른 마크다운 방언의 원고를 가져와 쓸 때를 위해 mapping에서 방언을 고를 수 있다.

`tilde:`	`~`	`^`
`latex` (기본)	그대로(묶음빈칸)	`\textasciicircum{}`
`pandoc`	`\textasciitilde{}`. `~x~`는 `\textsubscript{x}`	`^x^`는 `\textsuperscript{x}`
`multimarkdown` (`mmd`)	`a~b`는 `a\textsubscript{b}`, `~x~`도 아래첨자. 뒤에 글자가 없는 `~`는 `\ensuremath{\sim}`	`a^b`, `^x^`는 `\textsuperscript{}`
`wavedash` (0.20.5)	**언제나** wavedash. `wavedash:`로 문자를 정했으면 그것, 없으면 `\textasciitilde{}`	`latex`와 같다 (`\textasciicircum{}`)

- 닫힌 꼴(`~x~`, `^x^`)은 안에 공백이 없어야 한다. multimarkdown의 열린 꼴 (`a~b`, `a^b`)은 뒤따르는 글자·숫자 덩어리(공백·구두점 앞)까지다.
- `\~`는 어느 모드에서나 `wavedash` 규칙을 따른다. 그래서 multimarkdown 모드에서 `1~3쪽`이 아래첨자가 되는 것을 `1\~3쪽`으로 피할 수 있다.
- `~~취소선~~`, `^^방점^^`, `^[각주]`, 코드 스팬, 수식, `\매크로{…}`(인자 포함)는 어느 모드에서나 먼저 처리·보호되므로 영향이 없다. 링크로 감싸지 않은 맨 URL의 `~`는 pandoc에서 `\textasciitilde{}`, multimarkdown에서는 아래첨자가 되므로 `<…>`나 링크 문법을 쓴다.
- 제목 안의 `\textsubscript`·`\ensuremath`는 hyperref 책갈피 경고를 낼 수 있다.
- 진짜 pandoc을 쓰는 `--pandoc` 옵션에서는 이 키를 무시한다(pandoc이 직접 처리한다). 값 이름 `pandoc`과 옵션 `--pandoc`은 별개다.
- `wavedash` 모드는 `^`와 무관하다. 본문의 `~`는 첨자가 아니라 전부 물결표로 쓰겠다는 뜻이고, `\~`도 똑같이 처리되어(`wavedash:`가 `default`여도 `\textasciitilde{}`) 둘 사이에 차이가 없다. `~~취소선~~`은 먼저 처리되므로 그대로다. 코드 스팬은 위와 같이 항상 `\textasciitilde{}`이고, 수식 안의 `~`와 `\매크로{…}`는 건드리지 않는다.
- 알 수 없는 값은 경고 후 `latex`로 처리한다.

유일한 예외: 인라인 코드 스팬

인라인 코드 스팬 안의 `~`만은 항상 `\textasciitilde{}`로 이스케이프된다.

`` `rm -rf ~/tmp` ``

```
\texttt{rm -rf \textasciitilde{}/tmp}
```

여기는 저자가 쓴 LaTeX이 아니라 **코드를 글자 그대로 보여주는 자리**이기 때문이다. `~`를 날것으로 두면 묶음빈칸이 되어 `rm -rf /tmp`처럼 물결표가 증발한다. `_`, `$`, `&`, `{` 등 다른 특수문자도 여기서는 전부 이스케이프되니 `~`만 예외로 두는 편이 오히려 어색하다. 이 동작이 싫다면 스크립트의 `_with_tilde()` 한 줄을 없애면 된다. 코드 펜스 안은 verbatim/minted라 원래 아무것도 하지 않는다.

인라인 서식 규칙과의 관계

취소선 ~~...~~처럼 ~를 델리미터로 쓰는 규칙은 그대로 동작한다.

\~로 이스케이프한 물결표는 델리미터로 세지 않는다. a\~\~b\~\~c는 취소선이 아니라 a\~\~b\~\~c 그대로 나간다.

이게 가능한 이유는 \~를 보호하는 지점이 앞뒤 양쪽에서 고정돼 있기 때문이다. 스마트 따옴표 변환보다는 뒤여야 하고(그래야 \~"인용"의 큰따옴표가 여는 모양으로 나온다), 인라인 규칙보다는 앞이어야 한다(그래야 escape한 물결표가 델리미터로 둔갑하지 않는다).

예전 동작으로 되돌리려면

mapping.yaml에서 escape_chars에 ~를 도로 넣으면 모든 맨 ~가 \textasciitilde{}가 된다 (리스트형 키라 override 시 전체가 교체되므로 나머지 문자도 함께 적어야 한다 — 19장 참고). 다만 \~는 이 설정과 무관하게 언제나 그대로 나간다.

```
escape_chars: ["\\", "&", "%", "#", "_", "$", "{", "}", "~", "^"]
```

---

8. 디스플레이 수식 / raw 환경 (블록 단계에서 보호)

다음은 **블록 파서 이전 단계**에서 통째로 보호되어, 안의 내용이 이스케이프나 따옴표 변환 등 어떤 처리도 받지 않고 그대로 출력에 옮겨진다.

```
$$
\int_a^b f(x)\,dx = F(b) - F(a)
$$

\[
E = mc^2
\]

\begin{equation}
a^2 + b^2 = c^2
\end{equation}

\begin{align}
x &= 1 \\
y &= 2
\end{align}
```

지원하는 raw 환경: equation*?, align*?, gather*?, multline*?, eqnarray*? (수식), tabular, tabularx, tabulary, tblr, longtblr (직접 짠 표). \begin{...}와 \end{...} 이름이 정확히 일치해야 하며, 중간에 다른 내용이 끼어 있어도(비탐욕 매칭) 하나의 블록으로 인식된다.

9. 주석 (<!-- -->)

pandoc과 동일하게, <!--로 시작해서 그 뒤에 처음 나오는 -->까지 통째로 사라진다(HTML 표준처럼 중첩은 지원하지 않는다 — 안쪽에 <!--가 또 있어도 무시하고 바로 다음 -->에서 닫힌다). 여러 줄에 걸쳐도 되고, 그 안에 헤딩·리스트·수식·인라인 서식이 들어 있어도 **전혀 해석되지 않고 그대로 삭제**된다.

이 문장은 보인다. <!-- 이 부분은 사라진다 --> 이 문장도 보인다.

<!--
이 블록 전체가 통째로 사라진다.

이 헤딩도 사라진다

- 이 리스트도 사라진다
-->

이 문장만 남는다.

코드펜스나 인라인 코드 백틱으로 감싼 <!-- -->는 실제 주석으로 처리되지 않고 문법 예시 그대로 보존된다(단, 인라인 코드는 위 8장의 raw 환경과 같은 이유로, 백틱이 <!--/--> 바로 옆에 붙어 있어야 안전하다).

주석 제거는 코드펜스 보호 바로 다음, 디스플레이 수식/raw 환경 보호보다 먼저 실행된다 — 그래야 주석 안에 우연히 들어간 \begin{...} 같은 텍스트가 raw 환경으로 오인식되기 전에 먼저 사라진다.

9.1. 라인 코멘트 — linecomment: true

LaTeX에 익숙한 사람들이 아쉬워하는 것 중 하나가 한 줄짜리 주석이다. mapping.yaml에서 linecomment: true로 켜면, **행 머리(들여쓰기 없이 정확히 첫 칸)**에 //로 시작하는 줄이 통째로 사라진다. 기본은 꺼져 있다.

linecomment: true

문단 하나.

// 이 줄은 지워진다.

문단 둘.

- %는 이스케이프 대상이자 흔한 문장부호(백분율)라 못 쓰고, #은 헤딩·상호참조에 이미 쓰이며, --는 en dash 관례(1--6)로 이미 뜻이 있어서 그대로 두면 충돌한다. //를 고른 이유다.

- **들여쓰기가 조금이라도 있으면 지워지지 않는다.** `  // 텍스트`처럼 들여써진 줄은 저자가 실제로 쓰려던 텍스트일 가능성이 높다고 보고 손대지 않는다.
- **코드 펜스 안은 이 옵션을 켜도 절대 지워지지 않는다.** 코드 펜스는 이 검사보다 먼저 보호되므로, C/JavaScript 예제의 `// 진짜 주석`이 첫 칸부터 시작해도 안전하다. 위험한 것은 원고 본문에 우연히 첫 칸부터 `//`로 시작하는 줄이 있는 경우뿐이라, 기본값은 꺼둔다.
- **뒤따르는 개행까지 함께 지워진다.** 그래서 리스트 항목 사이에 빈 줄 없이 주석을 끼워 넣어도 리스트가 쪼개지지 않는다 — 애초에 이 기능을 만든 주된 이유가 그것이다.

```
- 항목 하나
// 이 항목에 대한 메모
- 항목 둘
```

```
\begin{itemize}
\item 항목 하나
\item 항목 둘
\end{itemize}
```

빈 줄로 감싸 쓰면(`문단\n\n// 주석\n\n문단`) 그 빈 줄은 그대로 남으므로 문단이 갈리는 자리에서는 평소처럼 문단이 나뉜다.

---

10. 코드 블록

10.1. 일반 코드 펜스

````
```python
def f(x):
    return x + 1
```
````

````
```text
그냥 verbatim으로 처리됨
```
````

````
```raw
\centering
이 안은 이미 LaTeX이라고 가정하고 그대로 통과시킨다.
```
````

- 언어 태그가 `raw` / `tex` / `latex` → 내용을 **그대로**(가공 없이) 통과.
- 언어 태그가 없거나 `text` / `verb` → `\begin{verbatim}...\end{verbatim}`.

- 그 외 언어 태그 → `\begin{minted}{언어}...\end{minted}` (기본 `codeblock_template`; mapping.yaml로 다른 템플릿으로 바꿀 수 있음).
- 인라인 코드 `` `code` `` 는 `\texttt{}`로 변환되며 내부 특수문자는 자동 이스케이프된다(백슬래시 포함).
- `\`` (백슬래시+백틱)는 "백틱 그 자체"를 리터럴로 출력한다. 단, 이 이스케이프는 **새 코드 스팬을 찾을 때만** 적용되고, 이미 열린 스팬의 닫는 델리미터 쪽에는 적용되지 않는다 — 그래서 `` `\\` ``처럼 내용이 백슬래시로 끝나는 정상적인 코드 스팬(예: TeX 줄바꿈 명령을 보여줄 때)은 깨지지 않는다.
- **닫는 펜스 바로 다음 줄에 빈 줄 없이 글이 이어지면, 그 글은 반드시 줄이 갈려서 나온다** — `\end{verbatim}` 등과 같은 줄에 붙지 않는다. 일부 verbatim류 패키지는 `\end{...}` 뒤에 오는 내용이 같은 줄에 있으면 그 환경이 바꿔 둔 catcode가 거기까지 번지므로, 이 줄 분리가 없으면 조판이 깨질 수 있다. 원문에 실제 빈 줄이 있으면(그래서 새 문단이면) 그 빈 줄도 그대로 유지된다 — 이건 "펜스 바로 다음 줄에 곧장 이어지는 경우"만 다루는 규칙이다.

이건 `` \`code` `` 처럼 실제 코드 델리미터 바로 앞에 있어도 안전하다.
→ 이건 `` `code` `` 처럼 실제 코드 델리미터 바로 앞에 있어도 안전하다.

강제 줄바꿈은 `` `\\` `` 이다. (정상적인 코드 스팬 - 안 깨짐)
→ 강제 줄바꿈은 `\texttt{\textbackslash{}\textbackslash{}}` 이다.

10.2. showexam — 원문과 변환 결과를 나란히

지금 읽고 있는 이 문서처럼 "이렇게 쓰면 이렇게 나온다"를 보여줘야 하는 글을 위한 블록이다 (0.2.1에서 추가). 왼쪽 상자에는 마크다운 원문이 그대로 들어가고, 오른쪽 상자에는 그 원문을 변환기에 **다시 통과시킨 결과**가 들어간다.

````
```showexam
& $ 100% *기울임* **굵게**
```
````

이 한 블록이 아래처럼 펼쳐진다.

```
\par\medskip\noindent
\begin{minipage}[t]{0.4600\linewidth}\vspace{0pt}
\setlength{\topsep}{0pt}\setlength{\partopsep}{0pt}
\begin{minted}[frame=single,breaklines,breakanywhere,fontsize=\small,%
               xleftmargin=0pt]{markdown}
& $ 100% *기울임* **굵게**
\end{minted}
\end{minipage}\hfill
\begin{minipage}[t]{0.5000\linewidth}\vspace{0pt}
\setlength{\topsep}{0pt}\setlength{\partopsep}{0pt}
\& \$ 100\% \emph{기울임} \textbf{굵게}
\end{minipage}
```

```
\par\medskip
```

같은 원문을 두 벌 쓰는 일이라 LaTeX 쪽에서는 만들 수 없다 — verbatim으로 한 번 찍어낸 내용을 되살려 다시 조판하도록 되돌릴 방법이 없기 때문이다. 그래서 스크립트가 원문을 붙잡아 두었다가 한 번은 그대로, 한 번은 파이프라인에 태워 내보낸다.

오른쪽 상자는 본문과 **완전히 같은 파이프라인**을 탄다. 리스트, 표, 수식, 커스텀 환경, 상호참조가 전부 그대로 동작하고, 예제 안에서 참조한 각주는 바깥 문서의 각주 정의를 찾아 쓴다.

아래는 설명이 아니라 이 문서가 실제로 조판한 결과다 — 추가 패키지 없이 컴파일되도록 `style=verbatim`으로 지정했다.

```
**굵게**와 *기울임*, 그리고 `코드`.

- 리스트도
- 그대로 동작한다
```

굵게와 기울임, 그리고 **코드**.

- 리스트도
- 그대로 동작한다

펜스 줄 옵션

펜스 줄에 **키=값**을 공백으로 나열하면 그 블록에만 적용된다.

```
```showexam ratio=0.40 style=lstlisting align=c frame=none
```
```

옵션	기본값	뜻
`ratio`	`0.46`	왼쪽(원문) 상자 폭. `\linewidth` 대비 비율
`gap`	`0.04`	두 상자 사이 여백. 오른쪽 폭은 `1 - ratio - gap`
`align`	`t`	minipage 세로 정렬. `t` / `c` / `b`
`style`	`minted`	원문 상자를 조판하는 방식(아래 표)
`lang`	`markdown`	minted에 넘길 언어 태그
`frame`	`single`	원문 상자의 테두리. `none`이면 안 그린다
`layout`	`side`	`side`(좌우) / `stack`(위아래) / `auto` — 아래 「위아래 배치」
`stack_width`	`36`	`auto`의 임계값. 원문 가장 긴 줄의 표시 폭(한글 2칸)
`sep`	`rule`	`stack`에서 원문과 결과 사이: `rule` / `space` / `none` / LaTeX 코드

값에 공백이 필요하면 큰따옴표로 감싼다(`lang="text"`). 이 블록 하나에만 쓸 listing 템플릿을 통째로 넘기는 `listing=` 옵션도 있지만, 템플릿에는 공백과 중괄호가 많아 펜스 줄에 쓰기 불편하다 — 그런 용도라면 `mapping.yaml`을 쓰는 편이 낫다.

`style` 선택 — 정말 중요한 것은 다국어 여부

`lstlisting`은 코드 안에 유니코드가 섞이면 감당하지 못한다. 소스 코드 자체는 대개 ASCII라 무리가 없지만, 주석에 한글이 한 글자라도 들어가면 `listings` 패키지가 그것을 처리하지 못한다. 한국어 코드 예제에서는 사실상 `minted`가 유일한 해법이다. `listings`가 pdfTeX 시절 폰트 인코딩 위에 지어진 패키지라 그렇다.

style	다국어(한글 등)	긴 줄 접힘	필요한 것
minted	**된다**	접힘	minted 패키지 (아래 참고)
lstlisting	**안 됨**	접힘	listings 패키지만 있으면 된다
verbatim	된다	**안 접힘**	아무것도 필요 없다

verbatim은 줄바꿈을 못 하므로, 원문에 긴 줄이 하나라도 있으면 그 줄이 오른쪽 상자를 뚫고 페이지 여백까지 넘어간다. 추가 패키지 없이 한글을 쓰고 싶다면 원문 줄을 짧게 유지하며 verbatim을 쓰는 수밖에 없다.

minted의 -shell-escape 요구 여부는 TeX Live 배포 시기에 따라 갈린다. **minted 3.0부터는 pygments 호출을 latexminted라는 별도 파이썬 패키지가 맡고, TeX Live 2025부터 그것이 함께 배포되어 -shell-escape 없이도 안전하게 동작한다.** 그보다 오래된 TeX Live나 3.0 미만의 minted에서는 여전히 -shell-escape가 필요하다. kpsewhich latexminted.py로 있는지 확인할 수 있다.

중첩 — showexam 문법 자체를 보여주기

showexam 블록을 예시로 인용하려면 바깥 펜스를 백틱 4개 이상으로 연다. CommonMark와 같은 규칙이라, 백틱 개수가 더 적은 안쪽 펜스는 전부 내용으로 흡수된다. 이 절의 첫 예제가 바로 그 방식이다.

반대로, 평범한 코드펜스 안에 들어 있는 showexam 펜스는 **잡히지 않는다**. 문법 설명 문서에서 예시로 적어둔 것을 진짜 블록으로 오인해 실행해버리는 일은 없다.

showexam 안에 다시 showexam을 넣는 것도 3단까지는 재귀적으로 렌더링된다. 그보다 깊어지면 안쪽은 평범한 코드펜스로 취급된다.

mapping.yaml로 배치 바꾸기

template을 통째로 갈아끼우면 상자의 모양과 배치를 마음대로 바꿀 수 있다. tcolorbox의 sidebyside가 정확히 이 용도라 잘 맞는다.

```yaml
# mapping.yaml - preamble에 \usepackage[most]{tcolorbox} 필요
showexam:
  ratio: 0.42
  frame: none      # tcolorbox가 이미 테두리를 그린다
  template: |-
    \begin{tcolorbox}[sidebyside,sidebyside align=top,
                      lefthand ratio=%(ratio)s]
    %(left)s
    \tcblower
    %(right)s
    \end{tcolorbox}
```

frame: none을 빼먹으면 **상자 안에 상자**가 생긴다. 기본 template은 테두리를 그리지 않으므로 listing이 직접 frame=single로 그리는데, tcolorbox처럼 이미 테두리가 있는 template으로

바꾸면 둘이 겹치기 때문이다. 반대로 특정 블록에서만 안쪽 테두리를 되살리고 싶으면 그 블록의 펜스 줄에 `frame=single`이라고 적으면 된다.

`sidebyside`와 `breakable`은 **함께 쓸 수 없다**. 둘 다 지정하면 tcolorbox가 조용히 세로 2단(위/아래)으로 떨어뜨린다 — 오류가 나지 않으니 알아채기 어렵다. 좌우 배치를 유지하려면 `breakable`을 빼야 한다.

템플릿에 꽂히는 값은 다음과 같다.

자리표시자	내용
`%(left)s`	원문 상자. `verbatim`/`minted` 등이 이미 완성된 형태로 들어온다
`%(right)s`	변환 결과
`%(lw)s` / `%(rw)s`	왼쪽 / 오른쪽 폭 계수. `0.4600` 같은 소수 문자열
`%(ratio)s`	`ratio` 값 그대로. `tcolorbox`의 `lefthand ratio` 등에 쓴다
`%(align)s`	`t` / `c` / `b`
`%(strut)s`	`align=t`일 때 `\vspace{0pt}`, 아니면 빈 문자열

`styles` 항목을 손대면 `style` 이름별 listing 템플릿도 바꿀 수 있다. 여기엔 `%(src)s`(원문), `%(lang)s`, 그리고 `%(frame)s`가 들어간다. `%(frame)s`는 `frame=single,`처럼 **쉼표까지 포함한 옵션 조각**으로 치환되며, `frame`이 `none`이면 빈 문자열이 된다.

기본 템플릿이 각 minipage 앞에 `%(strut)s`(= `\vspace{0pt}`)를 두고 `\topsep`을 0으로 만드는 데에는 이유가 있다. `[t]` 정렬은 두 상자의 "첫 줄 베이스라인"을 맞추는데, 왼쪽 listing에 `frame`이 붙거나 위쪽 여백이 얹히면 그만큼 두 단이 어긋나 보인다. 이 관용구가 정렬 기준을 "상자 윗변"으로 바꿔준다. `align`이 `c`나 `b`이면 오히려 방해가 되므로 자동으로 빠진다.

위아래 배치 — `layout=stack` (0.20.1)

한 줄이 길어서 좌우로는 비좁을 때, 원문 상자를 본문 전체 폭으로 놓고 그 아래에 결과를 잇는다. 원문과 결과 사이에는 가는 선(`sep=rule`)이 들어간다.

```showexam layout=stack
긴 한 줄 …
```

- `layout=auto`는 원문에서 가장 긴 줄의 표시 폭이 `stack_width`(기본 36칸)를 넘으면 stack, 아니면 side로 간다. 실제 상자 폭은 용지와 글꼴에 따라 달라서 어림셈이다. 펜스 줄의 `layout=`이 언제나 우선한다.
- 문서 전체의 기본값은 mapping의 `showexam: {layout: stack}`으로 바꾼다. stack용 템플릿은 `stack_template`이며 `%(left)s %(right)s %(sep)s`를 쓴다.
- `ratio`, `gap`, `align`은 stack에서 의미가 없어 무시된다(펜스 줄에 직접 적었으면 경고).
- stack은 minipage를 쓰지 않으므로 **쪽을 넘어 이어질 수 있다**. 원문 상자가 `lstlisting`이면 테두리째 쪽을 넘고, `minted`의 `frame=single`처럼 테두리 상자가 쪽을 못 넘는 조합은 `frame=leftline`이나 `lines`로 바꾸면 된다.

주의

- 좌우 배치(side)는 상자를 minipage로 만들기 때문에 **페이지를 넘어 쪼개지지 않는다**. 예제가 길면 통째로 다음 페이지로 밀린다. tcolorbox로 바꿔도 마찬가지다 — breakable은 sidebyside와 양립하지 않는다. 긴 예제는 위의 layout=stack을 쓴다.
- 리스트 항목이나 인용구 **안에 들여쓴 showexam은 지원하지 않는다**. 자리표시자가 0열로 나가면서 리스트를 빠져나온다. 최상위에서만 쓴다.
- template을 고칠 때 원문 상자를 \fbox{...}나 \parbox{...} 같은 **매크로 인자 안에 넣으면 안 된다** — verbatim은 인자 안에서 깨진다. minipage와 tcolorbox의 본문은 인자가 아니라 환경 본문이라 안전하고, 테두리가 필요하면 minted/lstlisting의 frame= 옵션을 쓰면 된다.

---

11. 리스트

```
- **첫 항목**: 이 문장은 여러 줄에 걸쳐
  이어서 작성해도 하나의 항목 본문으로
  합쳐진다.
- 둘째 항목
  - 중첩 항목 A
  - 중첩 항목 B
    - 더 깊은 중첩

1. 번호 리스트
2. 다음 항목
1) 괄호 스타일도 가능

- 불릿 안에 번호를 넣어도 되고
  1. 이렇게
  2. 종류가 단계마다 바뀌어도 된다
     - 이렇게도

{용어} ::   설명 (권장 표기 - 중괄호 + 공백 둘 이상)
{다른 용어} ::  다른 설명

용어 :: 설명 (예전 표기, 아직 인정된다)
```

- 불릿 기호: -, *, +. 번호는 1. 또는 1) 둘 다 인식(번호 자체 값은 무시하고 LaTeX enumerate가 자동 채번).

- **항목 이어쓰기**: 불릿/번호/`::` 마커가 없는 후속 줄은 직전 항목의 본문에 공백(또는 줄 끝 스페이스 두 칸이면 `\\`)으로 이어붙는다.
- **중첩**: 더 깊게 들여쓴 마커 줄만 하위 리스트로 재귀 처리된다. 하위 리스트의 종류는 **그 줄의 마커가 스스로 정한다** — 불릿 안에 번호를, 번호 안에 불릿을, 어느 쪽에든 설명 리스트를 넣을 수 있고, 단계마다 종류가 바뀌어도 된다.
- 한 단계 안에 서로 다른 마커가 섞이면 그 단계의 **첫 항목**이 환경을 정한다. 설명 리스트 안에 불릿 항목이 끼어들면 라벨 없는 `\item`으로 떨어진다.
- 항목 첫 줄에 `{!옵션}`을 붙이면 리스트 전체의 `\begin{itemize}[옵션]`에 반영된다. **느낌표(`!`)로 시작하는 중괄호만 옵션이다** — `!` 없는 `{...}`는 옵션이 아니라 항목 본문의 일부이고, 다른 본문 속 중괄호와 같이 글자 그대로 찍힌다. `!`는 제거된다. 쉼표로 추가 텍스트를 붙이면(`{!1., \firmlist}`처럼) `\begin{enumerate}[1.] \firmlist` 식으로 뒤에 그대로 덧붙는다. 리스트의 **첫 항목에서만** 인식한다.
- **`enumitem`처럼 옵션 자체가 쉼표로 여러 개 나열되는 패키지를 쓸 때는 대괄호 안에 들어갈 부분 전체를 큰따옴표로 감싼다.** 그냥 `{!1., \firmlist}`처럼 첫 쉼표에서 나누면, `nosep,label={\arabic*}, ref=\arabic*`처럼 옵션 자체가 쉼표투성이인 경우 첫 쉼표에서 엉뚱하게 잘려 버린다. 예시는 아래를 볼 것.

```
1. {!"nosep,label={\arabic*},ref=\arabic*",\tightlist} 항목

\begin{enumerate}[nosep,label={\arabic*},ref=\arabic*] \tightlist
```

따옴표 **안**의 쉼표는 전혀 구분자로 보지 않고 통째로 대괄호 안에 들어간다. 닫는 따옴표 바로 뒤에 오는 쉼표부터가 그 뒤에 그대로 붙는 raw LaTeX이다. 옵션 블록 안에 `label={\arabic*}`처럼 중괄호가 중첩돼 있어도 정확히 짝을 맞춰 읽으므로 안전하다.

- **항목 안의 블록 요소는 지원하지 않는다.** 항목 본문에 빈 줄을 두고 둘째 문단이나 코드 블록을 넣으면, 그 지점에서 리스트가 닫히고 블록이 밖으로 나온 뒤 새 리스트가 다시 열린다. 항목 본문은 언제나 인라인 한 덩어리다 (22장 참고).
- **문단이 우연히 리스트 마커처럼 보이는 글자로 시작할 때는 중괄호로 감싸서 리스트 인식을 피한다.** `1. 소요 시간은...`처럼 문장이 숫자와 마침표로 시작하면, 의도와 무관하게 번호 리스트로 인식된다. 마커 글자를 중괄호로 감싸면(구두점은 중괄호 밖에) 리스트로 인식되지 않고 그 글자 그대로 찍힌다. 이 검사는 문단의 **첫 줄에서만**, 그리고 실제 마커(`UL_RE`/`OL_RE`)를 검사하기 **전에** 이루어진다 — 그래서 이스케이프를 벗겨낸 결과가 다시 리스트로 오인될 걱정은 없다. 마커 뒤에 진짜로 리스트 옵션 블록(`{!옵션}`)을 쓰는 것과는 전혀 다른 상황이라 서로 충돌하지 않는다 — 옵션 블록은 이미 인식된 마커 **뒤에** 오지만, 이 이스케이프는 마커 **대신** 줄 맨 앞에 온다.

쓰기	찍히는 것
`{-}` / `{*}` / `{+}`	`-` / `*` / `+`
`{1}.`	`1.`
`{1})`	`1)`

- **설명 리스트에는 두 표기가 있다.** 새로 쓰는 원고에는 중괄호형을 권한다.

표기	조건
`{용어} :: 설명`	줄 첫머리에 `{..}`, `::` 뒤에 공백/탭 **두 개 이상**
`용어 :: 설명`	아래 세 조건 (예전 표기)

  - 중괄호형은 산문이 우연히 그 꼴이 될 일이 없어 **오인식이 사실상 없다.** 덤으로 용어에 공백이나 콜론을 넣을 수 있다 — `{C++의 std::vector} :: 동적 배열` 처럼. `mapping.yaml`에 `description_syntax: braces`를 적으면 중괄호형만 인정하고, 예전 표기를 만나면 **경고를 찍는다.** 조용히 문단으로 퇴화하지 않으므로 기존 원고를 옮길 때 자리를 찾을 수 있다.
  - **예전 표기가 인식되는 조건**은 셋이다. (1) `::` 앞에 공백이 하나 이상 있고, (2) `::`가 더 긴 콜론 덩어리(`:::` 등)의 일부가 아니며, (3) 용어가 비어 있지 않을 것. 그래서 `std::vector`, `12::30`, `:::{name}` 같은 것은 걸리지 않는다. 인라인 코드 스팬 안의 `::`도 무시하므로 `` `용어 :: 설명` ``처럼 문법 자체를 예로 드는 문장도 안전하다.
  - 그래도 `그는 말했다 :: 아니다`처럼 **공백으로 둘러싸인 `::`가 산문에 들어가면** 예전 표기로 오인된다. 이 경우는 문법상 구분할 방법이 없다. 그런 문장이 나올 만한 원고라면 `description_syntax: braces`로 아예 막는 편이 낫다.
  - **설명 리스트도 첫 항목에 `{옵션}` 블록을 쓸 수 있다.** itemize/enumerate와 똑같은 `{!옵션}` 문법(11장 위쪽 참고)을 설명 뒤(`::` 다음)에 쓰면 `\begin{description}[옵션]`으로 뽑힌다. Beamer에서 `\begin{description} [<+-| alert@+>]`처럼 오버레이 기본형을 설명 리스트 자체에 줘야 할 때 이 자리가 필요하다. 예시는 아래를 볼 것.

```
{uncover} ::  {!"<+-| alert@+>"} 주어진 슬라이드에서 visible.
{only} ::  주어진 슬라이드에서 show.
```

```
\begin{description}[<+-| alert@+>]
\item[uncover] 주어진 슬라이드에서 visible.
\item[only] 주어진 슬라이드에서 show.
\end{description}
```

옵션 블록은 그 항목의 **설명에서 뽑혀 사라진다** — 용어(`{uncover}`)에는 영향이 없다. 쉼표를 포함한 옵션을 통째로 넣고 싶으면 itemize/enumerate와 같은 큰따옴표 표기(`{!"a,b,c"}`)를 그대로 쓸 수 있다.

  - **항목이 진짜 대괄호로 시작하면, 마커와 대괄호 사이의 공백 유무로 뜻이 갈린다.** LaTeX은 `\item` 바로 뒤에 오는 `[...]`를 그 항목의 커스텀 라벨로 삼는다 — 편리한 기능이지만, 항목 본문을 정말로 대괄호로 시작하고 싶을 때는 훼방꾼이 된다(`\item \relax [...]`로 막는 것이 LaTeX 자체의 특이 케이스다). 이 방언은 마커에 대괄호를 붙여 쓰는지 띄어 쓰는지로 이 둘을 구분한다. 예시는 아래를 볼 것.

```
-[정의] 이 대괄호는 라벨이 된다
- [보통 텍스트] 이 대괄호는 그냥 본문이다
```

```
\item [정의] 이 대괄호는 라벨이 된다
\item \relax [보통 텍스트] 이 대괄호는 그냥 본문이다
```

이 판정은 **인라인 처리를 다 마친 뒤 대괄호가 실제로 리터럴로 남아 있을 때만** 적용된다 — `- [표시](url) 로 시작하는 항목`처럼 대괄호가 링크·각주 등 다른 문법으로 이미 소비됐다면 `\relax`는 끼어들 이유가 없다(문제 자체가 없으므로). 중첩 리스트에서도 같은 규칙이 그대로 적용된다. `enumerate`(번호 리스트)도 동일하다 — `\item[라벨]`은 번호 리스트에서도 똑같이 커스텀 라벨이기 때문이다.

---

12. 인용구(blockquote)

> 인용문 첫 줄
> 인용문 둘째 줄

내부는 다시 `render_blocks`로 재귀 처리된다. **단, 그냥 이어서 > 줄만 쭉 쓰면 전부 하나의 문단으로 합쳐진다.** 인용구 안에 리스트·헤딩 등 별개의 블록을 넣으려면, 일반 문단처럼 빈 줄로 구분해야 하는데 인용구 안에서는 그 "빈 줄"도 > 단독 줄로 표시해야 한다.

> 첫 문단
>
> - 인용구 안 리스트 항목 *A*
> - 인용구 안 리스트 항목 *B*

(> 뒤에 아무것도 없는 줄이 있어야 두 번째 문단/리스트가 별개 블록으로 분리된다. 이게 없으면 리스트 마커 -까지 그냥 평문 텍스트로 흡수되어 버린다.)

12.1. `quote`와 `quotation`

감싸는 환경은 **인용이 몇 문단인가**에 따라 갈린다.

인용	환경
한 문단	`quote`
두 문단 이상 (가운데에 > 단독 줄이 있음)	`quotation`

LaTeX의 관례가 그렇다. `quote`는 문단 들여쓰기가 없어 짧은 한 문단 인용에 맞고, `quotation`은 각 문단의 첫 줄을 들여쓰므로 여러 문단짜리 긴 인용에 맞는다.

> 첫 문단이다.
>
> 둘째 문단이다.

```
\begin{quotation}
첫 문단이다.

둘째 문단이다.
\end{quotation}
```

- 판정은 **처음과 끝의 빈 줄을 뺀** 범위에서 한다. > 단독 줄로 시작하거나 끝나는 인용은 여전히 `quote`다.

- 진짜 빈 줄(>도 없는 줄)은 블록 자체를 끊어 **인용구 두 개**가 된다. 한 인용 안의 문단 구분은 반드시 > 단독 줄로 적어야 한다.
- 환경 이름은 mapping.yaml에서 바꿀 수 있다. 둘을 같은 값으로 두면 이 구분이 사라진다.

```
blockquote_env: quote
blockquote_multi_env: quotation
```

---

13. 구분선

```
---     ← \noindent\rule{\linewidth}{0.5pt} (기본값, mapping.yaml의 hr 로 변경 가능)

***     ← \bigskip (기본값, mapping.yaml의 asterism 으로 변경 가능)
```

한 줄 전체가 -만 3개 이상이거나 *만 3개 이상이어야 한다.

문장 중간의 ---(독립된 줄이 아니라 다른 텍스트와 같은 줄에 섞여 있는, 앞뒤에 다른 대시가 붙지 않은 정확히 3개짜리)는 위 구분선과는 다르게 처리된다: 기본값 \textemdash{}로 치환된다(원고에서 em-dash 효과로 흔히 쓰이기 때문). 표준 LaTeX 명령이므로 preamble에 아무것도 정의하지 않아도 그대로 컴파일된다.

```
이것은 문장 중간에---강조를 위해---쓰이는 경우다.
→ 이것은 문장 중간에\textemdash{}강조를 위해\textemdash{}쓰이는 경우다.

이건 en dash 스타일 -- (2개)나 ---- (4개 이상)는 건드리지 않는다.
```

세 가지(가로줄 hr, asterism asterism, 문장 중 대시 mdash) 모두 mapping.yaml로 바꿀 수 있다. DEFAULT_CONFIG에는 없고 코드에 폴백으로만 있으므로, sample_mapping.yaml의 해당 절(주석 참고)을 보고 필요한 것만 적으면 된다.

```
hr: '\hrulefill'                    # 더 가벼운 가로줄
asterism: '\fancybreak{* * *}'      # 별 세 개를 실제로 조판(memoir, preamble에 정의 필요)
mdash: '\ndhmdash{}'                # 자체 매크로(preamble에 정의 필요)
```

---

14. 표

```
| 이름 | 나이 | 비고 |
|:---|:---:|---:|
| 철수 | 20 | 왼쪽/가운데/오른쪽 정렬 |
| 영희 | 21 | 두 번째 줄이 구분선(`---`)이어야 표로 인식됨 |
```

- 첫 줄=헤더, 둘째 줄=정렬 구분선(:---, :---:, ---:, ---), 그 다음이 본문 행. \begin{tblr}{정렬}...\end{tblr} (booktabs 스타일 \toprule / \midrule / \bottomrule 사용, tabularray 패키지 필요)로 변환된다.

- 열 개수는 **헤더 행의 열 개수**로 고정되며, 본문 행이 모자라면 빈 칸으로 채우고 넘치면 잘린다.
- **긴 셀은 자동으로 줄바꿈된다.** 표의 추정 폭이 줄 폭을 넘을 것 같으면 폭이 큰 열(대략 한글 8자 이상)만 `l/c/r` 대신 tabularray의 `X[l]`/`X[c]`/ `X[r]`(폭 비율을 `co=`로 준다)로 바꿔서 그 열 안에서 줄바꿈되게 한다. 추정 폭은 한글·CJK 한 글자를 2, 그 밖을 1로 세는 근사값이고, 이 값이 기준 `tablewrap`(기본 70 ≈ 한글 35자) 이하인 표는 **전혀 바뀌지 않는다.** `X` 열이 하나라도 있으면 tabularray가 표를 줄 폭 전체로 늘리기 때문에, 넘칠 것 같은 표에만 적용하는 것이다. 폭이 큰 열이 없는데 짧은 열만 많아 넘치는 표는 이 규칙으로 해결되지 않는다(raw `tblr`로 직접 짠다). `tablewrap`은 **mapping 키**다(front matter가 아니다): `mapping.yaml` 또는 `mappingrule:`에서 `tablewrap: 60`처럼 바꾸고, `0`/`false`면 끈다. 이 규칙은 파이프 표에만 적용되며 raw `tblr`/`tabular`는 건드리지 않는다.
- **표 속성 줄 — 쪽나눔 표(`longtblr`).** 마지막 행 **바로 다음 줄**(빈 줄 없이)에 `{…}`만 쓴 줄을 두면 그 표의 속성이 된다.

```
| 구분 | 두보 | 이하 |
|:---:|:---:|:---:|
| … | … | … |
{env=long, caption="두 시인의 비교", label=tbl:cmp}
```

`env=long`(또는 `longtblr`)이면 `\begin{longtblr}[caption={…},label={…}] {colspec={…},rowhead=1}`로 나가서 표가 쪽을 넘어 이어지고, 머리글 행이 쪽마다 되풀이된다(`rowhead=0`으로 끔). `caption`/`label`은 `longtblr` 자신의 선택 인자라서 float 래퍼(`:::{table}`) 없이도 캡션과 `[#tbl:cmp]` 참조가 된다. `env`를 안 주거나 `tblr`이면 지금처럼 `tblr`이고, 이때 `caption`/`label`은 쓸 곳이 없어 경고하고 무시한다(캡션이 필요하면 `:::{table}`). 알 수 없는 키도 경고한다. 속성 줄과 표 사이에 빈 줄을 두면 속성이 아니라 평범한 문단이다. `longtblr`는 쪽 나눔을 맞추려 두 번 컴파일되어야 하는데 `--latexmk`나 ksruncltex는 알아서 되풀이한다. 위 `tablewrap`의 `X` 열 자동 줄바꿈도 그대로 적용된다. 본문 앞뒤 간격을 `\AddToHook{env/tblr/before}{…}`로 잡아 둔 문서라면 `longtblr`용으로 `env/longtblr/before`/`after` 훅도 같이 걸어야 간격이 맞는다(기본 헤더에는 그런 훅이 없다). - 셀 안에 세로줄 문자를 그대로 쓰려면 `\|`로 이스케이프한다. 일반 마크다운과 같은 규칙이고, 구분자로 쓰이지 않은 채 그 자리에 `|`가 남는다. - 이미 직접 짠 raw `tabular`/`tblr` 등은 8장의 raw 환경 규칙으로 그대로 보호되어 통과한다.

14.1. grid 표 — 병합 셀과 여러 줄 셀

pandoc의 grid table도 읽는다. 셀 병합(행·열)과 여러 줄 셀이 필요할 때 쓴다. 파이프 표와 달리 셀 안에 `|`를 이스케이프 없이 쓸 수 있다.

```
+---------------------+-----------------------+
| Location            | Temperature 1961-1990 |
|                     | in degree Celsius     |
|                     +-------+-------+-------+
|                     | min   | mean  | max   |
```

```
+====================+=======+=======+=======+
| Antarctica         | -89.2 | N/A   | 19.8  |
+--------------------+-------+-------+-------+
```

Location	Temperature 1961-1990 in degree Celsius		
	min	mean	max
Antarctica	-89.2	N/A	19.8

- **인식:** 표의 첫 줄이 `+----+----+` 꼴이면 grid 표 후보다. 앞뒤에 빈 줄이 필요하다(파이프 표와 같다).
- **머리글·바닥글:** 첫 `=` 줄 위쪽이 머리글(여러 줄 가능)이고, 두 번째 `=` 줄이 있으면 그 아래가 바닥글이다(`\midrule`로 나뉜다). `=` 줄이 없으면 머리글이 없는 표다.
- **정렬:** 머리글 구분선(`=` 줄)의 콜론으로 정한다 — `+:====+`(왼쪽), `+:===:+`(가운데), `+====:+`(오른쪽). 머리글이 없으면 맨 윗줄의 콜론을 본다. pandoc과 같은 규칙이다.
- **선:** `-` 줄을 모두 긋지는 않는다. 파이프 표와 같이 booktabs 세 줄(위, 머리글 아래, 아래)만 긋고, 머리글 안쪽의 부분 구분선(위 예의 `Temperature` 아래)만 `\cmidrule`로 옮긴다.
- **셀 안의 글:** 줄은 공백으로 합쳐지고, 빈 줄은 문단 나눔(`\par`)이 된다. 셀 안에는 평문과 인라인 서식만 쓸 수 있다 — 리스트·인용 등 블록 요소는 아직 해석하지 않는다.
- **열 너비:** 기본은 **자연 너비**(정렬 글자만)이며, 파이프 표와 같이 넘칠 열만 `X`로 바꾸는 `tablewrap`이 적용된다. pandoc처럼 테두리의 대시 수 비율로 모든 열을 `X[정렬,co=폭]`로 두려면 표 바로 뒤 줄에 `{widths=dash}`를 쓴다.
- **속성 줄:** 파이프 표와 같다. `{env=long, caption=…, label=…, rowhead=N}`. `rowhead`의 기본값은 머리글 줄 수다. `widths`는 grid 표에서만 쓴다.
- **한글:** 열 경계는 문자 번호가 아니라 **화면 칸**(한글 2칸)으로 맞춘다. 고정폭 에디터에서 눈으로 맞춘 표가 그대로 읽힌다. 화면 칸으로 해석되지 않으면 한글을 1칸으로 센 표도 읽는다.
- **오류:** 모서리가 닫히지 않았거나 `|` 위치가 위아래 줄과 어긋난 표는 `--lint`가 오류로 알리고(몇 번째 줄인지 포함), 변환하면 표가 아니라 평범한 문단이 된다(경고가 함께 나온다).

---

15. 그림

```
![캡션 텍스트](path/to/image.png){width=0.75\linewidth, label=fig:foo, place=htbp}

![](path/to/image.png)   ← 캡션·라벨 없이 \includegraphics 만 출력

![""](path/to/image.png) ← figure 환경으로 감싸되 \caption 은 없음
```

- `{...}` 안은 쉼표로 구분된 `key=value` 또는 단독 키. `label`, `place`, `align` 셋은 특별 취급되어 각각 `\label{}`, `\begin{figure}[place]`의 배치 옵션, 그림 앞의 정렬 명령으로

빠지고, **나머지는 모두 `\includegraphics[...]`의 옵션으로 들어간다**. `place` 기본값은 `htbp`, `align` 기본값은 `\centering`이다.

- `align` 값은 raw로 나가므로 LaTeX 명령을 그대로 적는다 (`align="\raggedleft"`). `align=""`처럼 빈 값을 주면 정렬 명령 자체가 빠진다. 17.1의 `:::{figure}` 환경과 같은 규칙이다.
- 특별 취급되는 셋 말고는 전부 `\includegraphics`로 넘어간다는 점을 기억할 것. graphicx가 모르는 키를 주면 `Package keyval Error: ... undefined`로 컴파일이 깨진다.
- 대체텍스트와 `label`에 따라 세 갈래로 갈린다.

원고	출력
`![](x.jpg)`	`\includegraphics`만. 플로트로 감싸지 않는다
`![텍스트](x.jpg)`	`figure` 환경 + `\caption{텍스트}`
`![""](x.jpg)`	`figure` 환경, `\caption` 없음

- 세 번째가 **캡션 없는 그림 플로트**를 만드는 방법이다. 대체텍스트가 빈 따옴표 한 쌍(`""`, `''`, `“”`, `‘’`)뿐일 때만 적용된다. 따옴표 안에 무엇이든 들어 있으면 그냥 캡션이다.
- **따옴표는 캡션 구분자가 아니다.** `!["텍스트"](x.jpg)`는 따옴표까지 포함한 `\caption{``텍스트''}`가 된다(17장의 스마트 따옴표 변환을 거친다). 특별 취급되는 것은 속이 완전히 빈 따옴표 쌍뿐이다.
- **대체텍스트 자리에 링크를 하나 중첩할 수 있다.** `![[표시](url)](x.jpg)`처럼 쓰면 그림 캡션 자체가 링크가 된다(예시는 아래). 대괄호를 한 겹까지만 품으므로 그 이상 중첩하면 인식되지 않는다. `url` 안의 `%`는 자동으로 이스케이프된다(16장 참고). 이 표기는 대체텍스트 자리에서만 동작하고, `caption=` 속성 값 안에서는 `[표시](url)`를 다시 처리해 주지 않는다 — 속성 값은 그 환경의 raw 텍스트라서, 링크가 필요하면 `\href{...}{...}`를 직접 적어야 한다.
- **`caption=` 속성과 대체텍스트는 다른 것을 뜻한다.** 대체텍스트는 `figure`로 띄우고, `caption=`은 띄우지 않고 그 자리에 두면서 캡션만 단다(dry caption). 아래 표 참고. `\captionof`는 `captof`(또는 `caption`) 패키지가 주며, 번호와 `\label`은 떠다니는 그림과 같은 흐름을 이어받는다. 로딩은 저자 몫이다.

  ```
  ![[원주율](https://ko.wikipedia.org/wiki/%EC%9B%90%EC%A3%BC%EC%9C%A8)](pi.png)
  ```

```
\begin{figure}[htbp]
\centering
\includegraphics{pi.png}
\caption{\href{https://ko.wikipedia.org/wiki/%EC%9B%90%EC%A3%BC%EC%9C%A8}{원주율}}
\end{figure}
```

- `inline` 단독 키를 주면 `\inlinegraphics`로 나간다. 줄글 중간에 작은 그림을 끼울 때 쓰며 `inlinegraphicx` 패키지가 필요하다(로딩은 저자 몫). `inlinespec=` 값은 그 명령의 대괄호 인자로 **그대로** 넘어가고, 없으면 다른 속성들이 그 자리에 들어간다. 명령 이름은 `mapping.yaml`의 `inline_graphics_cmd`로 바꾼다.

- **wrap** 단독 키를 주면 떠다니는 figure 대신 본문이 그림을 둘러싸는 배치가 된다(wrapstuff 패키지 필요 — 로딩은 저자 몫).

```
![](x.png){wrap, wrapspec="type=figure,width=5cm", caption="test",
           label="fig:wrap", width=3cm, align="\centering"}
본문이 이어진다. 그림 줄 다음에 빈 줄을 두지 않는다.
```

```
\begin{wrapstuff}[type=figure,width=5cm]
\centering
\includegraphics[width=3cm]{x.png}
\caption{test}
\label{fig:wrap}
\end{wrapstuff} 본문이 이어진다. ...
```

- wrapspec= 값은 환경의 대괄호 인자로 **그대로** 넘어간다. 이 스크립트가 해석하지 않으므로 환경이 무엇을 받든 저자가 정한다. 값에 쉼표가 들어가므로 반드시 따옴표로 감쌀 것.
- 이 배치는 **환경이 문단 첫머리에 와야** 한다. 원고에서 그림 줄 바로 다음에 빈 줄 없이 본문을 이어 쓰면 그렇게 된다 — 그림 마크업은 인라인으로 처리되어 뒤따르는 본문과 같은 문단에 남기 때문이다.
- wrap일 때 place는 쓰이지 않는다(떠다니지 않으므로). 환경 이름은 mapping.yaml의 wrap_env로 바꿀 수 있다.
- **href=** 를 주면 그림 자체에 링크가 걸린다(아래 참고). 캡션이나 라벨이 아니라 \includegraphics만 감싼다. figure로 감싸질 때도, wrap 배치일 때도 마찬가지다. URL에 %, #, &가 들어가도 hyperref가 알아서 다루므로 그대로 적으면 된다. \href 는 hyperref가 주므로 로딩은 저자 몫이다(기본 헤더에 넣어두면 편하다).
- \includegraphics로 넘어가는 옵션 중 center 같은 것은 graphicx가 아니라 adjustbox(\usepackage[expo가 제공한다. 역시 저자 몫이다.
- 캡션 없이 label만 주면 \label이 번호를 이어받지 못해 \ref가 엉뚱한 값(직전 카운터 값)을 낸다. 경고를 찍는다. 띄우지 않으면서 참조하고 싶으면 caption=을 함께 줄 것 — \captionof가 번호를 맡는다.
- 캡션 텍스트는 재귀적으로 process_inline을 다시 거치므로 캡션 안에서도 굵게·기울임·따옴표 변환 등이 모두 정상 동작한다.

캡션과 띄우기:

띄울지 말지는 대체텍스트 자리로만 정한다. 대괄호가 비어 있으면 {} 안에 무엇을 적든 figure로 감싸지 않는다.

원고	결과
![캡션](x.png)	figure 환경 + \caption
![""](x.png)	figure 환경, 캡션 없음
{caption="캡션"}	그 자리에 + \captionof{figure}
	\includegraphics만

```
![](myfig.png){caption="dry caption", label=fig:d}
```

```
\includegraphics{myfig.png}
\captionof{figure}{dry caption}
\label{fig:d}
```

줄글 중간의 작은 그림:

```
![](smallfig.png){inline, inlinespec="scale=1.5,strut={()}"}
```

```
\inlinegraphics[scale=1.5,strut={()}]{smallfig.png}
```

그림에 링크 걸기:

```
![](image.png){scale=.3, href="https://ctan.org"}
```

```
\href{https://ctan.org}{\includegraphics[scale=.3]{image.png}}
```

15.1. 그림 내려받기와 형식 변환

세 가지를 서로 다른 스위치로 나눠 두었다.

걱정	스위치	기본
네트워크 접근	`--fetch-images`	꺼짐
형식 변환	없음 (Pillow가 있으면 한다)	켜짐
애니메이션	`--animation[=N]`	꺼짐

다운로드만 옵트인인 이유는 재현성이 아니라 **보안**이다. 남이 준 `.md`를 변환하는 것만으로 임의 주소에 요청이 나가면 곤란하다. 반면 형식 변환은 파일 하나를 더 만드는 일일 뿐이고, 로컬 `.webp`를 쓴 사람이 URL도 안 쓰면서 다운로드 옵션을 켜야 하는 것은 이상하다.

URL 그림

```
python3 ndhmd4tex.py 회람.md --pdfxe --fetch-images
```

평소 명령에 플래그 하나만 더하면 된다. 받은 파일은 원고 옆 `_images/`에 **URL 해시 + 실제 확장자**로 저장된다. URL 마지막 조각을 쓰지 않는 이유는 `330px-...` 같은 이름이 출처마다 겹치고, 퍼센트 인코딩과 유니코드가 파일 이름이 되면 그것대로 골칫거리이기 때문이다.

- 이미 받아둔 것이 있으면 네트워크를 쓰지 않는다. **두 번째 빌드부터는 오프라인에서도 된다.**
- **`--fetch-images` 없이도 `_images/`에 받아둔 것이 있으면 그것을 쓴다.** 이 옵션이 막으려는 것은 바깥으로 나가는 요청이지, 이미 손에 있는 파일을 쓰는 일이 아니다. 그래서 한 번 받아둔 뒤로는 평소 명령 그대로 빌드된다.
- 확장자는 `Content-Type`을 먼저 믿는다. URL 경로의 확장자는 없거나 틀릴 수 있고, 404 페이지가 200으로 오기도 한다.
- `--fetch-images`도 없고 받아둔 것도 없으면 그 그림만 원문 그대로 두고 오류를 알린다. 변환 자체는 계속된다.

`\graphicspath`로 위치를 조절할 수 있으니 `_images/`가 거슬리면 옮겨도 된다.

경로는 **언제나 슬래시로** 내보낸다. LaTeX 소스의 경로 구분자는 운영체제와 무관하게 `/`이고, 윈도우에서 역슬래시가 섞여 나가면 `_images\dn.jpg`의 `\d`가 정의되지 않은 명령으로 읽혀 컴파일이 깨진다.

형식 변환

LaTeX이 그대로 받는 것은 `.pdf .png .jpg .eps .bmp` 정도다. (`.bmp`가 되는 것은 실측 결과다 — 흔한 오해와 달리 xelatex은 받는다.)

형식	처리
`.tif .tiff .webp .gif`	Pillow로 PNG 변환
`.svg .ai .psd`	받아만 두고 알림

변환본은 **원본 옆에 나란히** 만든다. `사진.webp` → `사진-webp.png`. 원본 확장자를 이름에 남기는 것은 `p.webp`와 `p.tiff`가 둘 다 `p.png`가 되어 서로 덮어쓰는 것을 막기 위해서다. 원본을 지우지 않으므로 나중에 더 나은 도구로 다시 변환할 여지가 남고, 링크가 죽어도 복구할 수 있다.

Pillow는 선택적 의존성이다. 없어도 도구는 정상 동작하며, 변환이 필요한 형식을 만났을 때만 설치를 안내한다. 필요하면 `pip install Pillow`.

`.svg`는 벡터라 래스터화가 손해이고, PDF로 바꾸려면 inkscape나 rsvg-convert가 있어야 하는데 그 보장이 없다. 그래서 변환하지 않고 파일만 손에 쥐어준다 — PDF로 바꾸든, `\usepackage{svg}`와 `-shell-escape`를 쓰든, 다른 그림으로 바꾸든 저자가 고르면 된다.

애니메이션 (내세울 것은 못 된다)

재생되는 뷰어가 거의 없다. `animate` 패키지는 JavaScript를 쓰므로 Adobe Acrobat이나 PDF-XChange 정도에서만 움직이고, Preview·Skim·Okular·브라우저 내장 뷰어에서는 첫 프레임만 보인다. 그것을 알고 쓸 것.

```
python3 ndhmd4tex.py 글.md --animation      # 앞에서부터 40 프레임
python3 ndhmd4tex.py 글.md --animation=12   # 앞에서부터 12 프레임
python3 ndhmd4tex.py 글.md --animation=all  # 전부 (파일 수는 감당할 것)
```

```
\animategraphics[width=3cm]{10}{anim-gif-frames/}{0}{4}
```

- 프레임은 **원본이름**`-gif-frames/`에 `0.png`부터 저장된다. 프레임 하나가 파일 하나이므로 30프레임 GIF 세 개면 90개가 쌓인다. 기본 상한 40은 그래서 있다.
- **프레임별 지연은 살릴 수 없다.** GIF는 프레임마다 지연이 다를 수 있는데 `\animategraphics`는 단일 fps만 받는다. 값은 `mapping.yaml`의 `animation_fps`로 바꾼다(기본 10).
- 앞에서부터 자른다. 고르게 솎으면 루프는 온전해지지만 타이밍이 한 번 더 뭉개지는데, 이미 fps로 한 번 뭉갠 뒤다.
- `--animation` 없이 애니메이션 GIF를 만나면 첫 프레임만 쓰고 그렇게 알린다.

- **이미 뽑아둔 프레임은 재활용한다.** 필요한 개수만큼 이미 있으면 다시 인코딩하지 않는다. `--animation=all`처럼 상한을 늘려 부족분이 생기면 처음부터 다시 뽑고, 상한을 줄이면 이미 있는 것을 그대로 쓴다(그 폴더에 남는 초과분 프레임 파일은 무해하다). 원본 GIF를 바꿨는데 새 버전이 우연히 프레임 수까지 같다면 갱신되지 않으니, 그럴 땐 `*-frames/` 폴더를 지우고 다시 실행할 것.

15.6. 참조로 여러 번 쓰기 — tagged reference

같은 그림(또는 링크)을 여러 번 가리켜야 할 때, MultiMarkdown식으로 경로/URL을 한 곳에 정의해 두고 짧은 라벨로 재사용할 수 있다.

```
![그림 설명][myfig]
...
[myfig]: figurename.png width=3cm,respectratio

자세히 알고 싶으면 [][Markdown]을 보라.
...
[Markdown]: https://en.wikipedia.org/wiki/MultiMarkdown
```

```
\begin{figure}[htbp]
\centering
\includegraphics[width=3cm,respectratio]{figurename.png}
\caption{그림 설명}
\end{figure}
...
자세히 알고 싶으면 \url{https://en.wikipedia.org/wiki/MultiMarkdown}을 보라.
```

- **정의는 `[라벨]: 값` 한 줄**이다(여러 줄로 이어 쓰는 건 지원하지 않는다 — 그림 경로+속성이나 URL이 한 줄을 넘길 일은 실무에서 거의 없다고 보았다. 여러 줄로 쓰면 둘째 줄부터는 그냥 평범한 본문으로 남는다 — 조용히 사라지는 게 아니라 화면에 그대로 보이므로 실수를 바로 알아챌 수 있다). 문서 어디에나(참조보다 앞이든 뒤든) 둘 수 있고, 렌더링되는 본문에서는 빠진다(각주 정의와 같은 대우).
- **사용처마다 다른 `label`/속성은 참조 뒤 `{…}`로 준다.** 정의 줄의 속성은 "이미지의 공유 기본값"(경로, `width` 등)이고, `]` 바로 뒤에 붙인 `{…}`는 "이 그림 인스턴스만의 값"이다 — `![캡션](x.png){…}`와 같은 규칙이다(띄어 쓰면 속성이 아니라 본문).

```
![첫 사용][myfig]{label=fig:a}
![다른 곳][myfig]{label=fig:b,width=60pt}

[myfig]: mmd.png width=100pt
```

같은 키는 사용처가 이긴다. 값을 비우면(`{label=}`) 정의에 있던 그 속성이 지워진다(`align`만은 빈 값이 "정렬 명령 없음"이라는 뜻이라 그대로 덮어쓴다). `\label`은 그림 하나에 하나뿐이어야 하므로, 정의 줄에 `label=`을 두면 그 정의를 **한 번만** 쓸 때만 안전하다 — 두 번 이상 쓰면서 사용

처가 label을 따로 주지 않으면 \label 중복(LaTeX의 "multiply defined")이 되므로 경고한다. 참조의 이름(myfig)은 정의를 찾는 키일 뿐 LaTeX 라벨과는 별개다. 링크 참조([text][라벨])에는 속성이 없으므로 해당 없다. - **그림 참조와 링크 참조는 라벨 풀을 공유한다.** [라벨]: 값 하나를 이미지로도 링크로도 쓸 수 있다 — 어느 쪽으로 해석할지는 참조하는 쪽(![alt][라벨]인지 [text][라벨]인지)이 정한다. 그림 정의의 값은 "경로 + (있으면) 속성"(figurename.png width=3cm,respectratio, 15.1의 {width=...} 속성과 같은 문법을 중괄호 없이 그대로 쓴다), 링크 정의의 값은 URL 하나다. - **대괄호 한 겹짜리 shortcut([Markdown] 단독)은 일부러 지원하지 않는다.** 산문에서 강조나 메모 삼아 대괄호 하나만 쓰는 경우와 구별할 방법이 없어서, 이 방언의 "명시적 문법만 인정한다"는 태도와 어긋난다. 대신 **대괄호 두 겹**(][시퀀스)만 받는다 — [][라벨](표시 텍스트 비움) 또는 [텍스트][라벨]. 이 시퀀스는 일반 산문에 사실상 나타나지 않아 모호함이 없고, [[용어]](색인)·[^id](각주)·[#id](상호참조) 어느 것과도 시작 문자가 달라 충돌하지 않는다. - [][라벨](표시 텍스트를 비움)은 [](url)과 완전히 같은 규칙을 따른다 — 링크라면 \url{...}, 그림이라면 캡션 없는 삽입이 된다. 매번 URL이나 파일명을 두 번 타이핑하지 않아도 되는 게 이 문법의 핵심이다. - 정의되지 않은 라벨을 참조하면(오타 등) 오류로 멈추지 않고 **경고를 띄운 뒤 원문 그대로 남긴다** — ![][없는라벨]이라고 썼다면 결과물에도 ![][없는라벨]이 그대로 나온다. 컴파일 자체를 막지 않으면서도 눈에 띄게 알려주려는 절충이다.

---

16. 링크 / 각주 / 상호참조 / 색인

16.1. 아무 데나 라벨 — {#id}

제목 {label=...}(3장)과 ![캡션](x){label=...}(15장)은 각각 헤딩과 그림 전용 속성 문법이다. 그런데 리스트 항목이나 각주처럼 속성 문법 자체가 없는 자리에 라벨을 달고 싶을 때가 있다. {#id}는 그 자리를 메운다 — **인라인 어디서든** \label{id}로 바뀐다.

- 항목 하나{#item:a}
- 항목 둘

각주 내용도{#fn:a} 라벨을 받는다.

[^f1]: 각주 정의 안에서도{#fn:b} 된다.

참조: [#item:a]

```
\begin{itemize}
\item 항목 하나\label{item:a}
\item 항목 둘
\end{itemize}

각주 내용도\label{fn:a} 라벨을 받는다.
...
```

참조: `\ref{item:a}`

- 라벨 이름은 영숫자·`:`·`_`·`-`·`.` 만 받는다. 그 형식을 벗어나면(공백 등) 라벨로 바뀌지 않고 `{#...}` 글자 그대로 남는다 — 오류 없이 조용히 통과하므로, 라벨이 안 걸린다면 이름에 허용되지 않은 문자가 있는지부터 볼 것.
- 코드 스팬 안(`` `{#not:a}` ``)은 건드리지 않는다.
- 헤딩과 그림에는 이미 전용 문법이 있으므로, 그 자리에서는 계속 `{label=...}`나 `{#...}`(속성으로서의 `#`, 3장 참고)를 쓴다. 이 절의 `{#id}`와는 형태가 비슷해도 다른 자리에서 동작하는 별개의 문법이다.
- **별행수식 바로 뒤에 쓰면 죽은 라벨이 된다.** `\label`은 그 자리에서 가장 최근에 갱신된 카운터를 받아가는데, `align`/`equation`을 벗어나는 순간 그 상태가 사라져서 `\ref`가 빈 값을 낸다. 수식 번호를 참조하려면 라벨은 **환경 안에서** 달 것(raw로 직접 `\label{...}`을 적으면 된다). 이 절의 `{#id}`는 산문·리스트·표처럼 그 위치 자체가 참조 대상인 자리를 위한 것이지, 별행수식의 대체 문법이 아니다.

```
\begin{align}
a &= b + c \label{eq:good}    ← 여기서는 정상
\end{align}
{#eq:bad}                     ← 여기서는 죽은 라벨
```

```
[표시 텍스트](https://example.com)
[표시 텍스트](https://example.com "제목은 파싱만 되고 버려짐")
[](https://example.com)              ← 빈 라벨이면 \url{} 로
[](#sec:intro)                       ← url 자리가 #으로 시작하면 라벨 링크
[여기 보기](#sec:intro)              ← \hyperref[sec:intro]{여기 보기}
```

`(#라벨)`처럼 괄호 안이 `#`으로 시작하면 외부 URL이 아니라 문서 안 라벨을 가리키는 것으로 본다. 진짜 URL은 `http://`, `https://`, `mailto:`, 상대경로 파일명 등으로 시작하지 `#`으로 시작하는 법이 없으므로 구분에 모호함이 없다. `\ref`가 아니라 `\hyperref[라벨]{...}`를 쓰는 이유는 뜻이 다르기 때문이다 — `\ref`는 번호만 내놓지만, 이 문법이 원하는 것은 "그 절로 가는 클릭 가능한 링크" 자체다. 라벨이 비어 있으면(`[](#id)`) 라벨 이름 자체를 링크 텍스트로 쓴다. `hyperref` 로딩은 저자 몫이다. `[#라벨]`(대괄호 하나짜리 상호참조 문법, 아래 참고)과는 형태가 아예 달라 충돌하지 않는다.

```
본문에서 각주 참조[^note1]를 단다.

[^note1]: 각주 내용은 여기.  ← id 는 라틴 알파벳·숫자·_·-·한글만
    두 번째 줄부터는 스페이스 4칸(또는 탭)으로 들여써서
    이어서 쓸 수 있다.

본문에서 바로[^이렇게 각주 본문을 적어도 된다] 쓸 수 있다.
본문에서 바로^[Pandoc 표기도 같은 뜻이다] 쓸 수도 있다.
```

```
[#sec:intro] 를 참고하라.   → \ref{sec:intro}
[page:sec:intro] 쪽 참고.   → \pageref{sec:intro}
[@smith2020] 인용.          → \cite{smith2020}
[@t:smith2020]은 말했다.     → \citet{smith2020}     (natbib/biblatex)
근거가 있다 [@p:smith2020].  → \citep{smith2020}
[@a:smith2020] ([@y:smith2020]) → \citeauthor{} (\citeyear{})
[title:sec:intro] 참고.     → \titleref{sec:intro}
[name:sec:intro] 참고.      → \nameref{sec:intro}
[E:eq:euler] 참고.          → \eqref{eq:euler}      (amsmath)
[F:fig:sample] 참고.        → \figurename~\ref{fig:sample}   → 그림 1
[T:tab:sample] 참고.        → \tablename~\ref{tab:sample}    → 표 1
[C:sec:intro] 참고.         → \Cref{sec:intro}
[C:sec:a,sec:b] 참고.       → \Cref{sec:a,sec:b}   (cleveref의 콤마 다중 라벨)

[[Term]]                    → Term\index{Term}
[[표시|색인어]]              → 표시\index{색인어}
[[|색인어만]]                → \index{색인어}
[[TeX|typesetting!TeX]]     → TeX\index{typesetting!TeX}   (makeindex 계층)
[[LaTeX|LaTeX@\LaTeX]]      → LaTeX\index{LaTeX@\LaTeX}    (makeindex 정렬키)
```

- **`[표시](url)`의 `url` 안 `%`는 자동으로 `\%`로 이스케이프된다.** `\href`가 hyperref의 verbatim 스캔 트릭으로 첫 인자를 읽는데, 그 트릭은 인자가 최상위 토큰 스트림에서 바로 읽힐 때만 통한다. 이 링크가 `\caption{...}` 처럼 **한 번 더 매크로 인자로 감싸이면**(15장의 `caption=` 속성이 바로 이 경우다) verbatim 스캔이 아니라 보통의 매크로 확장을 거치므로, `url` 안의 `%`부터 그 줄 나머지가 LaTeX 주석으로 통째로 사라진다. 최종 PDF의 실제 링크 대상에는 영향이 없다 — `\%`는 hyperref 안에서 원래의 `%`로 복원된다. 빈 라벨(`\url{}`)은 애초에 이 문제가 없어 손대지 않는다.
- **본문에 직접 적은 `\href{...%...}{...}`는 이 보호를 받지 못한다.** 예를 들어 `caption="..."` 속성 값 안에 저자가 raw LaTeX으로 `\href`를 직접 적었다면, 그 안의 `%`는 스크립트가 건드리지 않는 raw 텍스트이므로 저자가 직접 `\%`로 이스케이프해야 한다.
- **같은 링크(또는 그림)를 여러 번 써야 한다면 `[텍스트][라벨]` 형태의 참조를 쓸 수 있다** — URL이나 파일명을 한 곳에만 적어 두고 라벨로 재사용한다. 자세한 내용은 15.6절.
- **색인 명령은 `index_template`으로 통째로 바꿀 수 있다.** TeX의 `\index`는 인자를 특수하게 읽는 탓에 사용자가 재정의하기 몹시 까다로운데, 직접 정의한 매크로를 쓰고 싶은 경우가 있어서다. `%(text)s`에 표시 텍스트가, `%(term)s`에 색인어가 들어간다. 표시 텍스트를 어디에 놓을지까지 템플릿이 정하므로, `[[|색인어만]]`처럼 표시가 없는 경우에는 `%(text)s`가 빈 문자열이 되어 세 경우가 템플릿 하나로 처리된다. 알 수 없는 자리표시자를 쓰면 경고를 찍고 기본 `\index` 형태로 폴백한다. 예시는 아래 참고.

- **색인어와 표시 텍스트는 raw 그대로 나간다** — 이스케이프도, 인라인 서식 처리도 거치지 않는다. makeindex가 `\index` 인자 안에서 `!`(계층), `@`(정렬키), `|`(서식)을 특수하게 쓰기 때문에 손대면 그 문법을 못 쓰게 된다. 대신 색인어에 `%`나 `&` 같은 문자를 넣어야 한다면 직접 이스케이프해야 하고, `[[**굵게**|색인]]`처럼 표시 텍스트에 마크다운 서식을 써도 변환되지 않는다.
- **별행 정의의 식별자에는 라틴 알파벳·숫자·밑줄(_)·붙임표(-)·한글 음절만 쓸 수 있다**(`[A-Za-z0-9_\-가-힣]+`). `[^note1]`, `[^ndh-2024]`, `[^fn_3]`, `[^주석1]`은 되고 `[^두 단어]`(공백 포함)는 안 된다. 공백을 뺀 것이 핵심이다 — `[^...]`는 "참조"와 "그 자리에서 바로 쓰는 인라인 각주 본문"을 똑같은 대괄호 모양으로 겸하는데(바로 아래 참고), 이 둘을 가르는 것은 오직 "안의 문자열이 이미 정의된 식별자와 일치하는가"뿐이다. 공백이 들어가면 "짧은 참조 식별자"와 "여러 단어짜리 즉석 각주 본문"이 구분이 안 된다 — 반대로 공백 없는 한글은 ASCII 식별자와 성격이 같아서 이런 모호성 없이 안전하게 허용된다.
- 이 규칙을 어기면 **조용히 이중으로 깨진다.** 정의 줄이 수집되지 않아 본문에 그대로 남고, 그 줄마저 인라인 각주 규칙에 걸려 `\footnote`로 둔갑한다. 참조 쪽도 정의를 못 찾아 식별자 자체가 각주 본문이 된다. 그래서 정의처럼 생겼는데 식별자가 규칙을 벗어난 줄을 만나면 경고를 찍는다 — 각주 번호가 이상하게 늘어났다면 이 경고부터 확인할 것.
- **각주는 참조와 인라인 두 가지로 쓸 수 있다.** `[^...]` 안의 내용이 수집된 각주 정의의 식별자와 일치하면 그 정의를 가리키는 참조이고, 아니면 그 내용을 각주 본문으로 그대로 읽는다. 정의 식별자는 `[A-Za-z0-9_-]+`로 제한되므로 한글이나 공백이 섞인 내용은 참조일 수가 없다 — 두 용법이 서로 헷갈릴 일이 없는 이유다.
- Pandoc과 같은 `^[각주 본문]` 표기도 받는다. 참조와 헷갈릴 여지가 아예 없는 형태라, 새로 쓰는 원고에서 의도를 분명히 하고 싶을 때 쓴다. `^^방점^^`과는 충돌하지 않는다(대괄호 유무로 갈린다).
- 각주 본문에는 인라인 서식이 그대로 먹는다. `[^**굵게**와 *기울임*]`은 `\footnote{\textbf{굵게}와 \emph{기울임}}`이 된다. 대괄호도 **한 겹까지** 들어가므로 `[^각주에 [#sec:a] 참조]`처럼 상호참조를 넣을 수 있다. 두 겹 이상 중첩하거나 줄바꿈을 넣는 것은 안 된다.
- **`^[가 방점(^^...^^) 바로 뒤에 붙으면 각주로 보지 않는다.`** `^^르베그 적분^^[[|르베그 측도]]`에서 닫는 캐럿의 뒤엣것이 대괄호와 붙어 각주로 오인되던 것을 막기 위해서다. 덕분에 방점 뒤에 색인이나 상호참조가 바로 오는 조합(`^^강조^^[#sec:a]`)이 모두 정상 동작한다. 방점 뒤에 진짜 인라인 각주를 붙이려면 사이를 한 칸 띄운다 — `^^방점^^ ^[각주]`.
- **주의**: 참조 식별자에 오타를 내면 경고 없이 그 오타가 각주 본문으로 조판된다. `[^ndh2024x]`가 "ndh2024x"라는 각주가 되는 식이다. 이 동작이 부담스러우면 `mapping.yaml`에서 `inline_footnote: false`로 끄면 되고, 그러면 예전처럼 정의되지 않은 참조에 경고를 찍고 원문을 그대로 남긴다.
- 각주 정의는 **문서 아무 곳에나** 둘 수 있고, 참조보다 먼저 나오지 않아도 된다(변환 전 단계에서 전부 수집됨). 정의되지 않은 `[^id]`를 참조하면 stderr에 경고만 찍고 원문 그대로 둔다.

- `E:`는 amsmath의 `\eqref`다. 프리픽스 이름을 `eq:`로 하지 않은 데는 이유가 있다. 프리픽스는 라벨에서 **잘려나가므로**, `eq:`로 두면 관례대로 이름 붙인 `\label{eq:c}`를 `[eq:c]`로 가리킨 것이 `\eqref{c}`가 되어버린다. `[eq:eq:c]`라고 겹쳐 적어야 하는데 그럴 이유가 없어 `C:`/`F:`/`T:`와 같은 꼴로 두었다.
- `F:`와 `T:`는 memoir의 `\fref`/`\tref`를 흉내낸 것이다. 참조할 때마다 "그림", "표"를 손으로 적는 수고를 던다. `\figurename`/`\tablename`은 어느 클래스나 정의하고 kotex·babel이 언어에 맞게 바꿔주므로, 한국어 문서에서는 `cleveref`(`C:`)보다 오히려 손이 덜 간다. 이름과 번호 사이를 `~`로 묶어 그 자리에서 줄이 갈리지 않는다.
- `@t:`(`\citet`), `@p:`(`\citep`), `@a:`(`\citeauthor`), `@y:`(`\citeyear`)는 natbib·biblatex 계열 명령이다. 단행본에서는 본문 흐름에 저자를 끌어들이는 `\citet`과 괄호 안에 넣는 `\citep`의 구분이 사실상 필수라 기본값에 넣었다. 프리픽스는 쓰일 때만 발동하므로 그 패키지를 쓰지 않는 문서에는 아무 부담이 없다.
- **페이지 지정처럼 인자가 붙는 인용은 `{rawopt=…}`로 준다.** 닫는 `]` 바로 뒤에 붙인 `{rawopt="…"}`의 값이 **원문 그대로** 명령과 `{키}` 사이에 들어간다.

```
[@Allen04]{rawopt="[p.~50]"}          →  \cite[p.~50]{Allen04}
[@Allen04]{rawopt="[see][p.~23]"}     →  \cite[see][p.~23]{Allen04}
[@p:Allen04]{rawopt="[cf.][ch.~2]"}   →  \citep[cf.][ch.~2]{Allen04}
```

값은 따옴표로 감싸도 되고(따옴표 안에는 `]`나 `}`가 있어도 된다) 감싸지 않아도 된다(`{rawopt=[p.~7]}`). 값이 `[`로 시작하지 않으면 `[…]`로 자동으로 감싼다(`{rawopt="p.~3"}` → `\cite[p.~3]{key}`). `[`로 시작하는 값은 해석하지 않고 그대로 넣으므로 대괄호를 **직접 맞춰 적어야** 한다 — `"[see][p.~23"`처럼 닫는 대괄호를 빠뜨리면 `\cite[see][p.~23{Allen04}`가 그대로 나가 컴파일이 깨진다. 모든 프리픽스에 쓸 수 있고(`@t:`, `@p:` 등), 템플릿에 `{%s}`가 없는 사용자 정의 프리픽스에서는 경고하고 무시한다. `{rawopt=` 로 시작하지 않는 중괄호는 인용과 상관없는 본문으로 남는다. 본문에 `\citep[p.~33]{key}`라고 직접 써도 7장의 매크로 규칙으로 그대로 통과한다. - **`[프리픽스라벨]` 형태의 상호참조/인용 문법은 `cfg["refcite"]`(기본값은 위 13개 프리픽스)에서 프리픽스 → LaTeX 템플릿 매핑을 가져오며, 이 매핑은 `mapping.yaml`로 얼마든지 바꿔치기할 수 있다** — 예컨대 `\nameref` 대신 프로젝트에서 직접 정의한 `\hbref`를 쓰고 싶다면 아래처럼 `name:` 항목만 override 하면 되고, 나머지 기본 프리픽스는 그대로 유지된다(19장의 deep merge 참고). 새 프리픽스를 아예 추가하는 것도 가능하다(예: `"eq:": '\eqref{%s}'`). - `title:`과 `name:`은 결과적으로 둘 다 "제목 참조"를 가리키지만, 편집 관행상 별도 매크로 이름으로 구분해 쓰는 경우가 있어 기본적으로 둘 다 제공한다. `name:`은 hyperref가 제공하는 표준 `\nameref`를 쓰고, `title:`은 `\dotemph`와 마찬가지로 **`preamble`에 직접 정의해야 하는 매크로**(`\titleref`)를 가정한다. - `C:`(cleveref의 `\Cref`)는 `preamble`에 `\usepackage{cleveref}`가 필요하다. 여러 라벨을 한 번에 참조하려면 `[C:a,b,c]`처럼 콤마로 이어 쓰면 cleveref가 그대로 처리한다. `\crefrange{a}{b}`처럼 인자가 둘로 나뉘는 "범위" 참조는 이 대괄호 문법으로 지원하지 않는다 — 7장의 raw 매크로 통과 규칙 덕분에 본문에 `\crefrange{sec:a}{sec:b}`를 직접 써도 그대로 통과되므로 그렇게 쓰면 된다. - `C:`처럼 짧은 프리픽스는 원고 본문에 우연히 같은 모양의 텍스트(예: 윈도우 경로 `[C:\Users\...]`)가 있으면

오인식될 수 있다는 점은 감안할 것. 문제가 되면 바로 아래 예시처럼 mapping.yaml에서 더 안전한 프리픽스(예: "cref:")로 바꿔 쓸 수 있다.

```yaml
# mapping.yaml - name: 프리픽스만 project 전용 매크로로 교체하는 예
refcite:
  "name:": '\hbref{%s}'
```

색인 명령 교체 예시:

```yaml
# 명령 이름만 바꾸기
index_template: '%(text)s\myidx{%(term)s}'

# 표시 텍스트와 색인어를 인자 둘로 받는 매크로
index_template: '\myidx{%(text)s}{%(term)s}'
```

16.2. 각주의 종류 — notes와 꼬리표

기본은 모든 각주가 \footnote{…}다. memoir의 \sidefootnote, \pagenote, \footnoteA/\footnoteB처럼 **종류가 여럿**이어야 할 때(비평판 등)는 mapping의 notes에 종류를 등록하고, 본문의 각주 바로 뒤에 {이름} 꼬리표로 고른다.

```yaml
notes:                          # 기본 등록분(default, side, page, sidepar, sidebar, verb)에 더
  A: '\footnoteA%(opt)s{%(text)s}'
  B: '\footnoteB%(opt)s{%(text)s}'
  default: '\footnoteA%(opt)s{%(text)s}'    # 꼬리표 없는 각주의 종류를 바꾸려면
```

```
이문이 많다.^[갑본은 "仁"으로 적었다.]{A} 한편 …[^k1]{B}
```

꼬리표	뜻
{A}	notes에 등록한 이름. 인라인 ^[…]와 참조 [^id] 둘 다 받는다
{A, opt=3}	번호 지정: \footnoteA[3]{…} (opt="[3]"도 된다)
{sym}	\fnsymbol 기호 각주(*, †, ‡…). 번호는 문서 안 등장 순서대로 1, 2, 3…
{sym=3}	기호 번호를 직접 지정(이후 자동 번호는 4부터 이어짐). {sym=1}로 다시 시작
{A, sym}	종류와 함께: \footnoteA 계열에 같은 방식

- 템플릿에는 %(text)s(본문)와 %(opt)s([3] 또는 빈 문자열)를 쓴다. 예전 꼴 %s 하나만 써도 된다.
- **sym은 일반 번호 카운터를 건드리지 않는다.** 내부적으로 {\renewcommand{\thefootnote}{\fnsymbol{foo로 나가며, 번호를 지정한 \footnote[N]은 카운터를 올리지 않기 때문에 저장·복원 코드가 필요 없다. \fnsymbol은 기호가 9개뿐이라 10번째부터 경고한다. 보장하는 것은 \footnote 계열(\footnoteA 포함)뿐이다.
- 등록되지 않은 이름({sidee})은 꼬리표로 읽지 않고 **경고 후 본문에 그대로 남는다**(LaTeX에서는 조용히 사라지므로 오타를 알려 주는 것이다). 이 경고는 변환할 때 나온다 — --lint는

mapping을 읽지 않아 등록 여부를 알 수 없다. `{\bfseries x}`처럼 역슬래시가 든 것이나 `{}`는 꼬리표가 아니므로 경고 없이 예전처럼 본문에 남는다.

- `\footnoteA`는 프리앰블에 `\newfootnoteseries{A}`, `\pagenote`는 `\makepagenote`와 `\printpagenotes`가 있어야 한다. `preamble:`에 직접 쓴다.
- `\verb|…|`(와 `\verb*|…|`) 구간은 **본문 어디서나** 변환 없이 그대로 통과한다(0.19.3부터). `\verbfootnote{verb}`가 주된 쓰임이다.

---

17. 커스텀 환경 (`:::`)

```
:::{theorem}
정리 본문. `preamble`에 `\newtheorem{theorem}{정리}[section]`이
정의되어 있어야 한다.
:::

:::{quote}
사전 정의된 quote 환경.
:::

:::{center}
가운데 정렬.
:::

:::{tcolorbox title="주의" colback=yellow!10}
mapping.yaml 이나 attrs 를 이용한 커스텀 박스도 가능.
:::

:::{adjustwidth raw={1cm}{1cm}}
왼쪽·오른쪽 여백을 1cm씩 넓힌 본문. adjustwidth처럼 이름 없는(positional)
인자를 받는 환경에 쓴다.
:::
```

- **문법은 반드시 `:::{이름 속성}` 처럼 `:::` 바로 뒤에 공백 없이 `{`가 와야 인식된다.** `::: {이름}`처럼 공백이 있으면 매치되지 않고 일반 텍스트로 흘러 들어간다(현재 알려진 제약사항).
- **`:::{이름}[인자]`처럼 대괄호를 바로 붙이는 문법은 지원하지 않는다.** 인자가 필요하면 아래 `title=`/`raw=` 방식을 쓸 것.
- 닫는 줄은 `:::` 단독으로 한 줄.

- quote, center, table, figure, thebibliography, hverse, verse는 기본 제공 매핑이 있고, 그 외 이름은 전부 \begin{이름}...\end{이름}로 생성된다(그 환경이 실제 LaTeX에 정의돼 있어야 함 — \newtheorem이든 패키지 제공 환경이든).
- **등록되지 않은 환경에 속성을 주면 그 속성은 버려진다.** default 템플릿이 쓰는 것은 %(name)s와 %(raw)s뿐이기 때문이다. 이런 경우 경고를 찍는다 — :::{mybox title="주의"}처럼 써놓고 title이 왜 안 나오는지 몰라 헤매는 일을 막기 위해서다.
- 속성은 key=value, key="따옴표로 감싼 값" 형태로 parse_attrs가 파싱하며, mapping.yaml의 environments.<이름>.begin/end 템플릿 안에서 %(key)s로 참조할 수 있다.
- **따옴표로 감싼 값 안에 따옴표 자체를 넣고 싶으면 두 번 겹쳐 쓴다** (CSV/SQL 문자열 리터럴과 같은 관례). title="A --- ""B"""이면 값이 정확히 A --- "B"가 된다. \"(백슬래시)는 안 통한다 — 이 파서는 애초에 이스케이프라는 개념 자체가 없어서, \"를 만나도 그 앞의 백슬래시와 무관하게 바로 다음 "에서 값을 끊어버린다. title=처럼 인라인 처리를 거치는 속성이라면, 겹따옴표로 살아난 안쪽 "..."도 스마트 따옴표("...")로 정상 변환된다.
- **속성 사이 구분자는 쉼표를 권한다.** 15장 그림의 {width=..., label=...}, 4장 헤딩의 {unnumbered, label=...}가 모두 쉼표 구분이라 표기가 통일된다. 파서가 key=value 조각을 훑는 방식이라 공백 구분도 그대로 동작하지만, 한 문서 안에서는 한쪽으로 통일하는 편이 읽기 좋다. 따옴표 없는 값에서는 쉼표가 값에 딸려 들어가지 않도록 공백과 쉼표 앞까지만 읽는다. 값 자체에 쉼표가 필요하면 따옴표로 감싼다(raw="{1cm},{2cm}"). **속성 값은 이스케이프나 인라인 서식 처리를 거치지 않고 raw 그대로 템플릿에 꽂힌다** — 값에 LaTeX 특수문자(&, %, _ 등)가 있으면 직접 이스케이프해서 써야 한다.
- \begin{theorem}[피타고라스의 정리]처럼 **이름 있는(named) 대괄호 인자**가 필요하면, mapping.yaml에 해당 환경을 등록해두고 속성으로 값을 채운다(아래 예시 참고). 속성을 아예 안 주면 %(title)s를 요구하는 템플릿이 KeyError를 내는데, 이 경우 경고를 찍고 대괄호 없는 기본 \begin{theorem}...\end{theorem}로 자동 폴백한다.
- adjustwidth처럼 **이름 없는(positional) 인자를 여러 개 받는 환경**은 key=value 속성 문법과 안 맞는다(인자에 이름이 없으므로). 이런 경우를 위해 **raw라는 속성 이름을 예약어로 취급**한다: raw 값은 이스케이프 없이 그대로 \begin{이름} 바로 뒤에 붙는다(기본 환경 템플릿 자체가 \begin{%(name)s}%(raw)s이기 때문에, mapping.yaml에 아무것도 등록하지 않아도 바로 동작한다 — 아래 예시 참고). raw를 안 쓰는 환경에는 전혀 영향이 없다.
- raw 값에 공백이 들어가야 하면(예: \hspace{1cm} {1cm}) 따옴표로 감싸서 raw="\hspace{1cm} {1cm}"처럼 쓴다 — 따옴표 없는 값은 공백 전까지만 읽힌다.
- ::: 블록은 중첩 가능(depth 카운팅)하며, 내부는 다시 render_blocks로 재귀 처리되어 리스트·헤딩·표 등을 자유롭게 넣을 수 있다.
- **닫는 ::: 바로 다음 줄에 빈 줄 없이 글이 이어지면, 그 글은 새 문단으로 찍히지 않는다.** 재귀 처리를 위해 내부적으로는 블록이 갈리지만(그래야 안쪽 내용이 독립적으로 처리된다), 원문에 빈 줄이 없었다는 사실 자체가 "이건 새 문단이 아니다"라는 원고 작성자의 뜻이므로

줄바꿈만 하고 들여쓰기되는 새 문단으로는 만들지 않는다. 빈 줄을 실제로 넣으면 그때는 당연히 새 문단이 된다.

```
:::{quote}
텍스트
:::
```

라는 예처럼, 인용을 끝내고 바로 문장을 이어갈 수 있다.

```
\begin{quote}
텍스트
\end{quote}
```

라는 예처럼, 인용을 끝내고 바로 문장을 이어갈 수 있다.

- **리스트 항목 안에는 ::: 블록을 넣지 말 것.** :::/코드펜스처럼 빈 줄이 낀 다중 줄 구조를 리스트 항목 중간에 넣으면, 그 지점에서 블록이 끊겨 뒤따르는 리스트 항목들이 리스트에서 이탈해 통째로 한 문단으로 뭉쳐버린다(11장 리스트 항목 이어쓰기는 "빈 줄 없는 하나의 블록" 안에서만 성립하기 때문). 코드펜스 예시나 ::: 블록은 리스트 밖에 독립된 블록으로 두는 것이 안전하다. - **빈 줄 없는 코드펜스 하나 정도는 리스트 항목 안에 둘 수 있지만, 1단계 리스트까지만이다.** 코드펜스는 문서 맨 앞 기준 절대 들여쓰기가 0 3칸일 때만 펜스로 인식된다(CommonMark 규칙) — 1단계 리스트 항목의 들여쓰기 (2 3칸)는 이 범위 안이라 괜찮지만, 2단계 이상 중첩되면 들여쓰기가 필연적으로 4칸을 넘어서서 펜스 자체가 인식되지 않고 백틱이 그대로 리터럴 텍스트로 남는다. 깊이 중첩된 리스트 안에 코드를 보여줘야 한다면, 위 항목처럼 코드펜스를 리스트 밖 독립된 블록으로 빼는 것이 안전하다.

이름 있는 대괄호 인자 예시 (`title=` → `mapping.yaml`에 등록):

```yaml
# mapping.yaml
environments:
  theorem:
    begin: '\begin{theorem}[%(title)s]'
    end: '\end{theorem}'
```

17.1. 떠다니는 상자 — table과 figure

table과 figure는 기본 제공이라 mapping.yaml에 아무것도 등록하지 않아도 바로 동작한다.

```
:::{table place="htbp", caption="T/B/L/R 도우미로 그린 표", label="tab:1"}
\begin{tblr}{colspec={lcr}}
  Head A & Head B & Head C \\
  Data 1 & Data 2 & Data 3 \\
\end{tblr}
:::
```

```
\begin{table}[htbp]
\centering
\caption{T/B/L/R 도우미로 그린 표}
```

```
\label{tab:1}
\begin{tblr}{colspec={lcr}}
  Head A & Head B & Head C \
  Data 1 & Data 2 & Data 3 \
\end{tblr}
\end{table}
```

인식하는 속성은 넷이다.

속성	기본값	비고
`place`	`htbp`	`\begin{table}[...]`의 배치 옵션
`align`	`\centering`	값이 raw로 나가므로 LaTeX 명령을 적는다
`caption`	(없음)	없으면 `\caption` 줄 자체가 빠진다
`label`	(없음)	없으면 `\label` 줄 자체가 빠진다

`figure`도 같은 속성을 받지만 **캡션과 라벨이 내용 아래**에 놓인다. 표는 위, 그림은 아래가 조판 관례이기 때문이다. 직접 그린 TikZ 그림처럼 15장의 `![](...)` 문법으로 표현할 수 없는 것을 감쌀 때 쓴다.

기본값을 끄고 싶으면 빈 값을 명시한다. `:::{table align=""}`이면 `\centering` 줄이 사라진다.

내용 부분은 재귀적으로 처리되므로 14장의 마크다운 표를 넣어도 되고, 위 예시처럼 직접 짠 raw `tblr`을 넣어도 된다(8장 raw 환경 규칙으로 보호된다).

17.2. `thebibliography` — 원고에 직접 쓰는 문헌 목록

`.bib` 파일과 biber/bibtex를 쓴다면 이 절은 필요 없다(그쪽은 이 스크립트의 소관이 아니다). 문헌 목록을 원고에 직접 적어야 할 때 쓴다.

```
:::{thebibliography widest="99"}

\bibitem{ndh2024} Nova de Hi, 2024, *What is markdown?*

\bibitem{kitty2026} Kitty the Cat, *Please Make me Pleasant,* 2026

:::

\begin{thebibliography}{99}
\bibitem{ndh2024} Nova de Hi, 2024, \emph{What is markdown?}

\bibitem{kitty2026} Kitty the Cat, \emph{Please Make me Pleasant,} 2026
\end{thebibliography}
```

`widest`는 `\begin{thebibliography}{...}`의 인자, 즉 **가장 넓은 라벨**이다. 항목이 아홉 개 이하면 `9`, 백 개 미만이면 `99` 식으로 자릿수를 맞춘다. 생략하면 `99`가 쓰인다.

항목 자체는 `\bibitem`을 직접 쓴다. 마크다운 문법을 따로 만들지 않은 이유는 두 가지다. 첫째, `\bibitem` 뒤 본문에는 `*이탤릭*` 같은 인라인 서식이 그대로 먹으므로 손으로 쓰는 것은

`\bibitem{키}` 조각 하나뿐이다. 둘째, 목록의 번호를 citekey로 삼으면 목록을 한 번만 재정렬해도 모든 인용이 조용히 다른 문헌을 가리키게 된다. 번호는 LaTeX이 매기게 두고 키는 의미 있는 이름을 쓰는 편이 안전하다.

`\bibitem[표시 라벨]{키}` 형태도 그대로 통과한다(natbib의 저자-연도 표시).

17.3. 운문 — `hverse`와 `verse`

보통 환경의 내용은 본문과 똑같이 처리된다. 즉 빈 줄로 나뉘지 않은 여러 줄은 **한 문단으로 이어 붙는다.** 산문에서는 그것이 옳지만 시에서는 행이 곧 의미라서 원문이 망가진다.

`hverse`와 `verse`는 그래서 특례다. 원문의 **행 구조를 그대로** 넘긴다.

```
:::{hverse}

죽는 날까지 하늘을 우러러
한 점 부끄럼이 없기를,

별을 노래하는 마음으로
모든 죽어 가는 것을 사랑해야지

:::
```

```
\begin{hverse}
죽는 날까지 하늘을 우러러
한 점 부끄럼이 없기를,

별을 노래하는 마음으로
모든 죽어 가는 것을 사랑해야지
\end{hverse}
```

- 빈 줄은 **연(stanza) 구분**으로 그대로 남는다.
- `:::` 바로 다음과 바로 앞의 빈 줄은 떼어낸다. 원고에서 그 자리를 비워 두는 편이 읽기 좋은데, 그대로 나가면 빈 연이 생기기 때문이다.
- 인라인 서식과 매크로는 정상 동작한다. `**굵게**`, `값\은`, 각주 모두 된다. 행을 이어붙이지 않을 뿐, 각 행은 평소처럼 처리된다.
- 블록 문법(리스트, 표, 중첩 `:::`)은 이 안에서 동작하지 않는다. 행 파서를 아예 태우지 않기 때문이다.

`hverse`는 oblivoir-misc가 제공하며 행 끝에 `\\`를 요구하지 않는다. 표준 `verse`는 요구하므로 자동으로 붙여 준다. 연의 마지막 행에는 붙이지 않는다.

```
\begin{verse}
첫 행 \\
둘째 행
```

새 연 첫 행
\end{verse}

다른 환경에도 같은 특례를 줄 수 있다. `mapping.yaml`에서 `lines: keep`을 적으면 되고, 행 끝에 붙일 것이 있으면 `line_end`로 지정한다.

```yaml
environments:
  mypoem:
    begin: '\begin{mypoem}'
    end: '\end{mypoem}'
    lines: keep
    line_end: ' \\'
```

17.4. 완전히 raw — `lines: raw`

`lines: keep`도 각 행에 인라인 서식과 매크로 처리는 그대로 돌린다. 그런데 어떤 환경은 그 처리 자체가 방해가 된다 — 특히 raw LaTeX이나 외부 언어의 코드를 그대로 박아 넣는 환경이 그렇다.

TikZ가 대표적인 예다.

```
:::{tikzpicture raw="remember picture,overlay"}
\draw[very thick,->,>=latex] (NoteY-S)to[out=-90,in=90](NoteZ-N) ;
:::
```

`[out=-90,in=90](NoteZ-N)` 부분이 마크다운 링크 문법 `[표시](url)`와 우연히 같은 모양이라, 아무 지정도 없으면 `\href{NoteZ-N}{out=-90,in=90}`로 잘못 바뀐다. 그림 코드 안에는 `%`, `_`, `$`, `**`처럼 마크다운이 손대는 문자가 얼마든지 나올 수 있으므로, 이런 충돌은 TikZ에 국한되지 않는다.

`lines: raw`는 인라인 처리 자체를 건너뛰고 원문을 글자 그대로(들여쓰기·개행 포함) 붙인다.

```yaml
environments:
  tikzpicture:
    begin: '\begin{tikzpicture}[%(raw)s]'
    end: '\end{tikzpicture}'
    lines: raw
```

```
\begin{tikzpicture}[remember picture,overlay]
\draw[very thick,->,>=latex] (NoteY-S)to[out=-90,in=90](NoteZ-N) ;
\end{tikzpicture}
```

- 매크로도, 수식도, 각주도, 색인도, 인라인 서식도 **전혀 해석되지 않는다.** 정의상 "전부 raw"이기 때문이다.
- 이 보호는 해당 환경 **안에만** 적용된다. 환경 밖으로 나오면 문서의 나머지 부분은 평소대로 처리된다.
- `lines: keep`과는 정반대다. `keep`은 행마다 여전히 `**굵게**`나 조사 매크로 같은 것이 살아있길 원할 때(운문), `raw`는 그 무엇도 건드리지 않길 원할 때(그림 코드, 외부 언어) 쓴다.

- `%(raw)s` 자리표시자(7장 참고)와는 별개다. 그건 환경 여는 줄의 **속성 값**을 raw로 넘기는 것이고, `lines: raw`는 환경의 **본문 전체**를 raw로 다루는 것이다. 위 예시처럼 둘을 함께 쓸 수 있다 — `raw="..."` 속성으로 `\begin{tikzpicture}[...]`의 대괄호 내용을 정하고, `lines: raw`로 그 안의 그림 코드를 보호한다.

17.5. 선택적 속성 — 기본값과 `?` 줄

`%(key)s`는 그 속성을 **반드시** 요구한다. 빠지면 경고와 함께 템플릿 전체가 기본 `\begin{이름}`으로 폴백한다. 캡션이나 라벨처럼 있을 수도 없을 수도 있는 속성에는 이것으로 부족해서, 템플릿에 두 가지 표기를 더 쓸 수 있다.

표기	뜻
`%(key|기본값)s`	속성이 없으면 기본값을 대신 꽂는다
줄 앞의 `?`	그 줄이 참조하는 속성이 하나라도 비면 **줄을 통째로 버린다**

`?`는 줄을 남길 때 떼어낸다. 진짜 물음표로 시작하는 줄이 필요하면 `??`로 적는다. 기본값도 `?`도 없는 속성이 빠졌을 때의 폴백 동작은 그대로다.

기본값 판정 기준은 "값이 비었는가"가 아니라 **"속성이 주어졌는가"**다. 그래서 `align=""`처럼 빈 값을 명시하면 기본값이 적용되지 않고, 그 줄이 `?` 줄이라면 통째로 사라진다.

`end` 템플릿에서도 똑같이 쓸 수 있다. 17.1의 `figure`가 캡션을 아래에 놓는 것이 그 활용이다.

바로 위 `table` 기본값이 이 표기로 만들어져 있다.

```yaml
# 기본 제공되는 table 환경의 정의
environments:
  table:
    begin: |-
      \begin{table}[%(place|htbp)s]
      ?%(align|\centering)s
      ?\caption{%(caption)s}
      ?\label{%(label)s}
    end: '\end{table}'
```

속성을 생략하면 해당 줄만 사라진다.

- `:::{table caption="캡션만"}` — `\label` 줄이 빠지고, `place`는 기본값 `htbp`가 쓰인다.
- `:::{table}` — 아무 속성도 안 주면 `\begin{table}[htbp]`과 `\centering`만 남는다.

`align="center"`처럼 LaTeX 명령이 아닌 값을 쓰면 그대로 `center`라는 글자가 나가므로 주의할 것 — 이 스크립트는 속성 값을 해석하지 않는다.

다음처럼 쓰면

```
:::{theorem title="피타고라스의 정리"}
...
:::
```

이렇게 변환된다.

```
\begin{theorem}[피타고라스의 정리]
...
\end{theorem}
```

이름 없는(positional) 인자 예시 (`raw=` → 등록 없이 바로):

다음처럼 쓰면

```
:::{adjustwidth raw={1cm}{1cm}}
...
:::
```

이렇게 변환된다.

```
\begin{adjustwidth}{1cm}{1cm}
...
\end{adjustwidth}
```

---

18. 자동 이스케이프되는 특수문자

일반 텍스트에 그대로 쓴(즉 위의 어떤 보호 문법에도 걸리지 않은) 다음 문자는 자동으로 안전한 LaTeX 표현으로 바뀐다.

문자	출력
\	\textbackslash{}
&	\&
%	\%
#	\#
_	_
$	\$
{	\{
}	\}
^	\textasciicircum{}

즉 수식이나 매크로로 의도한 게 아니라면 `$`, `%`, `&`, `_` 등을 마크다운 본문에 그냥 타이핑해도 안전하다.

~(물결표)는 이 목록에 없다. ~도 \~도 이스케이프하지 않고 LaTeX으로 그대로 흘려보낸다 — 자세한 내용과 이유는 7.2를 볼 것.

18.1. ^^M — 소스 자체에 강제 개행 넣기

^^M이라고 쓰면 그 자리에 **생성되는 .tex 파일 자체의 진짜 개행 문자**가 들어간다.

```
\linecommand 여기부터 인자^^M다음 문장은 평범하게 이어진다.
```

```
\linecommand 여기부터 인자
다음 문장은 평범하게 이어진다.
```

- 이 방언은 평문의 줄바꿈을 전부 공백으로 합쳐 버리므로(한 문단은 결국 한 줄로 나간다), `\def\linecommand#1^^M{...}`처럼 **그 입력 줄의 끝까지를 인자로 삼는 TeX 매크로**를 쓸 방법이 원래 없었다. `^^M`은 정확히 그럴 때만 쓰는, 아주 드문 탈출구다. `^^M`이라는 표기 자체도 TeX이 개행 문자(캐리지리턴, 문자 코드 13)를 캐럿 표기법으로 나타낼 때 쓰는 바로 그 표기를 그대로 가져온 것이다.
- **이건 어디까지나 소스의 개행이지, LaTeX에게 "여기서 줄바꿈하라"는 뜻이 아니다.** 화면에 보이는 줄바꿈이 필요하면 `\\`나 `\newline`을 쓴다(6장). `^^M`은 순수하게 "TeX이 이 지점을 파일의 줄 끝으로 보게 하라"는 것뿐이다.
- **`^^`방점`^^`(18장이 아니라 정확히는 `extra_inline` 기본값의 방점 표시)과 표기가 비슷해 보이지만 충돌하지 않는다.** 방점은 반드시 닫는 `^^`가 있어야 매치되는데, `^^M`은 짝이 없는 캐럿 두 개+M이라 그 패턴에 안 걸린다. 이 처리 자체도 방점보다 먼저 끝나므로, 뒤쪽 어딘가에 진짜 방점(`^^`강조`^^`)이 있어도 `^^M`의 캐럿과 잘못 짝지어지는 일이 없다.
- 인라인 코드 스팬(`` `^^M` ``)이나 코드 펜스 안에서는 그대로 리터럴 텍스트로 남는다 — 예시로 보여줄 때 걱정 없이 그대로 쓰면 된다.

---

19. `mapping.yaml` 로 매핑 재정의

기본 설정(`DEFAULT_CONFIG`)에 대해 깊은 병합(deep merge)되므로, 바꾸고 싶은 키만 적으면 된다.

```yaml
# mapping.yaml
headings:
  1: part
  2: chapter
inline:
  - name: bold
    pattern: '\*\*(.+?)\*\*'
    template: '\textbf{%s}'
environments:
  warning:
    begin: '\begin{tcolorbox}[colback=red!5,title=주의]'
    end: '\end{tcolorbox}'
refcite:
  "name:": '\hbref{%s}'
codeblock_template: "\\begin{lstlisting}[language=%(lang)s]\n%(body)s\n\\end{lstlisting}"
hr: '\hrulefill'
showexam:
  ratio: 0.42
  style: lstlisting
```

```
  frame: none
```

front matter의 `mapping: mapping.yaml`로 지정하거나 CLI에서 `--config mapping.yaml`로 넘긴다(`--config`가 우선).

딕셔너리형 키는 항목만 추가하면 저절로 병합된다. `environments`, `refcite`가 그렇다 — 이들은 `{이름: 값, ...}` 형태의 딕셔너리이므로, 깊은 병합이 항목 단위로 일어난다. 위 예시의 `environments.warning`이 그 경우다: `warning` 하나만 적었을 뿐인데 `quote`, `table` 같은 기본 환경은 그대로 남아 있다. `extra_macros`, `extra_inline` 같은 "extra_" 접두사가 붙은 키가 따로 있는 이유는 정반대다 — `literal_macros`나 `inline`은 **리스트**라서 통째로 교체되므로, 항목만 보태고 싶을 때 기본값을 전부 베껴 오지 않도록 별도의 "추가 전용" 키를 마련해 둔 것이다. `environments`는 이미 딕셔너리라 그런 별도 키(`extra_environments` 같은)가 필요 없다.

19.1. 원고 안에 바로 — `mappingrule:`

바꿀 것이 한두 항목뿐이라면 파일을 따로 두는 것 자체가 아깝다. `mappingrule:`은 front matter 안에 YAML 조각을 직접 적으면 그 자리에서 병합해 준다.

```
---
title: 예시
class: article
mappingrule: |
  environments:
    warning:
      begin: '\begin{tcolorbox}[colback=red!5,title=주의]'
      end: '\end{tcolorbox}'
  extra_inline:
    - name: doubleline
      pattern: '\=\=(.+?)\=\='
      template: '\uuline{%s}'
  ellipsis: '\kswdashCdots{}'
  hr: '\fancybreak{* * *}'
---
```

- 다른 front matter 키와 같은 규칙을 따른다. `setup:`으로 상속되고, `mappingrule+:`로 상속받은 것에 덧붙일 수 있다(21장).
- 우선순위는 `mapping:` 파일 **위에** 얹히고, `--config`가 그 위에 다시 얹힌다 — `--config` > `mappingrule:` > `mapping:` 순이다. `--config`가 명령줄에서 준 것이라 가장 뚜렷한 의도로 본다는, `--config`가 `mapping:`을 이기는 것과 같은 원칙이다.
- 안의 YAML이 깨졌거나 `key: value` 형태가 아니면 오류로 멈춘다 — 매핑 설정이 조용히 무시되면 원인을 찾기 어렵기 때문이다.

`mapping.yaml` 파일 전체를 복사해 올 필요가 없다는 것이 요점이다. 바꿀 항목만 적으면 나머지 기본값은 그대로 남는다.

20. 파일 포함 (`{{filename}}`)

multimarkdown의 트랜스클루전 문법을 본떠, 다른 `.md` 파일의 본문을 그 자리에 끼워넣을 수 있다. 여러 챕터 파일을 하나의 최상위 문서에서 모아 단행본을 조판할 때 쓴다.

```
# 서문

머리말 본문.

{{chapter1.md}}

{{sub/chapter2.md}}

{{/absolute/path/appendix.md}}
```

- **포함되는 파일의 YAML front matter는 무시된다.** 본문만 그 자리에 삽입된다(포함되는 파일에 `title`, `mapping` 등을 적어놔도 최상위 문서 설정에 아무 영향을 주지 않는다).
- **확장자를 반드시 정확히 써야 한다.** `{{chapter1}}`처럼 확장자를 생략하면 `.md`를 자동으로 붙여주지 않는다 — `chapter1.md`가 실제로 존재해도, `chapter1`이라는 이름의 파일을 그대로 찾다가 없으면 그냥 오류로 처리된다.
- **경로 규칙**: `{{chapter1.md}}`처럼 경로 없이 파일 이름만 쓰면 **포함하는 파일(=최상위 입력 파일)과 같은 폴더** 기준으로 찾는다. `{{sub/chapter2.md}}` 처럼 상대 경로를 쓰면 마찬가지로 최상위 입력 파일의 폴더를 기준으로 결합하고, 절대 경로를 쓰면 그대로 사용한다.
- 파일을 찾지 못하면 변환을 중단하고, "포함 파일을 찾을 수 없습니다: `{{...}}` (확인한 경로: ...)" 형태의 오류를 stderr에 출력한 뒤 종료 코드 1로 끝난다.
- **재귀적으로 해석된다 — 포함된 파일 안에 또 `{{서브서브.md}}`가 있어도 (그 안에 또 있어도, ...) 계속 파고든다.** 상대 경로는 재귀 깊이와 무관하게 언제나 **최상위 입력 파일의 폴더** 기준이다 — 2단계, 3단계로 중첩된 파일 안에서 상대 경로로 또 다른 파일을 가리키고 싶어도, 그 경로는 자기 자신의 폴더가 아니라 최상위 파일 기준으로 풀어서 적어야 한다("포함 관계가 몇 단계든, 경로만은 언제나 한 기준점에서 잰다"는 일관된 태도다).
  - **순환 포함**(A가 B를, B가 다시 A를 포함하는 식 — 자기 자신을 직접 포함하는 것도 포함해서)은 즉시 오류로 멈추고, 무엇이 무엇을 순환 포함했는지 사슬 전체를 보여준다.
  - **너무 깊은 재귀는 피하는 것이 좋다.** 순환이 없어도 포함 사슬이 20단계를 넘으면 오류로 멈춘다 — 이건 순전히 안전장치일 뿐, 정상적인 원고가 이 깊이까지 갈 일은 없다. 경로 기준이 항상 최상위 파일 하나로 고정돼 있어서, 깊이 중첩할수록 "이 내용이 실제로 어느 파일에서 왔는지" 원고 작성자 스스로도 추적하기 어려워진다 — 2 3 단계 정도로 얕게 쓰는 것을 권장한다.

  - `--lint`와 `--syncinfo`도 이 재귀를 그대로 따라간다 — 몇 단계로 쪼개 놓았든 모든 서브파일이 빠짐없이 린트되고, PDF ↔ md 역방향 검색(1.5절)도 서브서브파일까지 정확한 줄로 안내한다. `ndhmain:` (16장)도 몇 단계에 있는 서브파일이든 똑같이 붙이면 된다 — "이 서브파일이 궁극적으로 어느 최상위 문서의 일부인지"만 알면 되므로, 중간 단계가 몇 겹이든 상관없다.

- 코드펜스(```) 안이나, 백틱에 바로 붙은 인라인 코드(`` `{{file.md}}` ``) 안의 `{{...}}`는 문법을 보여주는 예시로 간주되어 치환되지 않는다(9·10장의 raw 보호 규칙과 동일한 방식).
- **주석 `<!-- ... -->` 안의 `{{...}}`도 치환하지 않는다**(0.20.6). 포함을 잠시 꺼 두려고 `<!-- {{ch3.md}} -->`처럼 감싸면 그 파일은 읽지도 않으므로, 없는 파일이어도 오류가 나지 않고 `--lint`도 검사하지 않는다. 한 줄에서는 주석 바깥의 `{{...}}`만 치환된다. 이전 버전은 주석이 지워지기 전에 포함을 먼저 풀었기 때문에, 주석 속의 없는 파일이 변환을 멈추게 하거나 포함된 파일 속의 `-->`가 주석을 일찍 닫아 뒷부분을 본문으로 흘리는 버그가 있었다. 닫히지 않은 `<!--`는 주석으로 보지 않는다(변환기의 주석 규칙과 같다).
- **`{{file.md}}`는 앞뒤로 빈 줄을 두고 독립된 한 줄로 쓰는 것을 권장한다.** 포함되는 파일이 여러 문단·헤딩을 담고 있을 수 있는데, 지시자가 다른 문단 텍스트 중간에 섞여 있으면 블록 파싱이 예상과 다르게 동작할 수 있다.

20.1. 날 LaTeX 파일 그대로 끼워넣기 (`{{tex:filename}}`)

`{{tex:filename.tex}}`처럼 `tex:` 접두사를 붙이면, 그 파일을 마크다운으로 해석하지 않고 **한 글자도 바꾸지 않은 채** 그 자리에 그대로 끼워넣는다. 직접 짠 표지(title page)나 손으로 다듬은 LaTeX 조각을 그대로 문서에 박아 넣고 싶을 때 쓴다.

`{{tex:titlepage.tex}}`

titlepage.tex

```
% 서문

본문...
```

- 마크다운 처리를 **전혀** 거치지 않는다. `*`, `_`, `[표시](url)`, `{{또다른포함}}` 같은 이 방언의 문법이 그 파일 안에 있어도 전부 글자 그대로 남는다 — 강조도, 링크도, 중첩 포함도 되지 않는다.
- **YAML front matter라는 개념 자체가 없다.** 일반 `{{file.md}}`와 달리 앞부분을 헤더로 떼어내려 하지 않는다 — `.tex` 파일이 우연히 `---`로 시작해도 그 줄 그대로 삽입된다.
- **`\documentclass`, `\begin{document}`, `\end{document}` 같은 것이 파일 안에 있어도 스크립트는 아무것도 손대지 않는다.** 이런 것을 걷어내거나 검증하는 일은 하지 않는다 — 그 파일에 뭘 넣을지는 전적으로 문서 작성자의 책임이라는 것이 이 방언의 원칙이다. 결과가 컴파일되지 않는다면 그건 파일 내용의 문제이지 이 스크립트가 나서서 고쳐 줄 일이 아니다.
- 경로 규칙과 오류 처리는 20장의 일반 `{{file.md}}`와 동일하다(최상위 입력 파일 기준 상대경로, 절대경로 지원, 못 찾으면 오류로 중단).

- 재귀 포함을 지원하지 않는 것도 동일하다 — `tex:`로 끌어온 파일 안에 `{{...}}`가 있어도 해석되지 않고 그대로 남는다(애초에 마크다운으로도 안 보므로 당연하다).
- `--syncinfo`는 이 파일 전체를 "그 tex 파일 자신, 1번째 줄"로 근사해 기록한다 — raw LaTeX은 원본이 마크다운이 아니므로 "정확히 몇 번째 줄"이라는 대응 자체가 의미가 없기 때문이다. 역방향 검색을 하면 이 tex 파일 자체가 열린다.
- `--pandoc` 모드에서도 그대로 지원된다(pandoc의 raw-attribute 코드펜스로 감싸 전달하므로, pandoc 쪽에서도 마크다운으로 해석되지 않는다).

20.2. 여러 파일을 한 번에 (`{{파일패턴;물음표=목록}}`)

`sub01.md`부터 `sub20.md`처럼 번호만 다른 파일을 20줄 늘어놓지 않고 한 줄로 쓸 수 있다.

```
{{sub??.md;??=01-20}}
```

`{{sub01.md}}`부터 `{{sub20.md}}`까지를 빈 줄로 갈라 늘어놓은 것과 정확히 같다 — 트랜스클루전이 "앞뒤로 빈 줄을 둔 독립된 줄"일 때 가장 안전하게 인식되므로, 그 규칙을 그대로 지켜서 펼쳐진다.

콤마로 여러 조각을 늘어놓으면 불연속 범위도 된다.

```
{{sub??.md;??=01-10,12-20}}
```

`01`부터 `10`, 그리고 `12`부터 `20`까지만 쓰이고 `11`은 정확히 빠진다. 조각 하나는 "숫자" 하나(`7`)여도, "숫자-숫자" 범위(`01-10`)여도 된다 — 섞어서 `?=1-3,7,9-10`처럼 쓸 수도 있다. 조각이 하나뿐이면(콤마가 없으면) 그냥 기존 단순 범위와 완전히 같은 모양이므로, 콤마 지원 이전에 쓰던 `{{sub??.md;??=01-20}}` 표기는 손볼 필요 없이 그대로 동작한다. 조각은 적힌 순서 그대로 이어붙는다 — 정렬하거나 중복을 걸러내지 않으므로, 겹치는 범위를 일부러 적으면(`1-5,3-7`) 3, 4, 5가 두 번 나온다.

- **자리 수는 물음표 개수로 정한다.** `??`는 언제나 2자리로 0을 채운다 (01, 02, ..., 20). `?` 하나뿐이면 자리 채움 없이 그대로(1, 2, ..., 9). 자릿수가 넘어가도(예: `{{sub?.md;?=1-15}}`) 잘리지 않고 자연스럽게 1, 2, ..., 9, 10, ..., 15로 나온다 — 채움은 항상 "최소 자릿수"만 보장할 뿐, 그보다 큰 수를 억지로 줄이지 않는다. 이 폭은 목록 전체에 하나로 통일 적용된다.
- **`?`/`??` 자리는 숫자 전용이다.** 셸의 glob(`?`가 "아무 글자 하나"를 뜻하는 것)이나 정규식의 와일드카드와는 다르다 — 이건 어디까지나 정수(범위)를 자릿수만큼 0으로 채워 넣는 문법이지, 일반적인 패턴 매칭이 아니다. 문자 범위(`a-z`)나 그 밖의 와일드카드는 지원하지 않는다.
- 물음표가 파일 이름 패턴 안에 없거나(`{{sub##.md;??=01-20}}`처럼), 목록의 어느 조각이든 숫자·범위 형식이 아니거나, 범위의 시작이 끝보다 크면 오류로 멈춘다.
- `{{tex:sub??.tex;??=01-03}}`처럼 `tex:` 접두사와도 함께 쓸 수 있다 — 펼쳐진 각 이름(`tex:sub01.tex` 등)이 그대로 20.1절의 규칙을 따른다.
- `--lint`는 펼쳐진 이름 하나하나의 존재를 확인하고, 하위 파일 재귀 검사(19장 근처 `--lint` 설명 참고)에도 펼쳐진 파일들이 전부 들어간다.

- `--syncinfo`도 완전히 지원한다 — 펼쳐진 각 파일의 내용이 각자 정확한 원본 줄 번호로 추적된다. 파일과 파일 사이에 끼워 넣는 구분용 빈 줄만 정답이 없는 자리라 근사한다(대략 이 근처라는 뜻).
- 이 문법 자체는 재귀 확장이 아니다 — 펼쳐진 각 이름을 읽어들이는 방식은 평범한 단일 `{{파일}}`과 완전히 같다(20장의 다른 규칙들이 그대로 적용된다).

끝을 모를 때는 `*`(열린 범위)를 쓴다.

```
{{contents/sub??.md;??=01-*}}
```

`sub01.md`부터, 그 폴더에 실제로 있는 가장 큰 번호까지(예: `sub27.md` 까지 있으면 27까지) 자동으로 잡는다. 파일 개수가 자주 바뀌는 원고에서 매번 끝 번호를 손으로 고칠 필요가 없다.

- **중간에 빠진 번호는 조용히 건너뛴다.** `sub11.md`가 없으면 `sub10.md` 다음에 `sub12.md`가 자연스럽게 이어진다 — 디렉터리를 실제로 뒤져서 존재하는 파일만 모으기 때문이다.
- **이건 고정 범위(`01-20`)와는 다른 철학이다.** 고정 범위는 그 안의 파일이 하나라도 없으면 여전히 시끄러운 오류로 멈춘다(오타를 조용히 넘기지 않기 위해서다) — `*`를 쓴다는 것 자체가 "있는 것만 있는 대로 가져오라"는 명시적 의도이므로, 이 경우만 예외적으로 건너뛴다.
- `*`는 **끝자리에서만** 쓸 수 있다. 시작을 `*`로 하거나(`*-20`), 양쪽 다 `*`로 하는 것(`*-*`)은 뜻이 모호해서 지원하지 않는다.
- 찾는 범위는 **문서 기준 디렉터리**로 한정된다 — `setup:`이나 공유 리소스 폴더까지 뒤지는 일반 트랜스클루전의 확장 탐색은 여기 적용되지 않는다. `contents/sub??.md`처럼 하위 폴더를 적으면 그 폴더 안에서만 찾는다.
- 자리 수(물음표 개수)는 여기서도 그대로 "몇 자리 숫자만 인정할지"를 정한다 — `??`면 `sub100.md`(3자리)는 대상에서 빠진다.
- 조건에 맞는 파일을 하나도 못 찾으면(폴더 이름을 잘못 적었다거나) 오류로 멈춘다 — 열린 범위라고 해서 "아무것도 못 찾아도 조용히 넘어간다"는 뜻은 아니다.
- 콤마 목록의 한 조각으로도 쓸 수 있다(`?=1-3,7-*`). `tex:` 접두사와도 함께 쓸 수 있다.

---

21. 마스터 헤더 상속 (`setup:`)

LaTeX 문서용 front matter는 금세 길어지고, 같은 판형으로 여러 문서를 만들면 그 20여 줄이 파일마다 복제된다. `setup:` 키는 그 공통부를 다른 파일에 빼두고 물려받게 해준다.

```
---
setup: mymaster.md
title: 이 문서만의 제목
---
```

이는 20장의 `{{filename}}` 포함과 **정확히 대칭**이다.

문법	가져오는 것	버리는 것
`{{file.md}}`	그 파일의 **본문**	front matter
`setup: file.md`	그 파일의 **front matter**	본문

따라서 setup 파일은 두 가지 형태 모두 가능하다.

- **완성된 마크다운 문서**: `---`로 감싼 front matter가 있으면 그것만 읽고 본문은 통째로 버린다. 즉 **이미 조판을 마친 기존 문서를 그대로 스타일 도너로 지정할 수 있다** — "지난번 그 책과 같은 판형으로"가 한 줄로 끝난다.
- **헤더만 적힌 파일**: `---` 구분선이 없으면 파일 전체를 YAML로 해석한다. 확장자는 무엇이든 상관없다(`.yaml`, `.txt` 등).

21.1. 병합 규칙 — 로컬이 이긴다

같은 키가 양쪽에 있으면 **현재 파일의 값이 setup 파일의 값을 대체한다.**

```yaml
# base.yaml
class: oblivoir
classoptions: 11pt, a4paper
author: Nova de Hi
preamble: |
  \usepackage{kotex}
---
setup: base.yaml
class: memoir          ← oblivoir 를 대체한다
---
```

21.2. 이어붙이려면 키+:

대체가 아니라 **덧붙이고** 싶으면 키 이름 뒤에 +를 붙인다. 이것이 실제로 가장 많이 쓰게 될 문법이다 — 공용 프리앰블은 그대로 두고 이 문서에만 패키지 하나를 더 얹는 경우.

```yaml
---
setup: base.yaml
preamble+: |
  \usepackage{tikz}
classoptions+: twoside
---
```

```
\documentclass[11pt, a4paper, twoside]{oblivoir}
\usepackage{kotex}
\usepackage{tikz}
```

- setup 파일의 값이 먼저 오고, 현재 파일의 값이 뒤에 붙는다. 이 순서 덕분에 개별 정의를 덮어쓰는 일은 LaTeX 자신의 `\renewcommand`가 해결해준다 — YAML 층에서 흉내낼 필요가 없다.
- 구분자는 `classoption`/`classoptions`가 `,`+공백, 나머지는 개행이다.
- 한 파일에 `preamble:`과 `preamble+:`이 **동시에** 있으면 대체를 먼저 적용하고 그 결과에 이어붙인다.
- setup이 아예 없는 문서에서 `키+:`를 쓰면 그냥 평범한 `키:`처럼 동작한다.
- 프리앰블에서 같은 패키지가 중복 로딩되어도 LaTeX의 `\usepackage`가 알아서 한 번만 처리하므로 대개 문제되지 않는다.

21.3. 여러 파일을 나열하기

`setup:`은 파일 하나뿐 아니라 **리스트**도 받는다. 판형은 공용 헤더에서 가져오고, 그 위에 문서 종류별 조각(용어집용 색인 설정, 발표자료용 여백 등)을 따로 얹고 싶을 때 쓴다.

```
---
setup: [base.yaml, glossary.yaml]
title: 이 문서만의 제목
---
```

- 왼쪽부터 순서대로 병합된다. **뒤에 적은 파일이 앞의 것을 덮어쓴다** — front matter 안의 일반적인 override 규칙(21.1)과 같다.
- `키+:`는 리스트를 가로질러서도 계속 이어붙는다. `glossary.yaml`에 `preamble+:`가 있으면 `base.yaml`의 `preamble:` 뒤에 이어지고, 그 뒤에 다시 최상위 문서 자신의 `preamble+:`가 있으면 거기에도 이어진다.
- 리스트 안의 각 파일이 **자기 자신의** `setup:`을 또 가질 수 있다(21.6의 체인 규칙이 그대로 적용된다). 그 경우 "지금까지 누적된 것" 위에 자신의 조상을 먼저 얹은 뒤, 자기 자신의 내용을 올린다 — 즉 리스트 안의 형제 관계도 상속 관계의 일부로 취급된다.
- **같은 파일을 리스트에 두 번 넣으면 순환 참조로 막는다.** `setup: [a.md, a.md]`는 의미가 없고, 대개 오타다.
- CLI에서는 `--setup base.yaml,glossary.yaml`처럼 **콤마**로 나열한다(공백이 아니다 — 입력 파일 인자와 헷갈리지 않도록). front matter의 `setup:`을 통째로 갈아치우는 것은 단일 파일일 때와 같다.

21.4. `title`은 상속되지 않는다

`title`은 문서의 **정체성**이지 스타일이 아니므로 setup 파일에서 물려받지 않는다. 완성된 기존 문서를 스타일 도너로 지정했을 때 그 문서의 제목이 조용히 딸려오면, 에러가 아니라 엉뚱한 제목의 *PDF*가 나오기 때문이다. `author`, `date`, `class` 등 나머지 키는 모두 정상적으로 상속된다.

21.5. mapping:은 언제나 최상위 파일 기준

유효한 매핑 설정은 **어떤 경우에도 정확히 하나**뿐이며, 여러 개가 병합되는 일은 없다. 우선순위는 다음과 같다.

```
--config (CLI)  >  현재 파일의 mapping:  >  setup 파일의 mapping:
```

setup 파일에서 상속된 경우라도 **상대경로는 언제나 최상위 입력 파일의 디렉터리 기준으로 풀린다** — setup 파일이 있는 디렉터리 기준이 아니다. 공용 폴더에 둔 setup 파일에서 그 옆의 매핑 파일을 가리키려면 절대경로를 쓸 것. 같은 이유로 `mapping+:`는 지원하지 않으며, 쓰면 경고를 찍고 무시한다.

21.6. 체인은 2단계까지

setup 파일이 또 자신의 `setup:`을 가질 수 있다. 이때 그 경로는 **그 키를 적은 파일**의 디렉터리 기준이다(`mapping:`과 다르다).

```
doc.md      →  series.md     →  base.yaml
(작업 파일)     (시리즈 공통)     (공용 기본)
```

이 3단 구성이 허용되는 최대 형태이고, 그보다 깊어지면 오류로 중단한다. 헤더를 어디서 물려받았는지 추적할 수 없게 되는 것을 막기 위한 의도적인 제한이다. 순환 참조(`a.md` → `b.md` → `a.md`)도 오류로 잡는다.

setup 파일의 `setup:`을 조용히 무시하지 않고 굳이 2단계까지 허용한 이유는, 완성된 문서를 스타일 도너로 지정했을 때 그 문서가 스스로 공용 헤더 위에 지어져 있는 경우가 흔하기 때문이다. 무시해버리면 **스타일의 절반만 딸려온 채 에러 없이 잘못된 출력**이 나온다.

21.7. 출처 확인하기

값의 출처가 여러 곳으로 나뉘므로 `--dump-meta`로 병합 결과를 확인할 수 있다.

```
$ python3 ndhmd4tex.py doc.md --dump-meta
# setup 체인: doc.md -> shared/series.md -> shared/base.yaml
author: Nova de Hi
class: oblivoir
classoptions: 11pt, a4paper, twoside
preamble: '\usepackage{kotex}

  \usepackage{tabularray}'
title: 내 문서
```

setup 파일을 찾지 못하거나, 체인이 너무 깊거나, 순환 참조가 있으면 stderr에 오류를 출력하고 종료 코드 1로 끝난다(20장의 포함 파일 오류와 같은 방식).

또한 `{{}}`로 포함되는 파일의 front matter는 통째로 무시되므로, 포함되는 파일에 `setup:`을 적어놔도 아무 영향이 없다.

22. 알려진 제약 / 주의사항

- 예전 표기의 설명 리스트(**용어** `::` **설명**)는 **그는 말했다** `::` **아니다**처럼 산문에 같은 형태가 나오면 구분할 방법이 없다. 중괄호형(`{`**용어**`}` `::` **설명**)을 쓰거나 `description_syntax: braces`로 막으면 된다(11장 참고).
- `:::{`**이름**`}` 커스텀 환경은 `:::`와 `{` 사이에 **공백이 있으면 인식되지 않는다**.
- 인용구(`>`) 안에 리스트·헤딩 등 별개 블록을 넣으려면 `>` 단독 줄로 빈 줄을 표시해야 한다. 그냥 죽 이어 쓰면 전부 한 문단으로 합쳐진다.
- `` `\begin{...}...\end{...}` `` 처럼 raw LaTeX 문법을 예시로 보여주려고 인라인 코드 백틱으로 감쌀 때, **백틱이 `\begin`/`\end` 바로 옆에 붙어 있어야** 안전하게 처리된다. 백틱과 내용 사이에 패딩 스페이스를 넣으면 (`` ` \begin{...}...\end{...} ` ``처럼) 여전히 진짜 raw 환경으로 오인식될 수 있다.
- 같은 이유로, 인라인 코드로 `<!-- -->` 문법 자체를 예시로 보여줄 때도 백틱이 바로 옆에 붙어 있어야 안전하다(패딩 스페이스가 있으면 진짜 주석으로 오인식되어 지워질 수 있다).
- 표는 헤더 행의 열 개수를 기준으로 고정 폭 처리한다.
- raw로 보호되는 블록(코드펜스/디스플레이수식/raw 환경)은 그 **안에서** 이 방언 문법(굵게, 따옴표 변환 등)이 전혀 적용되지 않는다 — 이미 완성된 LaTeX이라고 가정하기 때문이다.
- 리스트 항목 안에 **문단·코드 블록 등 블록 요소를 넣을 수 없다**. 빈 줄로 구분된 둘째 문단은 리스트를 닫고 밖으로 빠져나온다(11장 참고). 중첩 리스트만이 항목 안에 들어갈 수 있는 유일한 블록이다.
- `{{filename}}` 파일 포함은 코드 펜스와 인라인 코드 스팬 **안에서는 해석되지 않는다**(리터럴로 남는다). 그래서 `` `{{ user.name }}` ``처럼 다른 템플릿 문법을 예시로 보여주거나, 7.1의 raw 스팬에 `{{`가 들어가도 안전하다.
- `{{filename}}` 파일 포함은 **1단계만** 지원한다. 포함된 파일 안에 또 `{{...}}`가 있어도 재귀적으로 해석되지 않고 리터럴 텍스트로 남는다(20장 참고).
- `setup:` 체인은 **2단계까지만** 허용된다(즉 **작업 파일** → `setup` → `setup`이 최대). 더 깊어지면 오류다(21.6 참고).
- `setup:`으로 상속되는 `mapping:`의 상대경로는 setup 파일이 아니라 **언제나 최상위 입력 파일**의 디렉터리 기준으로 풀린다. 공용 폴더에 둔 setup 파일에서 매핑 파일을 가리킬 때는 절대경로를 쓰는 것이 안전하다(21.5 참고).
- `title`은 `setup:`으로 상속되지 않는다(21.4 참고). 반대로 `author`, `date`는 상속되므로, 완성된 문서를 스타일 도너로 쓸 때 저자가 다르면 현재 파일에서 명시적으로 덮어써야 한다.
- `showexam` 상자는 **페이지를 넘어 쪼개지지 않는다**. 예제가 길면 통째로 다음 페이지로 밀린다. `tcolorbox`로 바꿔도 `breakable`과 `sidebyside`는 양립하지 않으므로 해결되지 않는다(10.2 참고).

- 리스트 항목이나 인용구 안에 들여쓴 showexam 블록은 지원하지 않는다 — 최상위에서만 쓴다.
- showexam의 style: verbatim은 긴 줄을 접지 못해 상자 밖으로 넘친다. 줄바꿈이 필요하면 minted나 lstlisting을 쓴다. 한글 등 유니코드가 섞이면 lstlisting은 감당하지 못하므로 minted가 사실상 유일한 선택이다(10장의 -shell-escape 관련 설명 참고).

---

23. Beamer 프레임 (=\{4,\}\{속성\})

class: beamer로 발표 자료를 만들 때, 프레임 하나하나를 \begin{frame}/ \end{frame}로 손수 감싸지 않아도 되게 하는 문법이다. :::{환경}(10장)과 정확히 같은 관례 — 여는 쪽만 중괄호가 있고 닫는 쪽은 맨몸 — 를 등호에 적용했을 뿐이다.

```
======{title="프레임에 제목을 달자",subtitle="작은 제목",opt=fragile}
본문.
======
```

```
\begin{frame}[fragile]
\frametitle{프레임에 제목을 달자}
\framesubtitle{작은 제목}

본문.
\end{frame}
```

- **여는 줄은 등호 4개 이상 + 중괄호 속성**(=\{4,\}\{...\}), **닫는 줄은 등호 4개 이상뿐**(중괄호 없이). 4개를 최소로 둔 것은 ---/***가 이미 수평선으로 예약된 것과 같은 논리다.
- 속성 문법은 :::{환경}과 완전히 같은 key="값"/key=값(쉼표로 나열) 이다 — 새 파싱 규칙을 또 만들지 않았다.
- **name=을 주면 frame 대신 그 이름의 환경을 쓴다.** 값이 인라인 처리 없이(라텍 명령 이름 자리이지 화면 텍스트가 아니므로) 가공 없이 그대로 \begin{...}/\end{...} 양쪽에 쓰인다. ltx-talk 같은 Beamer 확장 패키지가 제공하는 frame*, itemizedframe처럼 "frame과 비슷하게 동작하되 이름이 다른" 환경을 쓸 때 필요하다 — title/subtitle/opt/raw 처리는 환경 이름과 완전히 무관하게 그대로다.

```
====={name="frame*",title="TITLE"}
본문.
====

\begin{frame*}
\frametitle{TITLE}

본문.
\end{frame*}
```

- `title=`/`subtitle=`은 각각 `\frametitle{}`/`\framesubtitle{}`이 되고 (값은 인라인 처리를 거친다), `opt=`은 `\begin{환경}[...]`의 대괄호 안에 가공 없이 그대로 들어간다. `raw=`는 `\begin{환경}[opt]` 바로 뒤에(같은 줄에) 가공 없이 그대로 이어붙는다(`opt`가 없으면 대괄호 없이 `\begin{환경}` 뒤에 바로 붙는다) — `:::{환경}`의 `raw` 속성과 같은 예약어 취급이다. 넷 다 안 쓰면 대응하는 줄/조각 자체가 아예 안 나온다.
- **대괄호나 중괄호가 두 개 이상 필요하면 `raw=`에 마저 이어붙인다.** `opt=`은 오직 **첫 번째** `[...]`만 맡는다. Beamer는 `\begin{frame}[<+->][fragile]`처럼 대괄호를 두 개(오버레이 기본값 + 옵션) 쓰기도 하는데, 새 속성을 또 만들 필요 없이 두 번째 대괄호를 `raw=`에 문자열째로 넣으면 그대로 이어붙는다. 예시는 아래를 볼 것.

```
======{opt=<+->,raw="[fragile]",title="Itemize"}

\begin{frame}[<+->][fragile]
\frametitle{Itemize}
```

실측(xelatex + beamer)으로 `[<+->]`가 프레임 전체의 기본 오버레이로 적용돼, 리스트 항목이 `\pause` 없이도 한 단계씩 순차 표시되는 것을 확인했다. 대괄호든 중괄호든, 몇 개가 더 필요하든 전부 `raw=` 문자열 하나에 순서대로 이어 적으면 된다 — `raw="[fragile]{추가 제목}"`처럼.

- **프레임 안은 완전히 평범한 본문이다.** 리스트, 설명 리스트, 표, 인용, 중첩된 `:::{환경}` 등 이 방언이 지원하는 모든 블록 문법이 바깥과 똑같이 그대로 동작한다 — "프레임 전용 파서"가 따로 있는 게 아니라, `:::{환경}`과 마찬가지로 안쪽 내용을 그대로 재귀 처리하기 때문이다. 그래서 프레임 여는 줄 바로 다음에 빈 줄이 없어도(원고에서 시각적으로 등호와 본문을 붙여 써도) 문제없다 — 재귀 호출이 그 첫 줄부터 새 블록으로 인식한다.
- **설명 리스트와 조합하면 Beamer의 오버레이 지정 문법을 쓸 수 있다.** 설명 리스트도 첫 항목의 설명에 `{!옵션}` 블록을 쓸 수 있으므로(11장), `\begin{description}[<+-| alert@+>]`처럼 항목이 단계별로 순차 강조되게 만들 수 있다. 예시는 아래를 볼 것.

```
{uncover} ::  {!"<+-| alert@+>"} 주어진 슬라이드에서 visible.
{only} ::  주어진 슬라이드에서 show.

\begin{description}[<+-| alert@+>]
\item[uncover] 주어진 슬라이드에서 visible.
\item[only] 주어진 슬라이드에서 show.
\end{description}
```

실측(xelatex + beamer 컴파일)으로 각 오버레이 단계에서 정확히 그 항목만 `\alert`로 강조되는 것을 확인했다.

- `--lint`는 짝이 안 맞는 프레임(닫히지 않음, 열린 적 없이 닫음)과 `====`와 `{` 사이의 공백(공백이 있으면 프레임으로 인식되지 않는다)을 잡아낸다.
- `--syncinfo`는 프레임 전체를 하나의 블록으로 추적한다 — 역방향 검색을 하면 그 프레임이 시작하는 원본 위치로는 정확히 가지만, 프레임 안의 특정 줄까지 정밀하게 짚어주지는 않는다(`:::{환경}`과 같은 한계).

- **`class: beamer`를 위한 별도의 기본 설정은 없다.** 이 방언의 기본 `classoptions`(예: `chapter`)는 book/article류를 염두에 둔 것이라 beamer에는 안 맞는 옵션이 섞여 있을 수 있다 — beamer로 조판할 때는 `classoptions: ""`처럼 front matter에서 직접 정리해 주는 것이 안전하다.